ALPHA MICROSYSTEMS
RIGHT. FROM THE START.

AlphaBASIC®
User's Manual

DSS-10073-00, Rev. A05

© 1998 Alpha Microsystems

REVISIONS INCORPORATED	
REVISION	DATE
00	May 1986
01	May 1986
02	December 1988
03	September 1996
04	June 1998
05	April 1999

AlphaBASIC User's Manual
To re-order this document, request part number DSS-10073-00.

This document applies to AMOS 2.3 and later.

The information contained in this manual is believed to be accurate and reliable. However, no responsibility for the accuracy, completeness or use of this information is assumed by Alpha Microsystems.

This document may contain references to products covered under U.S. Patent Number 4,530,048.

The following are registered trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AMIGOS	AMOS	Alpha Micro	AlphaACCOUNTING
AlphaBASIC	AlphaCALC	AlphaCOBOL	AlphaDDE
AlphaFORTRAN 77	AlphaLAN	AlphaLEDGER	AlphaMAIL
AlphaMATE	AlphaNET	AlphaPASCAL	AlphaRJE
AlphaWRITE	CASELODE	OmniBASIC	VIDEOTRAX

The following are trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AlphaBASIC PLUS	AlphaVUE	AM-PC	AMTEC
AlphaDDE	AlphaConnect	DART	*in*Sight/am
*in*Front/am	ESP	MULTI	

All other copyrights and trademarks are the property of their respective holders.

ALPHA MICROSYSTEMS
2722 S. Fairview St.
P.O. Box 25059
Santa Ana, CA 92799

Table of Contents

CHAPTER 1 - INTRODUCTION TO ALPHABASIC 1-1

WHO IS THIS MANUAL WRITTEN FOR? 1-1
WHAT IS BASIC? 1-1
HOW IS AlphaBASIC SPECIAL? 1-2
WHAT CAN AlphaBASIC DO? 1-2
WHAT TYPES OF DATA DOES AlphaBASIC SUPPORT? 1-3
THE AlphaBASIC SYSTEM 1-4
WHAT DOES AN AlphaBASIC PROGRAM LOOK LIKE? 1-4
GRAPHICS CONVENTIONS 1-5

CHAPTER 2 - GENERAL INFORMATION 2-1

MULTIPLE STATEMENT LINES 2-1
CONTINUATION LINES 2-1
LINE NUMBERS 2-2
COMMENTS (REM AND !) 2-2
LABELS 2-3
MEMORY ALLOCATION 2-4
EXPAND AND NOEXPAND MODES 2-4
CASE OF CHARACTERS 2-5
SEARCHING FOR A LIBRARY 2-5
HOW TO INCLUDE ONE AlphaBASIC PROGRAM INSIDE ANOTHER 2-6

CHAPTER 3 - ALPHABASIC INTERACTIVE MODE 3-1

WHAT IS INTERACTIVE MODE? 3-1
CLEARING MEMORY FOR A NEW PROGRAM (NEW) 3-2
CREATING A PROGRAM 3-2
SAVING A PROGRAM (SAVE) 3-3
LISTING A PROGRAM (LIST) 3-4
COMPILING A PROGRAM 3-5
Compiler Option 3-6
RUNNING A PROGRAM (RUN) 3-6
EXITING FROM INTERACTIVE MODE (BYE) 3-7
DIRECT STATEMENTS 3-7
A HINT FOR THE SYSTEM OPERATOR 3-8

CHAPTER 4 - MORE INTERACTIVE COMMANDS 4-1

INTERRUPTING A PROGRAM RUN (BREAK) 4-1
CONTINUING AN INTERRUPTED PROGRAM (CONT) 4-2
USING CONTROL-C TO INTERRUPT A PROGRAM RUN 4-3
DELETING PROGRAM LINES (DELETE) 4-3
LOADING A PROGRAM INTO MEMORY (LOAD) 4-4
THE SINGLE-STEP FEATURE 4-5

CHAPTER 5 - WRITING ALPHABASIC PROGRAMS USING ALPHAVUE 5-1

CREATING A PROGRAM 5-1
PROGRAM FORM 5-1
COMPILING A PROGRAM 5-2
Compiler Options 5-4
RUNNING A PROGRAM 5-4

CHAPTER 6 - ALPHABASIC VARIABLES 6-1

VARIABLE NAMES 6-1
NUMERIC VARIABLES 6-2
STRING VARIABLES 6-2
ARRAY VARIABLES 6-3

CHAPTER 7 - ALPHABASIC EXPRESSIONS 7-1

WHAT IS AN EXPRESSION? 7-1
MATHEMATICAL EXPRESSIONS 7-1
LOGICAL OPERATORS 7-2
OPERATOR PRECEDENCE 7-3
MODE INDEPENDENCE 7-3

CHAPTER 8 - WORKING WITH STRING VARIABLES (SUBSTRING MODIFIERS) 8-1

WHAT IS A SUBSTRING MODIFIER? 8-1
HOW DO YOU USE A SUBSTRING MODIFIER? 8-1
EXAMPLES OF SUBSTRING MODIFIERS 8-2

CHAPTER 9 - DATA FORMATS 9-1

FLOATING POINT FORMAT 9-1
STRING FORMAT 9-1
BINARY FORMAT 9-2
INTEGER FORMAT 9-2
UNFORMATTED 9-3

CHAPTER 10 - ALPHABASIC PROGRAM STATEMENTS 10-1

CALL 10-1
CHAIN 10-1
DATA 10-2
DIM 10-2
ELSE 10-3
END 10-3
FOR, TO, NEXT AND STEP 10-3
GOSUB (OR CALL) AND RETURN 10-5
GOTO 10-7
IF, THEN AND ELSE 10-8
INPUT 10-9
INPUT LINE 10-12
LET 10-13
NEXT 10-13
ON ERROR AND RESUME 10-13
ON - GOSUB (CALL) 10-14
ON - GOTO 10-14
PRINT 10-15
PRINT USING 10-17
PROGRAM 10-17
DISPLAYING VERSION INFORMATION: 10-17
RANDOMIZE 10-18
READ, RESTORE, AND DATA 10-18
RESTORE 10-19
RESUME 10-19
RETURN 10-19
SCALE 10-19
SIGNIFICANCE 10-19
STEP 10-20
STOP 10-20
STRSIZ 10-20
THEN 10-21
TO 10-21
USING 10-21
XCALL 10-21

CHAPTER 11 - ALPHABASIC FUNCTIONS 11-1

NUMERIC FUNCTIONS 11-1
Abs(X) 11-2
Asc(A) 11-2
Exp(X) 11-2
Fact(X) 11-2
Fix(X) 11-2
Int(X) 11-3
Log(X) 11-3

Log10(X) 11-3
Rnd(X) 11-3
Sgn(X) 11-3
Sqr(X) 11-4
Val(A$) 11-4
TRIGONOMETRIC FUNCTIONS 11-4
CONTROL FUNCTIONS 11-4
Err(X) 11-4
STRING FUNCTIONS 11-4
Asc(X) 11-5
Chr(X) 11-5
Instr(X,A$,B$) 11-5
Lcs(A$) 11-6
Left(A$,X) 11-6
Len(A$) 11-6
Mid(A$,X,Y) 11-7
Right(A$,X) 11-7
Space(X) 11-7
Str(X) 11-8
Ucs(A$) 11-8

CHAPTER 12 - SYSTEM FUNCTIONS 12-1

BYTE(X) AND WORD(X) 12-1
DATE 12-2
IO(X) 12-2
MEM(X) 12-2
TIME 12-3

CHAPTER 13 - FORMATTING OUTPUT (PRINT USING AND EXPANDED TABS) 13-1

THE USING MODIFIER 13-1
HOW TO SPECIFY FORMAT STRINGS 13-3
Formatting a Numeric Field (#) 13-3
Formatting a String Field (\) 13-4
One-Character String Fields (!) 13-5
Using Decimal Points in Numeric Fields (.) 13-5
Dollar Signs and Numeric Fields ($$) 13-6
Putting a Comma Every Three Digits (,) 13-7
Fill Leading Blanks With Asterisks (**) 13-7
Fill Leading Blanks With Zeros (Z) 13-7
Add a Trailing Minus Sign to a Number (-) 13-8
Printing Numbers as Exponents (^^^^) 13-8
FORMATTING EXAMPLES AND HINTS 13-8
EXPANDED TAB FUNCTIONS 13-9
Special Screen Handling Codes 13-9

CHAPTER 14 - SCALED ARITHMETIC 14-1

THE SCALE STATEMENT 14-1
HOW THE SCALING FACTOR WORKS 14-2

CHAPTER 15 - MAPPING VARIABLES 15-1

WHAT IS VARIABLE MAPPING? 15-1
THE FORMAT OF MAP STATEMENTS 15-2
MAP Level 15-3
Variable Names in MAP Statements 15-4
Type Code 15-4
Size 15-5
Value 15-5
Origin 15-6
USING MAP STATEMENTS 15-7
Examples 15-8
USING MAPPED DATA WITH DATA FILES 15-9
HOW VARIABLES ARE ALLOCATED IN MEMORY 15-10
LOCATING VARIABLES DURING DEBUGGING 15-11
Examples 15-12

CHAPTER 16 - ALPHABASIC FILE INPUT/OUTPUT SYSTEM 16-1

WHAT IS A DISK FILE? 16-1
FILE CHANNELS 16-2
INPUT/OUTPUT STATEMENTS 16-2
SEQUENTIAL FILES 16-3
Using Sequential Files for Output 16-3
Using Sequential Files for Input 16-4
Using Sequential Files in Append Mode 16-4
RANDOM FILES 16-4
Logical Records 16-5
Figuring Out How Many Blocks Your File Needs 16-5
Accessing Random Files 16-6
USING FILE LOCKING (LOKSER) 16-7
File Locking for Sequential Files 16-8
File Locking for Random Files 16-8
FILE STATEMENTS 16-9
ALLOCATE 16-9
CLOSE 16-10
EOF(X) 16-10
FILEBASE 16-10
INPUT 16-11
INPUT LINE 16-12
KILL 16-13
LOOKUP 16-13
OPEN 16-14

PRINT 16-16
READ 16-16
READL 16-17
UNLOKR 16-17
WRITE 16-18
WRITEL 16-18
SAMPLE PROGRAM 16-19

CHAPTER 17 - CHAINING TO ALPHABASIC AND SYSTEM PROGRAMS 17-1

CHAINING TO ANOTHER AlphaBASIC PROGRAM 17-1
CHAINING TO SYSTEM FUNCTIONS 17-2

CHAPTER 18 - ERROR TRAPPING 18-1

WHAT IS ERROR TRAPPING? 18-1
THE ON ERROR GOTO STATEMENT 18-1
ERR(X) FUNCTION 18-2
Error Codes Returned By ERR(X) 18-2
THE RESUME STATEMENT 18-3
CONTROL-C TRAPPING 18-3
A SAMPLE ERROR RECOVERY ROUTINE 18-3
Another Sample Routine 18-4

CHAPTER 19 - EXTERNAL ASSEMBLY LANGUAGE SUBROUTINES 19-1

WHY USE ASSEMBLY LANGUAGE SUBROUTINES? 19-1
AUTOMATIC SUBROUTINE LOADING 19-2

CHAPTER 20 - USING ISAM FILES 20-1

WHAT IS ISAM? 20-1
ISAM FILE STRUCTURE 20-2
SYMBOLIC AND RELATIVE KEYS 20-2
THE ISAM STATEMENT 20-3
OPENING AN INDEXED FILE 20-3
FILE LOCKING FOR ISAM FILES 20-5
READ AND WRITE 20-5
READL AND WRITEL 20-5
LOCK AND UNLOKR 20-6
CLOSING AN INDEXED FILE 20-6
INDEXED'EXCLUSIVE MODE 20-6
ERROR PROCESSING 20-7

APPENDIX A - ALPHABASIC MESSAGES A-1

APPENDIX B - RESERVED WORDS B-1

APPENDIX C - MAKING YOUR PROGRAMS MORE EFFICIENT C-1

MANAGING MEMORY C-1
INCREASING EXECUTION SPEED C-1
MAKING YOUR SOURCE PROGRAMS EASIER TO READ C-2

APPENDIX D - SPECIAL SCREEN HANDLING CODES D-1

APPENDIX E - WRITING ASSEMBLY LANGUAGE SUBROUTINES E-1

REGISTER PARAMETERS E-1
ARGUMENT LIST FORMAT E-2
CONVERTING ARGUMENTS TO BINARY FORMAT E-3
FREE MEMORY USAGE E-3
LOCATING OPEN FILES E-3
PROGRAM HEADERS E-3

APPENDIX F - THE ASCII CHARACTER SET F-1

APPENDIX G - SAMPLE PROGRAM G-1

Chapter 1 - Introduction to AlphaBASIC

This chapter introduces you to AlphaBASIC and discusses some of the concepts we will be using throughout the manual. The topics are:

- Who is this manual written for?
- What is BASIC?
- How is AlphaBASIC special?
- What can AlphaBASIC do?
- What types of data Does AlphaBASIC support?
- The AlphaBASIC system
- What does an AlphaBASIC program look like?
- Graphic conventions we'll be using

WHO IS THIS MANUAL WRITTEN FOR?

This is a reference manual for the AlphaBASIC language. It is not meant to be a tutorial for the BASIC language, or to teach concepts of programming.

However, we have tried to make this manual easy to use and understand. In general, the text of this manual is written with the assumption that the reader is inexperienced with BASIC.

A glossary is provided near the back of this manual. It defines terms that are used in AlphaBASIC, and other terms relating to this manual and to the Alpha Micro system.

If you are an experienced BASIC programmer, you may want to glance through chapters 2 through 20 to familiarize yourself with the syntax and features of AlphaBASIC. As you program you will probably find the *AlphaBASIC Quick Reference Card* to be helpful to remind you of commands and syntax.

If you are new to or relatively inexperienced with the BASIC language, you should read chapter 1 before beginning to use AlphaBASIC. You may also want to read a tutorial on BASIC, and/or a tutorial about programming concepts.

WHAT IS BASIC?

The acronym BASIC stands for:

- Beginners'
- All-purpose
- Symbolic
- Instruction
- Code

BASIC is a higher-level programming language that was created to be a versatile tool for learning computer programming, and also to provide a relatively simple language for a wide variety of applications.

Over the years since its inception, BASIC has been added to and modified as new concepts of programming have emerged. Some implementations of BASIC are more extensive than others; the use of these extended versions allows the programmer a wider range of applications, greater ease in programming, and greater efficiency and speed. Now most programming on small, interactive systems is done in BASIC. One of the reasons for this is that BASIC is so much like the English language.

HOW IS AlphaBASIC SPECIAL?

AlphaBASIC is an extended version of the BASIC language, with several features not found in other implementations. These features not only enhance the performance of traditional uses of the language but also make business applications easier to program. Here are some of AlphaBASIC's special features:

- MAP statements that make it easy to create hierarchical data structures, much like the structured data types available in the COBOL and PASCAL languages.
- The ability to call assembly language subroutines from within AlphaBASIC.
- Variable names that may be any number of alphanumeric characters, which helps to improve program source readability.
- The ability to use labels instead of or in addition to line numbers. Eliminating line numbers makes the program source code easier to read, understand, and update.

WHAT CAN AlphaBASIC DO?

AlphaBASIC can be used in two different ways–either in interactive mode, or by using disk files, a compiler, and a run-time package.

Interactive mode operates much like a traditional interactive interpreter; that is, you create, alter and test your program while it is in your memory partition. This mode is convenient for the creation and debugging of new programs. It also makes it easy to quickly test a program by altering lines or data within the program.

Using disk files, the compiler, and a run-time package is more useful for programs to be put into production use, or for testing programs which are too large to fit in memory in the interactive mode.

To use AlphaBASIC in this way, create the program file using the AlphaVUE, or AlphaEXD text editor, then compile the program at monitor level (using the COMPIL program) and store the compiled object code on the disk. Finally, use the run-time package (the RUN program) to execute the program.

One of the advantages of using AlphaBASIC in this way is that AlphaVUE makes it easy to write and edit your program. See Chapter 5 for more information about creating and editing programs with AlphaVUE.

When running the compiled program, only the object code and the run-time execution package are in memory. This conserves memory on the computer.

The compiler and the run-time package are both written as re-entrant programs. This means that in a timesharing environment, any or all users who are running or debugging programs may share one copy of the compiler and the run-time package. The way to do this is to load COMPIL.LIT (the compiler program) and RUN.LIT (the run-time package) into system memory. See the *System Operator's Guide* to the system initialization command file for information on loading files into system memory.

Once created by the compiler, an object program (also known as a compiled program) is also re-entrant and sharable, and may be placed in system memory or in the BASIC program account (DSK0:[2,2]) so it may be shared by all the users on the system.

WHAT TYPES OF DATA DOES AlphaBASIC SUPPORT?

AlphaBASIC supports the following data formats:

- FLOATING POINT (numbers including decimals)
- STRING (ASCII alphanumeric characters)
- BINARY (numbers expressed as patterns of zeroes and ones)
- UNFORMATTED (special variables set up by MAP statements)

All data formats may be simple variables or array structures. In addition, the unique memory mapping system allows you to specify the ordering of variables in special groups. This "grouping" of variables makes them easier to use, and makes your program more efficient.

The MAP statement is similar to the data formatting capabilities of the COBOL language, and is useful in business applications where the grouping of related information is important.

Variable names are not limited to the single character and single digit format of many BASICs, but may be any number of alphanumeric characters in length, as long as the first character is alphabetic. This is another feature which makes AlphaBASIC well suited for business applications.

Since the source code is compiled and need not be in memory when the program is eventually run, the length of the variable name is not a significant concern. Label names may also be used to identify points in the program for GOTO and GOSUB branches. Label names are alphanumeric and help to clarify the program structure. Some examples of label names are:

```
EXIT'ERROR:
EVALUATE'ANSWER:
```

THE AlphaBASIC SYSTEM

The AlphaBASIC system consists of three programs:

RUN.LIT	The monitor level AlphaBASIC run-time package
COMPIL.LIT	The monitor level disk-based AlphaBASIC compiler
BASIC.LIT	A BASIC module that combines an interactive compiler with a run-time package to give you an interactive BASIC environment

You use RUN and COMPIL from monitor command level to run and compile AlphaBASIC programs that exist as disk files. You use the BASIC command when you want to use AlphaBASIC in interactive mode.

With these three programs, then, you can use the AlphaBASIC system in two different ways–either as an interactive system (interactive mode) or as a disk-file/compiler system.

Your choice of how you will use AlphaBASIC depends on several factors:

- The amount of memory you have in your user partition
- What stage of development your program is in
- The physical form of your program
- The size of your program
- Your personal preference

Chapters 3 and 4 tell you more about the interactive mode, and Chapter 5 explains how to use AlphaVUE, the compiler, and the RUN program.

WHAT DOES AN AlphaBASIC PROGRAM LOOK LIKE?

Here is a small sample program written in AlphaBASIC:

```
PRINT "Hello!"
PRINT
INPUT "Type in a number that you would like squared: ",NUMBER
ANSWER = NUMBER * NUMBER
PRINT
PRINT "The square of ";NUMBER;" is: ";ANSWER
PRINT
END
```

Notice that the AlphaBASIC statements in the above program are English words (PRINT, INPUT, END) which resemble their function. Likewise, Alpha BASIC allows you to define meaningful variable names in English(NUMBER and ANSWER). You'll learn more about the above statements later in the manual.

GRAPHICS CONVENTIONS

This manual conforms to other Alpha Micro publications in its use of a standard set of graphics conventions. We hope these graphics simplify our examples and make them easier for you to understand.

GRAPHICS	DESCRIPTION
{ }	Optional elements of a command line. When these braces appear in a sample command line, it means that the part of the command line that they enclose may be left out of the command line if you don't want or need it
`Type`	This type face indicates examples of program code. It also shows examples of characters the computer sends and displays on your terminal screen, such as prompts and messages.
`Type`	This bold type face is used in examples to show what you type at your keyboard. For example: **`LOG DSK0:[1,2]`**
RETURN	This symbol marks the place where you press the RETURN key on your keyboard. For example: **`LOGOFF`** RETURN tells you: "After the monitor prompt, type 'LOGOFF' and press the RETURN key."
	This symbol says "Don't forget!" and signals information to remember.
	This symbol indicates a hint, and identifies a shortcut or an easier way to do something.
	This symbol means **STOP!**, and signals an important warning or restriction you must be aware of before continuing.

Chapter 2 - General Information

This chapter gives general information about the form that your AlphaBASIC programs may take. We discuss:

- Lines with more than one BASIC statement
- Statements using more than one line
- Numbering lines
- How to insert comments in your programs
- Using labels in programs
- How AlphaBASIC allocates memory to programs
- EXPAND and NOEXPAND modes
- Letter case handling in programs
- AlphaBASIC libraries
- Including one program file inside another

MULTIPLE STATEMENT LINES

AlphaBASIC supports multiple statement lines. Multiple statement lines can be formed by using colons to separate the statements. For example, these lines:

```
FOR I = 1 TO 10
   PRINT "This is a loop."
NEXT I
```

could be written as:

```
FOR I = 1 TO 10 : PRINT "This is a loop." : NEXT I
```

A DATA statement cannot contain other statements on the same line, and no other statements may follow a comment (designated by REM or !). Direct statements in interactive mode may also be multiple statement lines.

CONTINUATION LINES

COMPIL allows the use of continuation lines within the source program. That is, statements may be continued on the next line by using the ampersand (&) symbol as the last character on the line. For example:

```
IF STOCK'NUMBER > 599 THEN &
   GOTO OVER'STOCKED
```

Notice that the example above, though appearing on two lines in your source file, is treated as one statement by AlphaBASIC. Therefore, you must not precede a line that is being continued from the line before by a label or line number.

By using indentation, continuation lines, and eliminating line numbers (discussed in the next section) your source program will look more structured than those written in conventional BASIC Here is an example of a "structured" program:

```
TIME'OF'DAY = TIME/3600
   IF TIME'OF'DAY > 12 &
      AND TIME'OF'DAY < 13 &
         THEN PRINT "IT IS LUNCH TIME" &
            ELSE PRINT "GO BACK TO WORK"
```

The maximum size of any AlphaBASIC statement, including blanks, tabs, ampersands, and carriage return/line feeds is, 500 characters.

LINE NUMBERS

Program line numbers may range from 1 to 65533. Programs used in interactive mode MUST contain line numbers. Programs that will be compiled at AMOS level do not need to have line numbers. You may use line numbers only for certain lines in a compiled program if you like–not all lines have to have numbers.

If you have a line number on the line at which an error occurs, AlphaBASIC will be able to tell you where the error occurred by displaying that line number. Without line numbers, it cannot report where the error occurred.

You may want to use line numbers in special places within your program when you are debugging the program, so that you can easily locate errors. A disadvantage to line numbers is that they occupy space in memory, making your program larger.

COMMENTS (REM AND !)

AlphaBASIC allows you to insert comments into your source program in two different ways. The keyword REM may appear alone on a line followed by the comment, or may be inserted on the same line as a statement, to comment on the purpose of the statement. You may follow the REM (or "remarks") keyword with anything you want. For example:

```
REM  ANYTHING YOU WISH TO SAY
PRINT A     REM -->  VARIABLE A MEANS "ALLOWANCE"
```

Note that any statement that follows a REM keyword on a line will NOT be executed by AlphaBASIC. When the program is compiled, everything following the REM statement on the line is ignored.

The comment symbol ! is an abbreviation of the REM statement, and is used the same way. For example:

```
PRINT "TRY ANOTHER TIME"   ! IF THEY MISS BETWEEN
GOTO AGAIN                 ! ONE AND THREE TIMES.
```

Like the REM statement, anything following the ! symbol on the line is ignored. You may "comment out" a section of a program that contains comments. For example:

```
REM  PRINT "TRY ANOTHER TIME"  ! IF THEY MISS BETWEEN
REM  GOTO AGAIN                ! ONE AND THREE TIMES.
```

LABELS

AlphaBASIC allows the use of labels to identify locations in a program. A label is composed of one or more alphanumeric characters which are not separated by a space or other delimiter. The first character must be an alphabetic character. Since apostrophes are not recognized as delimiters, they may be used within labels in place of spaces for clarity.. For example:

```
ERROR'MESSAGE:
ADDITION'ROUTINE:
START:
```

A label must be the first item on a line and must be terminated by a colon (:).

Remember that you may not place a space between the label and its colon. This will cause AlphaBASIC to think that you have entered a multi-statement line rather than a label.

A label may be followed by a program statement on the same line, or it may be the only item on the line. The use of labels is similar to the use of line numbers with GOTO and GOSUB statements, except that labels make the program easier to document. Here is a program that uses labels:

```
START'PROGRAM:
     INPUT "Enter two numbers to get the sum: ",A,B
     PRINT
     PRINT A; "+"; B; "=" A + B
     IF A + B <> 0 GOTO SUM'NOT'ZERO
     PRINT "The sum is zero"
     GOTO END'PROGRAM
SUM'NOT'ZERO:
     PRINT : PRINT "The sum is not zero"
END'PROGRAM:
     END
```

START'PROGRAM:, SUM'NOT'ZERO:, and END'PROGRAM: are labels. Note that a reference to a label (such as "GOTO END'PROGRAM") is not the first item on a line nor terminated by a colon. The upper/lower case of letters and the placement of apostrophes in the reference label must be identical to the actual label. For example, "GOTO ENDPROGRAM" or "GOTO End'Program" would produce an error message.

MEMORY ALLOCATION

In interactive mode, memory is allocated dynamically as you edit your program, and also during its compilation and execution. Checks are made to tell you if you have run out of memory. If you do, you get an error message.

If you run out of memory while COMPIL is compiling a disk program, you will see an error message and the compilation will stop, returning you to the monitor.

EXPAND AND NOEXPAND MODES

AlphaBASIC normally scans the source text of the program in EXPAND mode, which dictates that reserved words (verbs, functions, commands, etc.) be terminated by a space or a character that is illegal in variable names.

This allows labels and variables to begin with reserved words. In other words, the variable name:

```
PRINTMASTER
```

is not interpreted as:

```
PRINT MASTER
```

in expanded mode. In EXPAND mode, the statement:

```
FOR A = 1 TO 10
```

cannot be written as:

```
FORA=1TO10
```

These two commands switch back and forth between the normal EXPAND mode and the NOEXPAND mode:

`EXPAND`	sets syntax scanner to expanded mode
`NOEXPAND`	sets syntax scanner to non-expanded mode

For example:

```
EXPAND
FOR I = 1 TO 10 : PRINT A$ : NEXT I

NOEXPAND
FORI=1TO10:PRINTA$:NEXTI
```

The default mode is EXPAND mode. Note that the object code which is generated as a result of a compilation is not affected in size, execution speed or anything else by the mode in which it is compiled.

NOEXPAND is usually used only when running programs written on other systems.

CASE OF CHARACTERS

AlphaBASIC supports lower case letters (a-z) and upper case letters (A-Z) in both the input source program and in the run-time execution of programs. The line editor built into the interactive system accepts and stores source input text in lower case or upper case characters. Lower case letters, when used within variable names and labels, are unique and separate from the corresponding upper case letters.

In other words, the variable "a" is separate from the variable "A" and the variable "Tom" is separate from the variables "TOM" and "tom". Lower case letters may be used as the first character of a variable name or program label just as upper case letters may be.

Reserved words are treated somewhat differently from the above system. AlphaBASIC temporarily translates all lower case letters to upper case when it interprets a line, and then checks for reserved word matches. If a word is not a reserved word, the translation is not retained and the lower case letters are used for variable name matches.

The following statements are all considered identical:

```
FOR A = 1 TO 100 STEP 2    For A = 1 To 100 Step 2
For A = 1 to 100 step 2    For A = 1 TO 100 stEp 2
```

The entire string processing system supports lower case characters. That is, lower case letters used within string literals (alphanumeric characters inside quotes) are retained and printed as lower case. Lower case letters which are entered into string variables by means of the INPUT statement are also retained as lower case letters.

Note that all lower case characters are considered greater than any upper case character due to their position in the ASCII collating sequence. To assist in processing and comparing data, a couple of translation functions are included in AlphaBASIC.

The UCS(A$) function returns a string identical to the argument string A$ with all characters translated to upper case. The inverse function LCS(A$) returns a string with all characters translated to lower case.

SEARCHING FOR A LIBRARY

When a program (called with a RUN command or a CHAIN statement) or a subroutine (called with an XCALL statement) is requested, AlphaBASIC tries to execute it. If the program/routine you requested is not specified with a complete file specification, AlphaBASIC will look for it assuming a number of default specifications. AlphaBASIC follows a specific pattern when looking for the requested .RUN or .SBR module. If you specify an account, then AlphaBASIC uses the current default device and the specified account. If you do not specify an account, the search sequence is as follows:

1. System memory
2. Your memory partition
3. Your disk:[Your account]
4. Your disk:[Your Library account (P,0)]
5. The BASIC Library account, DSK0:[7,6]

By "your account" we mean the account you are currently logged into. If you specify a device, AlphaBASIC does not search in memory but proceeds directly to that device, using steps 3 and 4.

HOW TO INCLUDE ONE AlphaBASIC PROGRAM INSIDE ANOTHER

The ++INCLUDE command allows the compiler to fetch another file during compilation and include it as part of your file. Say that you have a section of 20 lines of BASIC code in a file called SUBX.BSI that you use in most of your other programs. Rather than typing those 20 lines into a program you are creating, you can enter (at the point you want those 20 lines to appear):

```
++INCLUDE SUBX.BSI
```

++INCLUDE must be the first command on a line (do not put a line number before it), and the filespec can be any valid file specification that optionally includes a device name and/or account. The default extension is .BSI (BASIC Include file). If you do not specify a device and/or account the search pattern is the same as that for .RUN modules (see above). As COMPIL accesses each file defined in a ++INCLUDE command, you see:

```
Copying from xxxx
```

where ***xxxx*** is the filespec you specified in the ++INCLUDE command.

Multiple ++INCLUDE commands in the same file are allowed, but a file that is copied in may not itself contain ++INCLUDE commands.

If you load a file containing ++INCLUDE commands into interactive mode, AlphaBASIC will give you an error message (because that line cannot have a line number) and then ignore the ++INCLUDE commands.

Chapter 3 - AlphaBASIC Interactive Mode

This chapter discusses the AlphaBASIC interactive mode. The topics are:

- What is interactive mode?
- Clearing memory for a new program
- Creating a program
- Saving a program
- Compiling a program
- Running a program
- Direct statements
- Debugging features

WHAT IS INTERACTIVE MODE?

We call it "interactive mode" because you interact directly with the AlphaBASIC interpreter as you write or edit your program. As you enter each line and press RETURN , BASIC looks at the line and figures out if it has been entered correctly–if what you entered is a legal AlphaBASIC statement.

Interactive mode is like being "inside" BASIC, and gives you a number of advantages, such as:

- Helping you learn how to use the BASIC programming statements, as BASIC checks each line you enter.
- Easily changing lines or variables between program runs. This makes it easy to test various parts of your program or variable ranges and values.
- Running your program one line at a time. This is called the "single-step" feature.
- Setting "breakpoints"–places inside your program where the run will pause. These pauses, combined with the "single-step" feature, can help you locate errors in your programs.

There are also a few disadvantages to interactive mode:

- Because you are sharing memory with the AlphaBASIC interpreter program, your programs cannot be as large as if they were created with AlphaVUE (we'll discuss that method of writing programs in Chapter 5).
- You must use line numbers, which makes it more difficult to have your program look "structured." Line numbers also make it more difficult to modify your program.
- It is more difficult to edit your program, because you must retype an entire line to make a change.

Therefore, you will probably use interactive mode if:

- You are new to AlphaBASIC, and want to try out BASIC statements or small programs.
- Your program is in the early stage of its development, and you want to use the debugging features.
- You want to use direct statements to do mathematical calculations or test short processes.

To use interactive mode, first make sure that you are at monitor level (where you see the AMOS prompt symbol). If you do not see a prompt, press CTRL /**C**. That is, hold down the CONTROL key (sometimes labeled CTRL) and press the C key. If the prompt still does not appear, see your System Operator.

When you see the prompt, type the word BASIC and press RETURN :

```
BASIC RETURN
```

You then see:

```
AlphaBASIC Version X.X (XXX)

READY
```

READY is the AlphaBASIC prompt symbol. You are now in interactive mode.

To leave interactive mode and return to AMOS, type BYE after the READY prompt, and press RETURN :

```
READY
BYE RETURN
```

CLEARING MEMORY FOR A NEW PROGRAM (NEW)

If you want to create or load in a new program, you should first make sure that there is nothing in memory. To clear memory, enter NEW and press RETURN . For example:

```
READY
NEW RETURN
READY
```

If you do not erase the program in memory, AlphaBASIC will merge the new program into whatever is in memory. If any line numbers from the new program duplicate line numbers of an old program in memory, the new lines will replace the old.

CREATING A PROGRAM

To create a program in interactive mode, first make sure that you have nothing in memory (see NEW above), and then begin entering program lines. Each line must begin with a line number. You do not have to enter the lines in numerical order–BASIC will put them in order for you. For example, you might type in:

```
10  INPUT "Enter a number: ", NUM [RETURN]
30  PRINT NUM [RETURN]
20  PRINT [RETURN]
```

If you make a mistake when you enter a line, you will see:

```
40  A = A X A [RETURN]
?Syntax Error
```

This means that BASIC did not understand your input. Re-type the correct line:

```
40  A = A * A [RETURN]
```

To edit a program line, you must retype the entire line as we did above. If you want to delete a line, enter the line number and press [RETURN] :

```
READY
40 [RETURN]
```

SAVING A PROGRAM (SAVE)

Once you have your program entered (or even when it is partially entered), you may want to save the program on the disk. In interactive mode, your program is only in temporary memory, and it will be erased when you leave interactive mode. To save a program, type SAVE, a name for the program, and press [RETURN] . For example:

```
READY
SAVE NEWPRG [RETURN]

READY
```

The command above saves the source program NEWPRG.BAS as a disk file in the account you are logged into (.BAS is the default extension).

All program names must be made up of one to six alphanumeric characters. All file extensions must be made up of one to three alphanumeric characters.

Programs are saved as sequential ASCII (American Standard Code for Information Interchange) files. Since this is a standard format, you can use the AlphaVUE text editor to edit your file later.

If you want to SAVE the program to another account, or with a different extension, you can specify a full file specification. For example:

```
READY
SAVE NEWPRG.SAM[125,3] [RETURN]
```

or:

```
READY
SAVE DSK2:PAYROL.BAS[50,1] [RETURN]
```

You can save the compiled version (we talk about compiling programs and .RUN files later) of that program by specifying the .RUN extension:

```
READY
SAVE NEWPRG.RUN RETURN
```

If you have not previously compiled the source program, or if you have changed the program since the last time you compiled it, BASIC automatically compiles it for you when you save a .RUN file to ensure that you are saving the most current version.

If you try to save a file when there is no source program in memory, AlphaBASIC reports:

```
READY
SAVE NEWPRG RETURN

No source program in text buffer
```

If a previous version of the program (one with the same name) already exists on the disk in the account you are writing the file to, that program is first deleted before the new program is saved. AlphaBASIC does not automatically create a backup file.

The resulting object program (.RUN file) is re-entrant (that is, it can be used over and over again), and may be loaded into system memory so that everyone on your system can use the program (even at the same time).

In the interests of security, AlphaBASIC will not let you save a program in an account that is not within the same project as the account you are logged into. For example, if you are logged into DSK2:[100,2] and you try to save a program in DSK2:[340,1], you see:

```
READY
SAVE NEWPRG[340,1] RETURN

?Cannot OPEN NEWPRG.BAS[340,1] - Protection violation
```

Since AlphaBASIC cannot convert an object file back to a source program file, you should save the .BAS version of your program.

LISTING A PROGRAM (LIST)

If you have a program in memory, you can get a listing of it by typing LIST and pressing RETURN :

```
READY
LIST RETURN
10  INPUT "Enter a number: ", NUM
20  PRINT
30  PRINT NUM
40  INPUT "Enter a second number: ", NUM2
50  PRINT
60  PRINT NUM, NUM2, (NUM * NUM2)
```

If the program is longer than will fit on your screen, use the SCROLL key (or CTRL /**S** and CTRL /**R**) to stop and resume the display, or use CTRL /**C** to interrupt the listing.

If you only want to see part of your listing, you can ask for a section by adding the beginning and ending line numbers to the LIST command. For example:

```
READY
LIST 30,50 RETURN
30  PRINT NUM
40  INPUT "Enter a second number: ", NUM2
50  PRINT
```

Or, if you want to see a single line:

```
READY
LIST 60 RETURN
60  PRINT NUM, NUM2, (NUM * NUM2)
```

COMPILING A PROGRAM

Each statement that you enter is checked when you press RETURN. If the statement is correct, nothing happens. If there is an error in the statement, you receive an error message.

When you run a program (explained below) AlphaBASIC will automatically compile the program if it is not currently compiled. You can also use the COMPILE command to compile the program. To do so, type COMPILE and press RETURN. For example:

```
READY
COMPILE RETURN
COMPILING
Compile time was 0.003 seconds, elapsed time was 0 seconds

READY
```

In interactive mode, you may compile only the program currently in memory.

The resulting object program (.RUN file) is re-usable (that is, it can be used over and over again), and re-entrant (that is, it may be loaded into system memory so that everyone on your system can use the program–even at the same time).

Once the program is compiled, the object code resides in memory along with the source program.

Compiling a program sets all variables to zero and deletes all variables that may have been left over because of direct statements (direct statements are discussed below).

If no program is in memory, you get an error message and a prompt:

```
READY
COMPILE [RETURN]
COMPILING
?No source program in text buffer

READY
```

Compiler Option

You may specify the /O compiler option to the interactive compiler. For example:

```
COMPILE ACCT /O [RETURN]
```

The /O option tells COMPILE to remove the line numbers from the compiled object code. It does NOT change your source program, or the operation of your compiler object code. Even though line numbers are removed, the /O switch still detects and reports duplicate line number errors.

Removing line number references from your object program will make your compiled program smaller and improve execution time. However, if an error occurs while executing the program, the resulting error message will not show the number of the line where the error occurred.

RUNNING A PROGRAM (RUN)

To execute a program, use the RUN command. For example:

```
READY
RUN [RETURN]
```

(The program begins running)

RUN resets all of the variables to zero and all string variables to a NULL value (the string contains no characters) before running the program. If you wish to stop the program during the run, press [CTRL] /**C.**

If the program has been changed since the last time it was compiled, BASIC automatically recompiles it for you before executing the program. Therefore, if you need to compile the source program and then run it, you may simply use the RUN command. For example:

```
READY

10  REM This is a small test program [RETURN]
20  FOR I = 1 TO 3 [RETURN]
30  PRINT "Little tasks make large return." [RETURN]
40  NEXT I [RETURN]
RUN [RETURN]
COMPILING
Compile time was 0.003 seconds, elapsed time was 0 seconds
Little tasks make large return.
Little tasks make large return.
Little tasks make large return.
```

```
Runtime was 0.40 seconds
CPU time was 0.004 seconds, elapsed time was 0 seconds

READY
```

You cannot run any program other than the one currently in memory while in interactive mode.

EXITING FROM INTERACTIVE MODE (BYE)

If you have a program in memory when you use BYE, that program will be erased. Therefore, if you wish to keep a copy of the program, use the SAVE command before using BYE.

To exit from interactive mode, type BYE after the READY prompt and press RETURN :

```
READY
BYE RETURN
```

DIRECT STATEMENTS

Program statements that do not begin with a line number are considered direct statements, and AlphaBASIC executes them immediately. For example:

```
READY
A = 5 RETURN
PRINT A + 4 RETURN
9
```

The first direct statement (A=5) stores 5 as the value of the variable A in memory. The second (PRINT A+4) adds 4 to the value of A and, since this is a direct statement, it prints out the answer (9). Each direct statement is compiled and executed when it is entered. You can define variables and change variable values using direct statements.

Certain statements are meaningless as direct statements; therefore they are not allowed (for example, RESUME, GOSUB, etc.). If you enter a statement that is not allowed, you will see:

```
GOSUB ADD'NUMBERS RETURN
?Illegal in immediate mode
```

AlphaBASIC allows multi-statement lines as direct statements. Multi-statement lines are lines which contain more than one statement; the statements are separated by colons. As you enter direct statements, AlphaBASIC checks them to see that they are in proper form. If they are not, you will see:

```
?Syntax error
```

A HINT FOR THE SYSTEM OPERATOR

Because BASIC.LIT is re-entrant, you may place it in system memory to save room in user partitions. If you do so, you may also want to place RUN.LIT in system memory. However, since BASIC is a fairly large program, you should put it in system memory if most of the users on your system do a significant amount of BASIC program development.

Chapter 4 - More Interactive Commands

This chapter discusses:

- Using the BREAK command to interrupt program runs
- Continuing a program after a BREAK
- Exiting from a program run using CTRL /**C**
- Deleting lines from a program
- Loading programs into BASIC from the disk
- Using the single-step feature to debug programs

INTERRUPTING A PROGRAM RUN (BREAK)

The BREAK command is a feature that can aid you in finding errors in your programs. The BREAK command can be used to set breakpoints, clear breakpoints, and display breakpoints. For example:

`BREAK`	Lists all breakpoints currently set
`BREAK 100`	Sets a breakpoint at line 100
`BREAK 30,50`	Sets breakpoints at lines 30 and 50
`BREAK -30`	Clears the breakpoint at line 30
`BREAK 20,-120`	Sets a breakpoint at line 20, clears the one at line 120

When you set a breakpoint with the BREAK command, you are asking BASIC to stop just before the line you specified. When the program is run, it will proceed until it hits the breakpoint. Then you will see:

```
Break at line x
```

At this point you can enter interactive mode commands, proceed line by line using the single-step feature (see below), or enter direct statements. The advantage of entering direct statements is that you can find out what values the variables have, or even change those values. For example:

```
READY
RUN RETURN
Break at line 40
PRINT A RETURN
5
```

At this point, the value of the variable A is 5 before line 40 is executed. If your program is not giving you the answer you expect, you can use breakpoints at key points in your program to determine where the calculations are going wrong.

You can also change the value of a variable. Say that in the example above you wanted the value of A to be 4. By entering "A = 4" as a direct statement, you can change the value for the duration of the program run.

There is no limit to the number of breakpoints that may be set in one program. The program will not run any slower with breakpoints in it than normally.

You may continue to run the rest of the program from the breakpoint by typing **CONT** (for continue) and pressing RETURN:

```
Break at line 40
PRINT A RETURN
5
CONT RETURN
( The program continues )
```

Compiling a program clears all the breakpoints. Breakpoints are also cleared if you exit from BASIC.

If you want to see the breakpoints set, type **BREAK** and press RETURN.

```
READY                                  READY
BREAK RETURN                  or       BREAK RETURN
No breakpoints set                     30
                                       60
```

You may start the program over again by using the RUN command; again it will break at the first breakpoint set.

CONTINUING AN INTERRUPTED PROGRAM (CONT)

CONT, which stands for "continue," causes a suspended program to continue execution from the point at which it was suspended. You may suspend a program by using a BREAK command previous to program execution or by using a STOP statement within the program. You may not continue a program after it has finished. The following is an example of CONT after a STOP statement suspended a program:

```
Program stop in line 700

READY
CONT RETURN
```

The program continues by next executing the first line numbered higher than line 700. CONT also continues a program which you have partially executed using the single-step feature (see below).

USING CONTROL-C TO INTERRUPT A PROGRAM RUN

Pressing the CTRL and C keys simultaneously interrupts a running program. Depending on the operation being performed, the Control-C symbol (^C) may be displayed on the terminal screen. The line number of the source program at which the program was interrupted is displayed in the message. For example:

```
CTRL /C
^C
Operator interrupt in line 700
READY
```

or

```
CTRL /C
Operator interrupt in line 700
READY
```

You may restart a program from the beginning following a CTRL /**C** by using the RUN command or single-step (linefeed) command.

DELETING PROGRAM LINES (DELETE)

The DELETE command is used to delete lines from the program text. You can delete one line from the program by typing **DELETE**, a space, and the line number, and then pressing RETURN. For example:

```
READY
DELETE 40 RETURN
```

You can also delete a group of line numbers by specifying a beginning and ending number. For example, to delete all lines between (and including) 40 and 150:

```
READY
DELETE 40,150 RETURN
```

Although you usually separate the two line numbers with a comma, you can also use a dash, space, or other non-numeric character.

Here is an example of a program listing before and after a DELETE:

```
LIST RETURN
10  FOR I = 1 TO 10
20  PRINT TAB(I,5)"ONE"
30  PRINT TAB(I,5)"TWO"
40  PRINT TAB(I,5)"SIX"
50  PRINT TAB(I,5)"TEN"
60  NEXT I

READY
DELETE 20,40 RETURN
```

```
READY
LIST RETURN
10  FOR I = 1 TO 10
50  PRINT TAB(I,5)"TEN"
60  NEXT I

READY
```

LOADING A PROGRAM INTO MEMORY (LOAD)

If you have a program that has been saved as a disk file, you can load it into temporary memory so that you can edit or run it. To do this, type **LOAD**, the name of the file, and press RETURN. For example:

```
READY
LOAD NEWPRG.BAS[100,1] RETURN

READY
```

This program file could have been created using AlphaVUE, or might have been saved from a previous interactive mode session.

If you do not supply a file extension, AlphaBASIC uses the default extension of .BAS. BASIC will also assume the account and device you are logged into.

If BASIC can't find the file you want to load, it displays an error message:

```
?Cannot OPEN NEWPRG.BAS[100,1] - file not found
```

The LOAD command does not clear the text buffer before it loads the requested file, and therefore may be used to concatenate or merge several programs or subroutines together to be saved as a single program.

The separate routines must not duplicate line numbers in the routines they are to be merged with or the new line numbers will overlay the old ones, just as if the file had been typed in from your terminal.

If you want to load in a complete program, you should always use the NEW command prior to any LOAD command to make sure that memory is clear. If you don't, you may mix the contents of a program in memory with the program you are loading.

Here are two examples of LOAD:

```
READY
LOAD NEWBAS RETURN

READY
```

```
READY
LOAD DSK2:PWRS2.BAS[50,1] RETURN

READY
```

THE SINGLE-STEP FEATURE

The single-step function is a feature not found in many versions of BASIC, and is very useful in debugging programs and in teaching the principles of BASIC programming to people who are new to BASIC. Once you have a program (or even part of a program) in memory, you can use the single-step command by pressing the [LINE FEED] key (sometimes labeled LF). On some terminals, the [↓] key functions as a [LINE FEED].

When you press [LINE FEED], the next line of the program is displayed on the terminal, and then executed. Here is an example:

```
LIST [RETURN]
10  PRINT "This is a demonstration of Single-step."
20  CONSTANT = 7
30  FOR I = 1 to 3
40  PRINT : INPUT "Enter a number: ",NUM
50  VALUE = VALUE + (CONSTANT + NUM - I)
60  NEXT I
70  PRINT VALUE
[LINE FEED]
READY
COMPILE [RETURN]
COMPILING
Compile time was 0.008 seconds, elapsed time was 1 seconds
10  PRINT "This is a demonstration of Single-step."
This is a demonstration of single-step.
[LINE FEED]
20  CONSTANT = 7
[LINE FEED]
30  FOR I = 1 to 3
[LINE FEED]
40  PRINT : INPUT "Enter a number: ",NUM
Enter a number: 4 [RETURN]
[LINE FEED]
50  VALUE = VALUE + (CONSTANT + NUM - I)
PRINT VALUE [RETURN]
10
[LINE FEED]
60  NEXT I
[LINE FEED]
40  PRINT : INPUT "Enter a number: ",NUM
Enter a number: 21 [RETURN]
[LINE FEED]
50  VALUE = VALUE + (CONSTANT + NUM - I)
PRINT VALUE [RETURN]
36
[LINE FEED]
60  NEXT I
[LINE FEED]
40  PRINT : INPUT "Enter a number: ",NUM
Enter a number: 154.5 [RETURN]
[LINE FEED]
50  VALUE = VALUE + (CONSTANT + NUM - I)
```

```
PRINT VALUE [RETURN]
194.5
VALUE = 192 [RETURN]
[LINE FEED]
60  NEXT I
[LINE FEED]
70  PRINT VALUE
192

*** End of Program ***
```

Note that line 40 is a multi-statement line. When single-stepping, all statements on a line are executed at the same time.

At each point after a line has been executed, you can use direct statements to look at or change variable values (as we did with the PRINT VALUE and VALUE = 192 direct statements in the example above) before you continue the run of the program (by using the CONT command or another [LINE FEED]).

Note that any change in the source program causes AlphaBASIC to recompile the program. The next single-step command will then execute the first line of the program. When we changed the variable VALUE to 192, we did not change any of the program lines–therefore, BASIC did not recompile the program. Only a variable was changed.

The single-step feature can be used at the beginning of the program, after program STOP statements, breakpoint interrupts, and other functions that suspend program execution.

After partially single-stepping through a program, you may execute the remainder of it normally by using the CONT command. Also, you may start over at the beginning and execute it normally by using the RUN command. If you try to single-step past the end of the program, you see:

```
***End of Program***
```

and the next [LINE FEED] executes the first program statement again.

If you single-step a statement that asks for input from the terminal, enter the input and press [RETURN] (as we did in the example above).Then you may proceed to the next statement by pressing [LINE FEED] again.

Chapter 5 - Writing AlphaBASIC Programs Using AlphaVUE

In this chapter we will discuss:

- Using AlphaVUE to create a program
- The form of an AlphaBASIC program
- Compiling a program
- Running a program

Although interactive mode offers some good features, the true power of AlphaBASIC lies in using a combination of AlphaVUE, COMPIL, and RUN to create, compile, and run your programs.

CREATING A PROGRAM

There are two ways to create a source program: using interactive mode to type in the program, save the program on disk, and then exit BASIC; or using AlphaVUE.

The easiest way to create a program that is going to be compiled with COMPIL is to use AlphaVUE to create the source file (which usually has a .BAS extension).

AlphaVUE is a screen-oriented text editor that allows you to see your program on the terminal screen as you type it in. You can move the cursor around on the screen and change or delete text at the current cursor position. It also has some unique features designed especially to aid programmers in writing programs. See the *AlphaVUE User's Manual*, if you are not familiar with AlphaVUE.

PROGRAM FORM

The form your program takes may differ somewhat depending on whether you use AlphaVUE or the interactive mode to create your program. If you create and save your source program in interactive mode, that program must contain line numbers (otherwise, AlphaBASIC would interpret each statement as a direct statement when you tried to type the program in).

The monitor level compiler (COMPIL) does not require that a program contain line numbers. That means that if you create your program using AlphaVUE, you do not need to include line numbers in that program. In addition, you may indent your program lines in any fashion you desire.

By omitting line numbers and using labels, and by using indentation judiciously, you can give your source program a much more structured look than is usually possible with BASIC programs. A "label" is a special name defined by you that identifies a location within a program.

Although COMPIL does not require your program to have line numbers, it does check for duplicate line numbers within your file. Even when the /O switch is in effect, the compiler will report duplicate line number errors. It does NOT, however, check for lines that are out of numeric order. See below for information on the /O option.

The compiler also allows the use of statements that start on one line and continue on to another line. These are called continuation lines. An ampersand (&) is the symbol used to indicate a continuation line. For example:

```
IF Answer = Right'Number THEN &
          PRINT "Very Good!" &
     ELSE &
          PRINT "Try again."
```

As you may have noticed, the above example is only one AlphaBASIC statement, even though it is written on four lines.

However, you may not want to use continuation lines if you want to work on your program in interactive mode. Interactive BASIC puts continuation lines back together to form one line.

An AlphaBASIC statement cannot be longer than 500 characters, even if continuation symbols are used.

Below is a small example of a valid program that uses continuation lines, indentation, and labels, and has no line numbers:

```
     ! Program to print a name in reverse.

     STRSIZ 20
START:
     PRINT
     INPUT LINE "Enter your name: ",NAME$
     IF LEN(NAME$) = 0 &
          THEN GOTO START
     COUNTER = LEN(NAME$)
     PRINT
LOOP:
     FOR COLUMN = COUNTER TO 1 STEP -1
          PRINT NAME$[COLUMN;1];
     NEXT
     PRINT : PRINT
     INPUT "Would you like to try another? ", QUERY$
     IF LCS(QUERY$[1;1]) = "y" &
          THEN GOTO START &
               ELSE PRINT : PRINT "All done."
     END
```

COMPILING A PROGRAM

Compiling an AlphaBASIC program is done by using the COMPIL program. Compiling a program translates your BASIC source program into a language that the RUN program can execute. To compile a

program, make sure that you are at AMOS command level, then enter COMPIL and the name of your AlphaBASIC program, plus any options you wish. For example:

```
COMPIL PAYROL RETURN
```

The default extension that COMPIL uses is .BAS. That is, if you do not add an extension to the filename, COMPIL will look for a file with that name and a .BAS extension.

You then see a number of statistics on your terminal as COMPIL compiles your program. A typical display might look something like this:

```
COMPIL PAYROL RETURN
Phase 1 - Initial work memory is 2310 bytes
Phase 2 - Adjust object file and process errors
Memory usage:
          Total work space - 4712 bytes
          Label symbol tree - 322 bytes
          Variable symbol tree - 1186 bytes
          Data statement pool - 0 bytes
          Variable indexing area - 274 bytes
          Compiler work stack - 140 bytes
          Excess available memory - 11918 bytes
```

COMPIL will tell you if any errors exist within your source program when it processes your file. The "Excess available memory" message gives you an idea of how much memory your program used in compiling, and how close you were to running out of memory.

The COMPIL.LIT and RUN.LIT programs may be loaded into your user memory partition to make the compiling and/or running process faster. See the LOAD command reference sheet in the *System Commands Reference Manual* that applies to your system.

The COMPIL program uses memory in your user memory partition to process your program. If there is not enough memory in your partition for COMPIL to complete its task, you will see:

```
(Out of memory - compilation aborted)
```

and COMPIL will return your terminal to monitor command level. If this occurs, you may want to remove files that were loaded into your partition (see the DEL command reference sheet in the *System Commands Reference Manual*, DSS-10004-00*)*. If there are no programs loaded into memory, or deleting files does not help, see your System Operator about increasing the size of your memory partition.

When COMPIL has finished processing your file, it returns you to monitor level and, if there are no syntax errors, writes the object program to the disk as a file bearing the name of your source program and a .RUN extension (in this case, PAYROL.RUN). If you do have errors displayed, no .RUN file will be created.

The resulting object program (.RUN file) is re-entrant, and may be loaded into system memory for use by multiple users.

When you have a .RUN file, you can execute it at any time with the RUN program (see below).

Compiler Options

The compiler has a number of optional "switches" that let you control how the compilation works.

To choose an option, include the slash symbol "/" at the end of the file specification that you supply to COMPIL, followed by the appropriate letter. If you want to add more than one switch, just add another "/" and letter. Each letter must be preceded by a slash. For example:

```
COMPIL PAYROL.BAS/O RETURN
```

or:

```
COMPIL PAYROL/O/N RETURN
```

Here are the options available:

/O Tells the compiler to take out any line number references in your compiled object code file. COMPIL will, however, still report duplicate line number errors. The /O option makes your object code file smaller and makes the program run faster, but any error messages do not include the number of the line at which the error occurred.

/T Primarily for debugging purposes. It tells the compiler to display each line of your source program as it scans that line. If a problem occurs during compilation, you can use the /T option to determine the line in which the problem occurs. You can also use /T to gauge the speed with which certain statements compile.

/N Cancels the COMPIL statistical display. You will see appropriate error messages, but none of the COMPIL statistics.

/A Increases the size of the area in memory used to calculate program addresses. Normally, the compiler uses 16 bits to calculate addresses–/A increases this to 24 bits. This increases the size of your program, but allows you to transfer control (when you use statements like GOTO, GOSUB, RESUME, etc.) across a span greater than 64K.

/M Causes an error message to be displayed any time COMPIL encounters an unmapped variable. This option aids programmers who wish to verify that all variables have been mapped. See Chapter 15 for information on mapping variables.

/I Makes old ISAM (ISAM.SYS) the default ISAM.

RUNNING A PROGRAM

To run a program enter RUN and the name of the .RUN program you want to execute, and press RETURN. For example:

```
RUN LOOP RETURN
```

You may supply a full file specification, including device name and account number. The monitor looks for the run-time package, RUN.LIT, in memory; if it is not found in system or user memory, it loads RUN into memory from the SYS: account on the disk.

RUN prepares an area in memory for your program to use, then searches for your program in memory. If it is not there, RUN loads the specified .RUN file from the disk account you specified, or from the current account if you didn't include an account number. Then RUN executes your program. Upon completion, or if you use Control-C to interrupt the program, RUN returns you to monitor level.

Do not use the monitor command RUN on files with .BAS extensions. If you try to run a .RUN file that was produced on an AMOS AM-100/T system, or any non-.RUN file, you will get the error message:

```
?RUN file is in an incompatible format
```

Chapter 6 - AlphaBASIC Variables

This chapter discusses the following topics:

- Variable Names
- Numeric Variables
- String Variables
- Array Variables

VARIABLE NAMES

An AlphaBASIC variable name may contain any number of alphanumeric characters, and is not limited to a single letter or to a letter and a digit, as in many BASIC languages. The first character of the name must be alphabetic (from a to z or A to Z), and the variable name may begin with a reserved word (unless NOEXPAND mode is set—see Chapter 2). For a list of AlphaBASIC reserved words, see Appendix B.

You may also use apostrophes in variable names to improve clarity. You may use both upper and lower case characters in your variable names. You may not use spaces within variable names.

Note that although AlphaBASIC folds reserved words to upper case, it does not translate variable names. For instance, the variable name:

```
REC'SIZE
```

is considered unique and separate from the variable name:

```
Rec'Size
```

See Chapter 2 for a discussion of how AlphaBASIC handles upper and lower case characters.

AlphaBASIC considers normal (unmapped) variables as floating point variables unless their names have a dollar sign at the end—in which case they are considered string variables.

Variables defined by a MAP statement (called mapped variables) are defined by an explicit type code and therefore do not follow the standard convention of using a dollar sign for string variables.

Mapped variables are a special form of AlphaBASIC variable that allow you to organize and use data more efficiently. For information on mapped variables, see Chapter 15.

Specify integer variables by appending a percent sign to the variable name.

The integer variable was added for compatibility reasons. However, AlphaBASIC does not perform integer arithmetic. Following a variable name with a % symbol is equivalent to using the integer function on that variable. For example, COUNTER% has the same value as INT(COUNTER).

Subscripting of array variables follows the standard conventions of other BASICs by enclosing the subscripts within parentheses.

The following are examples of legal variables:

```
A                                   A$
NUMBER                              STRING$
MASTER'INVENTORY'RECORD             HEADER1
MOM'ALWAYS'LIKED'YOU'BEST           Z1234567
NEW'ARRAY(3,3)                      Inventory'control'counter
```

NUMERIC VARIABLES

A numeric variable is one that contains numbers. The numbers in a numeric variable can be used in mathematical calculations.

The normal mode of processing numeric variables (as opposed to string variables) is in 11-digit accuracy, which might be termed "single-and-one-half" precision compared to normally accepted standards.

Integer and binary variables are also considered numeric variables, but AlphaBASIC always converts them to floating point format before any mathematical operations are done.

Numeric variables can be entered in scientific format, but will be converted to normal format unless they are larger than the number of significant digits.

The significance of the variables (how many digits will be stored and printed) is something you can control with the SIGNIFICANCE statement — from 1 to 11 digits. Normally, numbers will be rounded down to a default of 6 digits (from the 11 digits that result from any calculations). See Chapter 10 for more information on SIGNIFICANCE.

Numeric variables have an approximate range from 2.9 * 10^-39 to 1.7 * 10^38.

STRING VARIABLES

A string is any ASCII character or group of ASCII characters. String variables can contain letters, numbers, typographical symbols, and special characters. A string variable is identified by a $ after the variable name. A string literal is identified by being enclosed in quotation marks. In this example:

```
NAME$ = "Mortimer T. Smith"
```

the NAME$ to the left of the equal sign is a string variable, and the "Mortimer T. Smith" to the right of the equal sign is a string literal. A string variable cannot contain quotation marks.

AlphaBASIC supports string variables in both single and array form. The memory that is allocated for each string variable is the number of bytes representing the maximum size that the string is allowed to expand to. Each string is variable in size within this maximum limit and a null byte is stored at the end of each string to indicate its current actual size if the string is shorter than the maximum.

At the start of each compilation, the default size to be used for strings is 10 characters. By using the STRSIZ statement within the program, you can alter the string size value for all the following new string variables

String variables may be assigned values by enclosing string literals in quotes, or by taking on the value of another string variable. For example:

```
FirstName$ = "Theodred"
FirstName$ = Name'that'was'input$
```

You may join string variables together into one variable using a plus sign between two strings. For example:

```
FullName$ = FirstName$ + LastName$
```

String functions such as LEFT$, RIGHT$, MID$, etc. can be used to work with and change portions of strings or substrings. In addition, a powerful substring modifying system may be used to operate on portions of strings within expressions. Chapter 8 explains this unique option of AlphaBASIC.

Unformatted, mapped variables are also considered string variables when they are used in expressions or printed. See Chapter 15 for information on mapped, unformatted variables.

An unformatted variable may contain non-string data. If this is the case, then using the PRINT statement to display either that variable or an expression containing it will result in a very odd display, since the data is not in a printable form.

ARRAY VARIABLES

You can use both numeric and string variables in arrays. An array is a powerful way of storing and manipulating variable data. An array consists of a set of ordered variables, all under one variable name. Each array variable consists of a standard variable name:

```
NUMBER
```

plus a subscript:

```
NUMBER(1)
```

The subscript tells you which variable within the array is being worked on. For instance, when you say:

```
NUMBER(3) = 5
```

AlphaBASIC assigns the value of 5 to the 3rd element in array NUMBER. The subscript number can also be a variable, allowing you a lot of flexibility in handling arrays. Here is a small program that fills an array with values:

```
For COUNT = 1 to 10
     NUMBER(COUNT) = COUNT * COUNT
Next COUNT
```

Using the loop control variable COUNT (we'll explain For-Next loops later on) as the subscript variable allows us to fill the array with values. The first time through the loop, COUNT has the value of 1, so NUMBER(COUNT) = 1. The second time, COUNT = 2, so NUMBER(COUNT) = 4 [2 times 2]. And so on.

When you use an array variable, a certain amount of memory space is allocated for that variable and its subscripts. The default value is 10; that is, you can have up to 10 subscripts. If you want to have more than 10, or you don't need that many, you can change the size by using a DIM (dimension) statement. For example, to set the array NUMBER to hold 20 values, place this statement in the program BEFORE you use the array variable:

```
DIM NUMBER(20)
```

You can set each array variable to whatever size you need. It is a good idea to set up the array without much "overhead." Dimensioning an array for more subscripts than you need can use up valuable memory space. Once an array has been dimensioned by a DIM statement, it may not be redimensioned by a subsequent DIM statement in the same program.

You can also make multi-dimension arrays. For instance, a double-subscripted array would be referred to like this:

`NUMBER(1,3)` or: `NUMBER(X,Y)`

Arrays may be any number of levels deep but practicality dictates some reasonable limit of 20 or so. The amount of memory that you have will dictate how much data you can store in an array.

At no time may the number of subscripts vary in any of the references to an array. For example, you cannot define an array as:

```
A$(10,20)
```

and then refer to it in this manner:

```
READ A$(1,2,5)
```

The number of subscripts in each element reference must also match the number of subscripts in the corresponding DIM statement which defined the array size. See Chapter 10 for more information on the DIM statement.

Chapter 7 - AlphaBASIC Expressions

This chapter discusses:

- What is an expression?
- How to use mathematical expressions
- The precedence of mathematical operators
- Mode independence

WHAT IS AN EXPRESSION?

An expression can contain variables, constant values, operator symbols, functions, or any combination of the above. When we are talking about expressions, we mean a section of AlphaBASIC code that is treated as a unit within an AlphaBASIC statement. For example, in the statement:

```
IF (TOTAL'INVENTORY - SUPPLIES'USED) < 100 &
     THEN GOTO AUTO'ORDER'ROUTINE
```

(TOTAL'INVENTORY - SUPPLIES'USED) < 100 is an expression. If the expression evaluates to a true condition, then the instructions in the THEN clause will be carried out. Even though it is made up of two variables, a number, and two operators, it can be seen as one logical unit in the way it functions within the IF - THEN statement.

MATHEMATICAL EXPRESSIONS

A mathematical expression is any expression that contains numbers and mathematical operators. For example:

```
1 + (FIX(TOTAL'RECS * REC'SIZE)/512)
```

Parentheses can be used to tell AlphaBASIC which operation to perform first. If no parentheses are included, AlphaBASIC follows its rules of operator precedence (explained below) to determine which part of the expression will be performed first.

AlphaBASIC recognizes the following mathematical operators:

+	addition, or unary plus	=	equal
-	subtraction, or unary minus	<	less than
*	multiplication	>	greater than
/	division	<>	unequal
^	raise to power	><	unequal
**	raise to power	#	unequal
"	string literal	<=	less than or equal
NOT	logical NOT	=<	less than or equal
AND	logical AND	>=	greater than or equal
OR	logical OR	=>	greater than or equal
USING	expression formatting	EQV	logical equivalence
XOR	logical XOR	MIN	minimum value
MAX	maximum value		

AlphaBASIC automatically evaluates expressions for you. For example, consider the statement:

```
PRINT (32 * 100/2 MAX 25 + 30/54)
```

First, AlphaBASIC computes the division operations. It determines that 100/2 equals 50, and that 30/54 equals .5556. Then it multiplies 32 by 50 to get1600. Next it adds 25 and .5556 to get 25.5556. Finally, AlphaBASIC applies the MAX operator to the parts of the expression (i.e., 1600 MAX 25.5556), and returns the greater or maximum value of the two sub-expressions, 1600.

LOGICAL OPERATORS

The logical operators, AND, OR, XOR, EQV and NOT, are used in IF-THEN statements to evaluate and compare expressions. The AND statement compares two expressions. If both expressions are TRUE, the IF condition is also TRUE. If either or both statements are FALSE, the IF condition is FALSE. For example:

```
QUESTION:
      INPUT "Enter a number between 0 and 100: ", NUMBER
      IF NUMBER > 0 AND NUMBER < 100 &
            THEN GOTO START &
                  ELSE PRINT "Number incorrect!" &
                        : GOTO QUESTION
START:
```

If either of the expressions (NUMBER > 0 or NUMBER < 100) are FALSE, the program above will go back and ask the question again. We can demonstrate OR with a similar section of code:

```
QUESTION:
      INPUT "Enter a number between 0 and 100: ", NUMBER
      IF NUMBER < 1 OR NUMBER > 99 &
            THEN PRINT "Number incorrect!" : GOTO QUESTION
```

With OR, the IF statement is TRUE if any one of the expressions is TRUE.

With XOR, which stands for Exclusive-OR, the IF statement is TRUE only if one of the expressions is TRUE. If more than one is TRUE, or all are FALSE, the IF statement is FALSE.

EQV, which stand for equivalence, causes the IF statement to be TRUE if both expressions evaluate to the same condition (TRUE or FALSE).

NOT negates the value of an expression. For example, the following statements are equivalent:

```
IF (I = 1) THEN GOTO WRONG'ANSWER

IF NOT(I <> 1) THEN GOTO WRONG'ANSWER
```

OPERATOR PRECEDENCE

The precedence of operators determines the sequence in which mathematical operations are performed when evaluating an expression that does not have overriding parentheses to dictate hierarchies. We showed you an example of it above in "Mathematical Expressions." Here are the rules of precedence that AlphaBASIC uses:

1. exponentiation
2. unary plus and minus
3. multiplication and division
4. addition and subtraction
5. relational operations (comparisons)
6. logical NOT
7. logical AND, OR, XOR, EQV, MIN, MAX
8. USING

The USING operator allows you to format numeric or string data using a format string. For information on USING, see Chapter 13.

MODE INDEPENDENCE

Expressions may contain any mixture of variable types and constants in any arrangement. AlphaBASIC performs automatic string and numeric conversions as necessary to ensure that the result is in the proper format. For example, if two strings are multiplied together they are first automatically converted to numeric format before the multiplication takes place. If the result is then to become a string, it is reconverted back to string format before the assignment is performed. In other words, the statement

```
A$ = B$ * "345"
```

is perfectly legal and will work correctly, as long as B$ contains a number (and no alphabetic characters). This is a powerful feature which can save programming effort when used correctly.

There is a seemingly ambiguous situation which arises from this mode independence. The plus symbol (+) is used both as an addition operator for numeric operations and as a concatenation operator for string operations. The value of 34 + 5 is equal to 39 but the value of "34" + "5" is equal to the string "345".

The operation of the plus symbol is unambiguous, but may take a little thought to figure out in a given situation. A few examples might help:

If the first operand is numeric and the second is string, we convert the second to numeric form and perform addition:

```
34 + "5" = 39
```

If the first operand is string and the second operand is numeric, we convert the second to string and perform concatenation:

```
"34" + 5 = "345"
```

The above two examples apply only when the compiler is not "expecting" a particular type of variable or term. This generally occurs only in a PRINT expression such as PRINT "34" + 5. At other times, the compiler expects a specific type of variable; the conversion of the first variable is then performed prior to inspecting the operator (plus sign). What the plus sign will do will then depend upon that first variable—if the variable is a numeric one, the plus sign will perform addition; if string, it will concatenate. In the following example:

```
NUM = 5 * "34" + 4
```

The multiplication operator (*) forces us to expect a numeric term to follow. The "34" string is therefore immediately converted to numeric 34 and multiplied by the 5. The plus sign then performs numeric addition instead of concatenation. The result is in numeric format and is converted to string format if its destination is a string.

The following are a few examples as they would be seen if you were to use them in an actual program:

A = 34 + 5	B = 34 + "5"	C = "34" + 5	D = "34" + "5"
A$ = 34 + 5	B$ = 34 + "5"	C$ = "34" + 5	D$ = "34" + "5"

```
PRINT A,B,C,D          :     PRINT A$,B$,C$,D$

39          39          39          39
345         345         345         345
```

You can see that conversion is affected by the type of variable being used.

You might like to try a few examples of your own on your system to see what the results are. Any potentially ambiguous expression may always be forced to one type or the other by use of the STR(X) and VAL(A$) functions (see Chapter 11).

For more examples of mode independence, see the sample programs in Chapter 8.

Chapter 8 - Working With String Variables (Substring Modifiers)

This chapter discusses:

- What a substring modifier is
- How to use a substring modifier
- Examples of substring modifiers

WHAT IS A SUBSTRING MODIFIER?

We talked about what a string variable was in Chapter 6. These strings of characters can also be broken down into substrings.

A substring is a portion of an existing string, and may be as small as a single character or as large as the entire string. AlphaBASIC supports a unique method of manipulating substrings using substring modifiers.

Substring modifiers allow the substring to be defined in terms of character positions within the string, relative to either the left or right end of the string.

The length of the substring is defined either in terms of its beginning and ending positions or in terms of its beginning position and its length. You will define a substring by using the string variable (for example, A$) followed by the substring modifier. The substring modifier is two numeric arguments enclosed within square brackets (for example, A$[1,3]).

HOW DO YOU USE A SUBSTRING MODIFIER?

The substring modifier takes on two distinct formats:

[beginning-position , ending-position]

[beginning-position ; substring-length]

The first format defines the substring in terms of its beginning and ending positions within the string and uses a comma to separate the two arguments. The second format defines the substring in terms of its beginning position within the string and its length, using a semicolon to separate the arguments.

The beginning and ending positions are defined as character positions within the string relative to either the left or right end. A positive value represents the character position relative to the left end of the string, with character position 1 representing the first (leftmost) position.

A negative value represents the character position relative to the right end of the string, with character position -1 representing the last (rightmost) position. For example, assume a string has the letters ABCDEF in it. The positions are defined in terms of positions 1 through 6 (left-relative) or positions -1 through -6 (right-relative).

A	B	C	D	E	F	(6 characters within main string)
1	2	3	4	5	6	(left-relative position values)
-6	-5	-4	-3	-2	-1	(right-relative position values)

Allowing negative values for right-relative positions provides the ability to pick out digits within a numeric string without having to calculate the total size of the string first and then working from the left.

The substring-length argument used by the second format may also take on negative values for a more flexible format. Normally the length is a positive value which represents the number of characters counting the beginning position and counting to the right. A negative length causes the index to move to the left and returns a substring whose last character is the one marked by the beginning-position argument.

EXAMPLES OF SUBSTRING MODIFIERS

```
If:    A$          =  "ABCDEF"

Then:  A$[2,4]     =  "BCD"
       A$[-3,-2]   =  "DE"
       A$[3,-2]    =  "CDE"
       A$[3;-2]    =  "BC"
       A$[-3;-2]   =  "CD"
       A$[4;1]     =  "D"
```

For example, A$[3,-2] tells AlphaBASIC to return the substring that begins at character position 3 (from the left) and ends with character position 2 (from the right); that is, to return all characters between C and E, inclusive.

A$[3;-2], however, tells AlphaBASIC to return the substring that begins with character position 3 (from the left) and extends 2 character positions toward the left; that is, to return all characters starting with C and working backward two positions to B, inclusive.

Any position values or length values which would cause the substring to overflow out of either end of the main string are truncated at the string end. For example:

```
A$[3,10] = "CDEF"          A$[-14,34] = "ABCDEF"
```

The main string to which the substring modifier is applied is actually any expression and does not need to be a defined single string variable. For example:

```
ANSWER$ = (A$ + B$ + C$)[2;10]
ANSWER$ = ("ABLE" + A$ + "QQ34")[4,10]
```

The mode independence feature allows substring modifiers to be applied to numeric expressions. See Chapter 7 for information on mode independence. A string is returned, but if the destination is a numeric variable, another conversion is made on the substring to return a numeric value.

```
INPUT "Enter two numbers: ",NUMBER,NUMBER2
SUM = NUMBER + NUMBER2
? NUMBER;" + ";NUMBER2;"= ";SUM    ! Test rightmost digit:
IF SUM[-1;1] = 0 THEN PRINT "Divisible by 5 and 2"
IF SUM[-1;1] = 5 THEN PRINT "Divisible by 5"
```

Be sure you understand the concept of mode independence before you begin to use substring modifiers or you may get answers you don't expect. For example, the second and third lines in the small program below return different answers, even though the subscripting is performed exactly the same in both cases. This is because the mode independence feature examines the data type of the destination variable before allowing any operations to be performed.

When it scans the second line, AlphaBASIC knows that a string result is expected (because STRING$ is a string variable), and so reads the "+" symbol as a string concatenation operator. In the third line, BASIC knows that a numeric result is expected (because NUMERIC is a numeric variable), and so reads the "+" symbol as an addition operator.

```
VALUE1$ = "123" : VALUE2$ = "456"
STRING$ = (VALUE1$ + VALUE2$)[1;3]
NUMERIC = (VALUE1$ + VALUE2$)[1;3]
PRINT "NUMERIC= ";NUMERIC,"STRING$= ";STRING$
```

The program above prints:

```
NUMERIC = 579         STRING$ = 123
```

You may apply substring modifiers to subscripted variables or expressions containing subscripted variables. Be careful not to confuse substring modifiers with subscripted variables. For example:

`A$(2,3)`	designates a location in array A$
`A$[2,3]`	designates a substring of string A$
`A$(1,5)[2,3]`	designates a substring in location A$(1,5)

These are valid uses of the substring modifiers:

```
Q$ = A$(1,9)[2,5]     Q$ = (A$(1) + B$(3))[-5,3]
```

Substring modifiers return a string value. These may be used as part of string expressions. For example:

```
Q$ = A$ + B$[2;5] + (A$[2,2] + C$)[-5;-3]
```

You may apply substring modifiers to the left side of an assignment in order to alter a substring within a string variable. Only that portion of the string defined by the substring modifier is changed. The other characters in the string are not altered. If A$ contains "ABCDEF", then A$[2,4]="QRS" causes A$ to contain the string "AQRSEF". This procedure may not be applied to numeric variables (for example, A[3;2] = "23" is not valid).

Chapter 9 - Data Formats

This chapter discusses the various forms which your data may take:

- Floating point
- String
- Binary
- Integer
- Unformatted

Note that if you do not use MAP statements (explained in Chapter 15) to define your data, your variables may only take on floating point numeric values or string values.

If you use MAP statements, however, you have a great deal more versatility in the format of your data, and can define binary and unformatted data as well. MAP statements also give you a way to define powerful hierarchical data structures that allow sophisticated data manipulation. Chapter 15 also discusses how AlphaBASIC assigns memory locations to data.

FLOATING POINT FORMAT

All numeric variables are assigned floating point format unless specified otherwise in the program. The standard precision in use by the Alpha Micro system can be called "single-and-one-half," since it lies midway between what are known as single precision and double precision formats.

The reason for this is that floating point numbers occupy six bytes of storage. Of the 48 bits in use for each 6-byte variable, the high order bit is the sign of the mantissa.

The next 8 bits represent the signed exponent in excess-128 notation, giving a range of approximately 2.9*10^-39 through 1.7*10^38. The remaining 39 bits contain the mantissa, which is normalized with an implied high-order bit of one. This gives an effective 40-bit mantissa which results in an accuracy of 11 significant digits.

STRING FORMAT

The string format is used for the storage of alphanumeric text data. String variables require one byte of storage for each character and may be fixed in position in memory using the memory mapping system. If a string is shorter than the maximum length defined for that string, a null byte is stored following the last character to terminate the string.

When AlphaBASIC compares a string of spaces and a null (empty) string, it sees them as equal. This is by design and demonstrates how AlphaBASIC compares strings. If two strings are of equal length, AlphaBASIC compares the strings on a character-to-character basis.

If they are of different lengths, AlphaBASIC pads the shorter of the two with spaces until the strings are of equal length, and the comparison proceeds. For example, the string "PAST DUE" is equal to the string "PAST DUE ".

As you can see, using this algorithm causes a null string to be treated as a string of spaces during comparison. The proper way to check for a null string is to use the LEN function, rather than to see if it is equal to "". If LEN(string-variable) returns a zero, the string is null (empty).

BINARY FORMAT

Binary variables are specified by using MAP statements, and are similar to integer variables in other implementations of BASIC. A binary variable may be from 1 to 5 bytes in length and may be signed when all 5 bytes are specified.

When less than 5 bytes are specified in a MAP statement as the length, the binary value may be loaded as a negative number, but it is always returned as a positive number of full magnitude.

The upper bit (preloaded as the sign) takes on its specific value in the equivalent positive binary variable. For instance, a 1-byte binary may be loaded with positive numbers from 0 through 255 (decimal), or negative numbers from -1 through -128, but the negative numbers are returned as the positive values of 255 through 128 respectively. Only 5-byte binary variables return the original sign and value when loaded with a negative number.

Binary variables may be used in expressions but they are slower than floating point variables because they are always converted first to floating point format before any mathematical operations are performed on them.

Binary variables are useful in integer and logical (Boolean) operations or for storing values in small amounts of memory (floating point numbers always take 6 bytes of memory regardless of their values).

All logical operations performed within expressions (AND, OR, XOR, NOT etc.) cause the values to be converted first to signed 5-byte binary format before the logical operation is performed. The value -1 represents a 40-bit mask of all ones. Any relational comparison between two expressions or variables returns -1 if true, or 0 if false.

INTEGER FORMAT

Integer variables and constants are specified by appending a percent sign (%) to the variable name, which is the standard convention in use by other BASICs. AlphaBASIC generates floating point variables and performs automatic integer truncation for all integer variables specified in this manner.

Integer constants are generated as their equivalent floating point values and are included only for compatibility with existing program structures. Since integer variables are effectively floating point

variables with an additional INT(X) conversion performed, they are actually slower than regular floating point variables.

This is the opposite of most other BASICs, which usually store integer variables as 2-byte signed values and perform special integer arithmetic on them. True integer variables may be defined by using the MAP statement and the "B" binary type code. See "Type Code" in Chapter 15 for information.

UNFORMATTED

Unformatted data must be specified using a MAP statement. An unformatted numeric variable defines a fixed size area of storage used to contain absolute unformatted data. This data can be of any of the above types.

Unformatted data is normally used in the mapping system to define contiguous storage which is subdivided into multiple variables of different types. No conversion ever takes place when moving data to and from this format. Unformatted variables are treated as string variables when used in expressions.

Chapter 10 - AlphaBASIC Program Statements

An AlphaBASIC source program contains statements which are executed in sequence, one at a time, as AlphaBASIC encounters them. Each of these statements normally starts with a statement followed by optional variables or statement modifiers. Many of these statements can also be used in interactive mode as direct statements.

This chapter lists most of the program statements and gives some examples for clarity. The program statements not discussed in this chapter are the statements dealing with file input/output and ISAM files. See Chapters 16 and 20, respectively, for those statements. See also the AlphaBASIC Quick Reference Card for the formats of all AlphaBASIC statements and commands.

CALL

See GOSUB, below.

CHAIN

The format is:

```
CHAIN filespec
```

Where the filespec is a quoted string literal or a string variable. The filespec may take the forms:

```
{Devn:}AlphaBASIC .RUN-file-name{[p,pn]}
{Devn:}monitor-command-name{[p,pn]}{arguments}
{Devn:}.CMD-file-name{[p,pn]}
{Devn:}.DO-file-name{[p,pn]}{arguments}
```

The CHAIN statement causes control to be passed to the specified AlphaBASIC program, command file, or monitor command program. The program name may be a full file specification, including device and account specifications. Certain AMOS commands and DO files can have various arguments specified with them—the way you would enter the command at AMOS level is the way you enter it after the CHAIN statement.

The CHAIN statement causes the current program to be cleared from memory. The specified file is then located and executed from the beginning.

A chained AlphaBASIC program must be a fully compiled program with the extension .RUN in order to be referenced by the CHAIN command. It may be in user memory (having previously been loaded with the monitor LOAD command) or it may be in system memory (the System Operator may place a file in system memory by modifying the system initialization command file).

If it is not already in memory, it is loaded from the specified disk account into user memory and then executed. If it cannot be located, you are returned to monitor level with the error message:

```
?Cannot find program NAME.RUN
```

Examples of the CHAIN statement:

```
CHAIN "PAYROL"
70 CHAIN "DSK1:ACTRUN.CMD[100,7]"
```

There is no provision to start the chained file at any point other than the beginning. You may pass common variables between chained AlphaBASIC programs either by writing them out to a file and then having the chained program read them back in, or by using the COMMON assembly language subroutine. For information on COMMON, see the *AlphaBASIC XCALL Subroutine User's Manual* that applies to your system.

For more information on CHAIN, see Chapter 17.

DATA

See READ, RESTORE, and DATA below.

DIM

The format is:

```
DIM variable1(expr1...,exprN){,variableN(expr1...,exprN)}
```

The dimension statement defines an array which is allocated dynamically at execution time. Once allocated, an array cannot be redimensioned during the execution of the program. DIM statements are not used with mapped variable arrays.

There is no set limit to the number of subscripts that may be used to define the individual levels within the array—the size of your program and the amount of memory you have will determine how many levels you can have. In most cases, you will be able to define many more levels than you actually would need.

The statement DIM A(20) defines an array with 20 elements, referenced as A(1) through A(20). Multiple arrays may be dimensioned by a single DIM statement by separating them with commas. For example:

```
DIM A(20),B(15),C(10)
```

Subscripts are evaluated at execution time and not at compile time, thereby allowing variables as well as numeric constants to be used as subscripts. The statement DIM A(B,C) allocates an array whose size depends on the actual values of B and C at the time the DIM statement is executed.

If a reference to an array is made during program execution without a previous DIM statement to define the array, AlphaBASIC assigns a default array size of 10 elements for each subscript level.

String arrays may be allocated, such as DIM A$(5). The size of the array depends on the current string size in effect as specified by the last STRSIZ command (or the default of 10), since each element in the array must be this number of bytes, or less.

For instance, if the current STRSIZ is 10, the statement DIM A$(5) would allocate 5 elements * 10 bytes per element, or 50 bytes of memory for the array. Below are some examples of valid DIM statements:

```
DIM A(10)            TEST(A,B*4)
DIM A(B(4))          C(8,8), C$(10,4)
```

For more information on arrays, see Chapter 6.

ELSE

See IF, THEN, ELSE below.

END

The format is:

```
END
```

This statement causes the program to terminate execution. The END statement does not terminate compilation of the program nor is it required at the end of the program. If other program statements follow the end of the program (e.g., subroutines), terminating the program with END prevents your program from incorrectly entering those statements and executing them.

FOR, TO, NEXT AND STEP

The format is:

```
FOR control-variable = expr TO expr {STEP {-}expr}
          …
   {Statements}
          …
NEXT {control-variable}
```

The FOR and NEXT statements initialize and control program loops. A loop is a structure in which the same statement or statements can be performed a specific number of times. Whether or not a loop is executed depends upon the value of a special "control-variable." The control-variable used may even be subscripted. The control-variable MUST be a floating point variable.

The delimiters (expr) may be any valid expression. FOR initializes the control-variable to the first expression.

NEXT increments or decrements the value of the control-variable each subsequent loop. The control-variable name may be omitted in the NEXT statement, in which case the control-variable of the previous

FOR statement is the one that is incremented. The control-variable is incremented or decremented in units indicated by the STEP statement (if STEP is used).

If no STEP modifier is used, the step value is assumed to be a positive 1. Unlike some other BASICs, an AlphaBASIC FOR-NEXT loop will always be performed at least once, even if you specify:

```
FOR I = 0 TO 0
```

FOR and NEXT statements are illegal as direct statements except when they are incorporated into the same multi-statement line. For example, this line is valid:

```
FOR I = 1 TO 10 : PRINT I : NEXT I
```

Here are some samples of the different forms FOR-NEXT loops may take:

```
PRINT "The COUNTER ";
FOR COUNTER = 1 TO 10
IF COUNTER/2 = INT(COUNTER/2) &
   THEN PRINT COUNTER "is even." &
      ELSE PRINT COUNTER;"is odd."
NEXT COUNTER

10  ? "Enter the date of the first Sunday in the month: ";
20  INPUT DAY
30  PRINT "The Sundays this month are on these dates: ";DAY
40  FOR A = DAY + 7 TO 31 STEP 7
50  PRINT A : NEXT A

FOR I = 10 TO 1 STEP -1
   PRINT I
NEXT
```

Loops within loops are legal and are called "nested" loops. Loops may be nested to many levels. Each time the outermost loop is incremented (or decremented) once, the loop nested within it is executed from beginning to end. During the execution of the second loop, the third loop (if any) is fully executed each time the second variable is incremented, and so on, for each nested loop in the series. For example:

```
DIM MATRIX(5,5)
   FOR I = 1 TO 5
      FOR J = 1 TO 5
         MATRIX(I,J) = I - J     ! Nested loops
         PRINT MATRIX(I,J);
      NEXT J
      PRINT
   NEXT I
```

It is not good programming practice to branch out of a loop before its completion (for example, by using a GOTO statement). A cleaner way of exiting a loop is simply to set the control-variable to the terminal value specified in the FOR statement. For example:

```
START'LOOP:

   FOR I = 1 TO 10
      INPUT "Enter number of pennies: ",PENNIES
                              ! Don't leave the loop:
      IF PENNIES < 0 GOTO NEGATIVE'VALUE
      PRINT "You have ";PENNIES/100;" dollars."
      GOTO END'LOOP
!  If # < 0, print message, set I to 10
      NEGATIVE'VALUE:
      PRINT "You can't have negative pennies!"
      I = 10                  ! Set I to loop maximum value
   END'LOOP:
NEXT I                        ! If I = 10, we're all done.
```

GOSUB (OR CALL) AND RETURN

The formats are:

```
GOSUB label or line number
CALL label or line number

RETURN
```

Calls a subroutine which starts at the line number or label referenced by the GOSUB or CALL statements. The subroutine returns to the statement immediately after the GOSUB or CALL when the RETURN statement is encountered. Executing a RETURN statement without first executing a GOSUB statement results in an error message.

Both GOSUB and RETURN are illegal as direct statements. Note that the CALL statement is merely another way of specifying GOSUB for those programmers used to this syntax from other versions of BASIC or other programming languages.

It is often the case that you want to perform the same operation at various points within your program. A subroutine is a set of program statements that you may execute more than once simply by including a call for that subroutine within your program at the point where you would like to execute the routine. For example:

```
   !  This program contains a subroutine that validates
   !  numeric entries to make sure that they are greater
   !  than 0 and are less than 100.

PRINT "We're going to perform several mathematical "operations."
RE'START:
   PRINT "Your entries must be greater than 0 and less than
100."
   PRINT : INPUT "Enter two numbers to be added: ",A,B
   GOSUB VALIDATE  ! Check to make sure numbers are valid.
   IF FLAG = 1 THEN GOTO RE'START     ! Check error flag
   PRINT A;" + ";B;" = ";A + B
   PRINT : INPUT "Enter two numbers to be subtracted: ",A,B
   GOSUB VALIDATE  ! Check to make sure numbers are valid.
```

```
   IF FLAG = 1 THEN GOTO RE'START
   PRINT A;" - ";B;" = ";A - B
   PRINT : INPUT "Enter two numbers to be divided: ",A,B
   GOSUB VALIDATE  ! Check to make sure numbers are valid.
   IF FLAG = 1 THEN GOTO RE'START
   PRINT A;" / ";B;" = ";A/B
   PRINT : PRINT "Th-th-th-that's all, folks!"
   END

VALIDATE:            ! Subroutine to validate the data

   IF A <= 0 OR B <= 0 &
      OR A >= 100 OR B >= 100 &
         THEN PRINT &
            "Error - incorrect number entered in input!" &
            : FLAG = 1 &
         ELSE FLAG = 0
   RETURN
```

Remember that & (ampersand) is the symbol for a continuation line.

Note that we included an END statement to separate the main program from our subroutine; otherwise, AlphaBASIC executes the VALIDATE subroutine again after printing out the end message, and we would get a "RETURN without GOSUB" error message.

Also note that the use of GOSUBs helps to modularize your programs, and thus makes them easier to design and maintain. Even before you completely "flesh out" your programs, you can insert dummy routines that will later contain complete code. For example:

```
!  This program is an example of a dental package.
PRINT "Welcome to the Dr. Plak Rental Dental Package."
PRINT
GOSUB INIT      ! Perform initialization of data files
GOSUB MENU      ! Ask user to pick function from main menu.
GOSUB DAY'END   ! Do End-of-day Processing
GOSUB FINISH'UP ! Finish up, close files, and exit.
END
                ! The subroutines start here.

   INIT:
      PRINT "This section will initialize files."
   RETURN

   MENU:
      PRINT "This section will display the main menu and"
      PRINT "ask user for selections."
   RETURN

DAY'END:
   PRINT "This section will perform day-end processing."
RETURN
```

```
FINISH'UP:
   PRINT "This section will close files and clean up final PRINT
"data."
RETURN
```

You can nest subroutines. For example:

```
PRINT "Main Program:"
GOSUB OUTER'MOST  ! OUTERMOST calls NEXTMOST and INNERMOST
PRINT "Return from Outermost"
END

OUTER'MOST:
   PRINT " Outermost subroutine"
   GOSUB NEXT'MOST
   PRINT "    Return from Nextmost"
   RETURN
      NEXT'MOST:
         PRINT " Nextmost subroutine"
         GOSUB INNER'MOST
         PRINT "      Return from Innermost"
      RETURN
         INNER'MOST:
            PRINT "      Innermost subroutine"
         RETURN
```

The program above prints:

```
Main Program:
      Outermost subroutine
        Nextmost subroutine
          Innermost subroutine
          Return from Innermost
        Return from Nextmost
      Return from Outermost
```

It is good programming form to exit a subroutine by using the RETURN statement for that subroutine rather than using a GOTO statement.

GOTO

The format is:

```
GOTO label or line number
```

or: `GO TO label or line number`

The GOTO statement transfers execution of the program to a new program line. This program line must be identified either by a line number or a label somewhere in the program. You may use GOTOs to transfer control to a program line that is either before or after the program line containing the GOTO statement itself. For example:

```
! Program to demonstrate use of GOTOs.

PRINT "This program computes your account balance.  Use"
PRINT "^C to stop; enter deposits as negative amounts."
INPUT "Enter old account balance: ",BALANCE

CALCULATE'BALANCE:
   PRINT : INPUT "Enter debit amount: ",DEBIT
   BALANCE = BALANCE - DEBIT
   PRINT "Your debit was :";DEBIT
   PRINT "Your current balance is :";BALANCE
GOTO CALCULATE'BALANCE
```

You can see that there is an endless loop in which control is eternally transferred between `GOTO CALCULATE'BALANCE` and `CALCULATE'BALANCE`. The program above will continue to run until you use CTRL /**C**.

If you use GOTOs on a multi-statement line, remember to place it last on the line; any statements after the GOTO will never get executed. For example:

```
NET = GROSS - DEDUCTION : GOTO GET'TAX : PRINT DEDUCTION
```

the last statement, `PRINT DEDUCTION`, can never be executed.

IF, THEN AND ELSE

The format is:

```
IF expression {THEN}{statement}{label/line#}
   {ELSE{ statement }{ label/line#}}
```

The IF statement evaluates the expression, and conditionally performs the specified operations in response to that evaluation.

A multi-statement line may take the place of a single statement in an IF-THEN statement. For example:

```
IF A > 0 THEN PRINT "The number is: ";A &
   :  PRINT "The number is greater than zero." &
      ELSE PRINT &
         "The answer is less than or equal to zero."
```

If A equals three, the statement above prints:

```
The number is: 3
The number is greater than zero.
```

If A is not greater than zero, it prints:

```
The answer is less than or equal to zero.
```

The conditional processing features in AlphaBASIC give a wide variety of formats. Some of the format combinations that are acceptable are:

```
IF expression THEN line#
IF expression THEN GOTO line#/label
IF expression GOTO line#/ label
IF expression THEN line# ELSE GOTO line#/label
IF expression THEN GOTO line#/label &
   ELSE GOTO line#/label
IF expression THEN statement
IF expression statement
IF expression statement ELSE statement
IF expression THEN statement ELSE statement
```

Notice from the examples above that you may sometimes omit the GOTO keyword when transferring control to another program location. You may NOT omit the GOTO keyword when:

1. You are referring to a label; or:
2. When you are in an ELSE clause.

You may often omit the THEN keyword also. You cannot omit the THEN if you are transferring control to another area (a line number or label) and you do not have a GOTO statement. In other words, a transfer of control must have either a THEN or a GOTO statement.

The above formats may be nested to any depth and, rather than go into detail, we suggest that you experiment with them to determine the actual restrictions that exist. Some examples:

```
IF A = 5 THEN GOTO PROGRAM'EXIT
IF A = 5 GOTO PROGRAM'EXIT
IF A > 14 THEN 110 ELSE GOTO 220
IF B$ = "END" PRINT "END OF TEST"
IF TOTAL > 14.5 GOTO START
IF P = 5 AND Q = 6 IF R = 7 PRINT 567 &
   ELSE PRINT 56 ELSE PRINT "NONE"
IF A = 1 PRINT 1 ELSE IF B = 2 &
   THEN 335 ELSE GOTO 345
IF A AND B THEN PRINT "A and B are nonzero."
```

Note that the expression evaluated by the IF statement usually contains relative operators (e.g., IF A = B; IF A > 0; etc.). However, it may be any legal expression. For example:

```
A = 0: B = 1
IF B THEN PRINT "B is not zero."
IF (B AND A) PRINT "nonzero numbers" &
   ELSE PRINT "at least one zero number."
```

INPUT

The format is:

```
INPUT {"prompt-string",}var1{,var2...,varN}
```

Allows data to be entered from your terminal and loaded into specific variables at run-time. The INPUT statement contains one or more variables separated by commas. If you omit the optional prompt string, AlphaBASIC displays a question mark on the terminal display to signal a request for data entry. For example:

```
INPUT A
PRINT A
```

when the program is run, you see:

```
? 5 RETURN
5
```

If you provide the prompt string, AlphaBASIC displays it instead of a question mark to prompt the user of your program for data. If you wish to have no prompt at all, use a null prompt string, as in: `INPUT "",A$,B$`. Your prompt string must be in the form of a string literal; that is, it must be enclosed with quotation marks. For example:

```
INPUT "Enter your account number: ",ACCOUNT'NUM
```

prints:

```
Enter your account number:
```

You may specify both numeric and string variables in an INPUT statement. A numeric variable should be in floating point format. String variables should be in ASCII format. Some examples of valid INPUT statements are:

```
INPUT A
INPUT "Enter account #, name, and age: ",ACCOUNT,NAME$,AGE
INPUT "",A,B,C
INPUT "Enter a positive number:",NUMBER
INPUT Q(8)
```

If you specify multiple variables in the INPUT statement, you are expected to enter multiple items of data. If the data being entered is numeric, you may separate data items with either commas or spaces. If the data being entered is a string, you must separate data items with commas.

If you mix floating point and string input, you must use commas to separate the data being input. For example, if A, B, and C are numeric variables and D$ and E$ are string variables, consider the following legal examples:

```
INPUT A,B,C
? 1,2,3 RETURN

INPUT A,B,C
? 1 2 3 RETURN

INPUT A,B,D$,C
? 1,2,DAY,3 RETURN
```

```
INPUT D$,E$
? DAY,MONTH RETURN
```

For information on the statement to use if you want to enter strings that contain commas, quotes, and other special characters, see "INPUT LINE" below.

When you use INPUT, remember the default size of unmapped string variables is ten bytes; if you want to use strings larger than that, use the STRSIZ statement to reset the default string size or use a MAP statement for the string variable, specifying the size you need. See below for information on STRSIZ.

If a user of your program does not enter as many items of data as are expected by the variables in the INPUT statement, AlphaBASIC displays a double question mark to ask for more. For example:

```
INPUT A,B,C
? 1,2 RETURN
?? 3 RETURN
```

The direct statement above asks for three items of numeric data. Because we only entered two values, AlphaBASIC responded with a "??" symbol to ask for the third value.

Be careful to correctly enter the type of data the variables in the INPUT statement expect. If you enter a string for a numeric variable, AlphaBASIC sets that variable to zero. For example:

```
INPUT A1
? ME RETURN
PRINT A1
0
```

If you enter a numeric value for a string variable, AlphaBASIC accepts it a string. Therefore, your programs should make sure the correct data has been entered.

If a value has not been assigned to a variable, AlphaBASIC assumes the variable contains a zero (if a numeric variable) or a null (if a string variable). If you press RETURN or use CTRL /**C** in response to an INPUT statement request for data, AlphaBASIC leaves the variable being input set to a zero or null (if a value has not yet been assigned) or to the value previously assigned to the variable. For example:

```
A = 3 RETURN
INPUT A RETURN
RETURN
? PRINT A RETURN
3
```

If you press RETURN or use CTRL /**C** in response to a data request, and the INPUT statement contains several variables, AlphaBASIC skips over any variables remaining in the INPUT statement, leaving their values unchanged. An example might help to clarify:

```
START:
   INPUT "Enter day, month, year: ",DAY,MONTH,YEAR
   PRINT "Day:";DAY,"Month:";MONTH,"Year:";YEAR
   PRINT : GOTO START

 RUN DAY [RETURN]

Enter day, month, year: 21,4 [RETURN]
?? [RETURN]
Day: 21    Month: 4   Year: 0

Enter day, month, year: 8 [RETURN]
?? [RETURN]
Day: 8     Month: 4   Year: 0

Enter day, month, year: 31,12,1980 [RETURN]
Day: 31    Month: 12  Year: 1980

Enter day, month, year: [CTRL] /C

Operator interrupt in DAY.BAS
```

You may also use the INPUT statement to read data from sequential files. For more information on this use of the statement, see Chapter 16.

INPUT skips over nulls in data, and just waits for the next character. This is important to know if you plan to input from devices.

INPUT LINE

The format is:

```
INPUT LINE {"prompt-string",}variable
```

Although you may specify a numeric variable, the real purpose of INPUT LINE is to allow you to enter string data from your terminal that includes commas, quotation marks, blanks, and other special characters.

You will usually want to use INPUT (see the section above) for inputting numeric data or multiple items of string data.

INPUT LINE loads into the specified string variable an entire line of data, up to but not including the first LINE FEED character it encounters. It will remove a Carriage Return character if one immediately precedes the LINE FEED character, but not if it is embedded in the line. Therefore, *do not* specify more than one string variable in an INPUT LINE statement.

AlphaBASIC never prints a question mark prompt for INPUT LINE as it does for INPUT, but you may include your own prompt string, which AlphaBASIC will display as a request for data. Such a prompt string must be a string literal enclosed in quotation marks.

Unlike INPUT, if you press RETURN in response to a data request, INPUT LINE sets the variable to zero (if numeric variable) or null (if string variable). Remember, in a like case, INPUT leaves the value of the variable unchanged.

When you use INPUT LINE, remember the default size of unmapped string variables is ten characters; if you want to use strings larger than that, use the STRSIZ statement to reset the default string size or use a MAP statement for the string variable, specifying the size you need. See below for information on STRSIZ.

Some examples of INPUT LINE:

```
INPUT LINE A$
INPUT LINE "ENTER YOUR FULL NAME, PLEASE: ",NAME$
```

You may also use the INPUT LINE statement to read data from a sequential file. For more information on using INPUT LINE and files, see Chapter 16.

LET

The format is:

```
LET variable = expression
```

Assigns a value to a specific variable during execution of the program. You do not have to specify the LET keyword in an assignment statement. Some examples:

```
LET A5 = 12.4
LET SUM(4,5) = A1 + SQR(B1)
LET C$ = "JANUARY"
A5 = 12.4
SUM(4,5) = A1 + SQR(B1)
C$ = "JANUARY"
```

NEXT

See FOR, TO, NEXT, STEP, above.

ON ERROR AND RESUME

Enables and disables error trapping. The formats are:

```
ON ERROR GOTO 500
ON ERROR GOTO TRAP'ROUTINE
ON ERROR GOTO
```

In the first two cases, if an error occurs, the program goes to the specified area. In the third case, the program ends and the appropriate system error message prints out.

After your error recovery procedure has been done, you use the RESUME statement to continue the program execution:

```
RESUME { line number or label }
```

For more information on error trapping, see Chapter 18.

ON - GOSUB (CALL)

The ON GOSUB statement allows multi-path branching to one of several subroutines within the program based on the result of evaluating an expression. The formats are:

```
ON expression GOSUB label/line#1{,label/line#2,...label/line#N}
ON expression CALL label/line#1{,label/line#2,...label/line#N}
```

The expression can be any valid expression which is evaluated and truncated to a positive integer result. The result of the expression evaluation is then tested. The subroutine at label/line#1 is executed if the result is 1, the subroutine at label/line#2 is executed if it is 2, etc. If the result is zero, negative or greater than N, the program goes on to the next statement.

As with the GOSUB statement, the verb CALL may be used in place of the verb GOSUB, giving an ON CALL statement. Here is an animation program using ON-GOSUB:

```
START:
   I = 3 * RND(0) + 1                  ! Random number 1 to 3
   ON I GOSUB UP,DOWN,STRAIGHT         ! Go to 1 of 3 subroutines
   GOTO START:
UP:
   PRINT /"; TAB(-1,3);
   RETURN                              ! Draw symbol, up 1 row
DOWN:
   PRINT TAB(-1,4);"\";                ! Down 1 row, draw
   RETURN
STRAIGHT:
   PRINT "__";                         ! Draw symbol
   RETURN
```

ON - GOTO

The format is:

```
ON expr GOTO label/line#1{,label/line#2,...label/line#N}
```

The ON GOTO statement allows multi-path GOTO branching to one of several points within the program based on the result of evaluating an expression.

The expression can be any valid expression which is evaluated and truncated to a positive integer result. The result is then tested to branch to label/line#1 if 1, label/line#2 if 2, label/line#3 if 3, etc. If the result is zero, negative or greater than N, the program falls through to the next statement. The following is a portion of a menu-selection program:

```
PRINT TAB(22)"Select One of the Following Operations:":PRINT
PRINT TAB(25)"1. Insert/Edit NAME Information."
PRINT TAB(25)"2. Insert/Edit PHONE NUMBER Information."
PRINT TAB(25)"3. Quit without insertion or editing."
PRINT : INPUT "Your choice (1, 2 or 3)? ",A
ON A GOTO NAME, PHONE, QUIT
NAME: INPUT "Select a name: ",N
```

{ The program continues with all three alternatives }

PRINT

The format is:

```
PRINT expression-list
```

or:

```
? expression-list
```

The PRINT statement tells AlphaBASIC to evaluate and display on your terminal the expressions that you specify. For example:

```
PRINT 3 + 4; " HAVE " + "A NICE " + "DAY"
```

prints:

```
7 HAVE A NICE DAY
```

AlphaBASIC prints a carriage return/linefeed after the expression list, unless the expression list ends in a comma or semicolon. Remember that an expression may consist of a string or numeric variable, numeric constant, string literal, function with arguments, operator symbols, or a combination of these elements. For example, the following is one string expression:

```
"STRING DATA" + NAME$ + MID$(A$,1,2).
```

AlphaBASIC displays numeric data with a trailing blank. It also prints one leading blank if the number is positive. It does not print a leading blank if the number is negative. AlphaBASIC displays string data with no leading or trailing blanks.

You may place more than one expression after the PRINT keyword if you separate them with commas or semicolons. If you separate the expressions by semicolons, AlphaBASIC does not print extra spaces when it prints the evaluations of those expressions. For example:

```
? 12 + 12;-32;8/2
```

prints:

```
24 -32  4
```

There are no blanks between the numbers above except for the normal leading and trailing blanks displayed with numeric data.

If you separate the expressions by commas, AlphaBASIC prints the data in "print zones." AlphaBASIC divides the area in which data is to be displayed into five zones of 14 spaces each. If an expression in a PRINT statement is followed by a comma, AlphaBASIC prints that expression in the next available print zone. For example, the AlphaBASIC statements:

```
PRINT 34,1024,-32,-100.2,20
PRINT "AA","BB","C","DDD","A","B","C"
```

display:

```
34            1024        -32         -100.2          20
AA            BB           C           DDD            A
B             C
```

When you look at the display above, remember that AlphaBASIC prints numeric data with a leading and trailing blank if the number is positive, but just a trailing blank if the number is negative.

Note that the strings in the second line were displayed on two different lines—when AlphaBASIC still has an expression to print after it has printed something in the fifth zone, it starts over again with the first zone on the next line.

If you end the PRINT statement expression list with a semicolon or comma, AlphaBASIC does not output a carriage return/linefeed when it finishes displaying that expression list. This will make the output resulting from the next PRINT or INPUT statement appear on the current display line.

The next output will appear in the next print zone if the current PRINT statement ends with a comma; or, the next output will appear immediately following the last character of the current PRINT statement if the PRINT statement ends with a semicolon.

Here are a few examples of the PRINT statement:

```
A$ = "HERE" :  A = 7

PRINT                          ! Yields a blank line
PRINT A                        ! Yields 7
PRINT A$                       ! Yields HERE
PRINT 1 + 2                    ! Yields 3
PRINT "ANY TEXT"               ! Yields ANY TEXT
PRINT "NOTE THE SPACE",A$      ! Yields NOTE THE SPACE       HERE
? "YOU ARE NUMBER ";A          ! Yields YOU ARE NUMBER 7
? "YOU ARE #";A;" IN CLASS."   ! Yields YOU ARE #7 IN CLASS.

PRINT "THERE ARE";             ! Semicolon suppresses carriage
PRINT A;" DAYS LEFT."          ! return/linefeed and yields:
                               ! THERE ARE 7 DAYS LEFT.
```

You may also use the PRINT statement for writing data to sequential files; see Chapter 16.

PRINT USING

The formats are:

```
variable = expression USING format-string
PRINT USING format-string, expression-list
PRINT expression USING format-string
```

PRINT USING is supported for formatting output and is described extensively in Chapter 13.

PROGRAM

The PROGRAM statement allows you to specify a revision level for your program. The format is:

```
PROGRAM Name,a.b{c}{(d)}
```

Where:

Name	=	A reference name (non-functional)
a	=	The major revision number (from 0 to 255)
b	=	The minor revision number (from 0-15)
c	=	The revision subscript (optional, letter from a-o)
d	=	The edit number (optional, from 0 to 4095)

The group of characters following the PROGRAM keyword are evaluated when the program is compiled, and therefore must be literal characters or numbers and not variables. AlphaBASIC checks each of the fields to determine if the proper range of number or letter is specified. There must be a least one space after the PROGRAM keyword. If part of your revision specification is incorrect, you will get an error. Here is an example of a full PROGRAM statement:

```
PROGRAM Test3,1.0b(103)
```

If your AlphaBASIC program contains a PROGRAM statement, the compiled version of that program (the .RUN file) contains the version information you specified.

DISPLAYING VERSION INFORMATION:

The AMOS command DIR displays general information about specified disk files. If you use the /V option with the DIR command, you will also see version information for any .RUN files included in that directory display. For example:

```
DIR/V *.RUN RETURN

MYPROG     RUN   2          1.0B(103)  DSK2:[50,0]
RECEIV     RUN   15         4.2E(50)
INVEN      RUN   102        0.2
Total of 3 files in 119 blocks
```

If the original program file from which the .RUN file was generated did not contain a PROGRAM statement, DIR displays the version number as 0.0.

RANDOMIZE

The format is:

```
RANDOMIZE
```

Resets the random number generator seed to begin a new random number sequence starting with the next RND(X) function call. See Chapter 11 for information on the random number generator.

READ, RESTORE, AND DATA

The formats are:

```
READ variable1{,variable2,...variableN}
RESTORE
DATA data1{,data2,...dataN}
```

These calls allow data to be an integral part of the source program with a method for getting this data into specific variables in an orderly fashion.

DATA statements are followed by one or more literal values (that is, not variables) separated by commas. String literals need not be enclosed in quotes unless the literal data contains a comma.

Do not place comments on a DATA statement line—the comment will be interpreted as data! All data statements are placed into a dedicated area in memory no matter where they appear in the source program.

READ statements are followed by one or more variables separated by commas. Each time a READ statement is executed, the next item of data is retrieved from the DATA statement pool and loaded into the variable named in the READ statement.

If there is no more data left in the data pool, the program can only continue to read data if a RESTORE statement is executed, which reinitializes the reading of the data pool from the beginning again. Otherwise, an error message:

```
Out of data in MYPROG
```

results, and the program is aborted. Here are some forms that READ and DATA may take:

```
DATA 1,2,3,4,5
DATA 2.3,0.555,ONE STRING,"4,4"
READ A,B,C
READ A$
READ C(2,3),B$(4)
```

The following is a program example using READ, RESTORE, and DATA:

```
START:
   INPUT LINE "How much did you pay for your car?$",WORTH
   ? "Based on national averages, your car will depreciate: " : ?
   FOR I = 1 TO 5
   ? "After the "; : READ YEAR$ : ? YEAR$;" year, ";
   ? "your car will be worth about"; : READ PERCENT
   WORTH = WORTH * PERCENT
   ? WORTH USING "$$#####,.##" : ?
NEXT I
DATA first,.77,second,.78,third,.79,fourth,.81,fifth,.84
RESTORE
INPUT LINE "Would you like to see another schedule? ",L$
IF UCS(L$[1,1]) = "Y" THEN GOTO START ELSE PRINT "Goodbye."
END
```

RESTORE

See READ, RESTORE and DATA, above.

RESUME

See ON ERROR and RESUME, above.

RETURN

See GOSUB, above.

SCALE

The format is:

```
SCALE value
```

The scaling factor represents the number of decimal places that the 11-digit window is effectively shifted to the right in any floating point number. SCALE is discussed in detail in Chapter 14.

SIGNIFICANCE

The format is:

```
SIGNIFICANCE value
```

The significance statement allows you to dynamically change the default value of the numeric significance of the program for unformatted printing.

The significance value can be any value from 1 through 11 and represents the maximum number of digits to be printed in unformatted numbers. Rounding off to the specific number of digits is not performed until just before the printing of the result.

The statement SIGNIFICANCE 8, for instance, sets the number of printable digits to 8. The value is interpreted at run-time and therefore may be any valid numeric expression, including variables. The current significance is ignored when PRINT USING is in effect.

Note that the SIGNIFICANCE statement only affects the final printed result of all numeric calculations. The calculations themselves and the storage of intermediate results are always performed in full 11-digit precision to reduce the number of errors.

The significance default is 6 digits. This is equivalent to standard single-precision formats used in most of the popular versions of BASIC. The significance is not reset by the RUN command and therefore may be set in interactive mode in a direct statement just prior to the actual running of a test program. Of course, any SIGNIFICANCE statements encountered during the execution of the program reset the value.

STEP

See FOR, TO, NEXT and STEP, above.

STOP

The format is:

```
STOP
```

Causes the program to suspend execution. If you are in interactive mode, you will see the message:

```
Program stop at line nnnn
```

You may then continue to the next statement in sequence by executing a CONT command or a single-step command.

If you are running the program at AMOS command level, STOP displays:

```
Program stop {in line nnnn} of [program-name]
Enter CR to continue:
```

Press the RETURN key to continue the program or use Control-C to stop the run.

STRSIZ

The format is:

```
STRSIZ value
```

The STRSIZ statement sets the default value for all unmapped string variables which are encountered for the first time when the program is compiled.

The default value of all strings in the absence of a STRSIZ statement is 10 characters. The statement:

```
STRSIZ 25
```

for example, causes all newly allocated strings which follow to have a maximum size of 25 characters, instead of 10. This includes the allocation of string arrays. The size value is evaluated at compilation time and so must be a single positive integer. The maximum size you can allocate depends on the amount of memory on your system, and on how many variables you use.

THEN

See IF, THEN, ELSE, above.

TO

See FOR, TO, NEXT and STEP, above.

USING

See PRINT USING, above.

XCALL

The format is:

```
XCALL routine{,argument1{,argument2,...argumentN}}
```

Executes an external assembly language subroutine. Assembly language subroutines are discussed in detail in Chapter 19.

For information on the assembly language subroutines available for use with AlphaBASIC programs, see the *AlphaBASIC XCALL Subroutine User's Manual* that applies to your system.

Chapter 11 - AlphaBASIC Functions

Functions compute and return a value. A function either operates on or is controlled by the argument, which is enclosed in parentheses. There are four main categories of functions:

- Numeric
- Trigonometric
- Control
- String

Numeric and trigonometric functions return numeric values. Control functions indicate the status of file input and output operations and system operations. String functions operate on numeric values or strings of one or more characters in length, and return string values.

Functions are different from program statements. Functions return a value that must be either assigned to a variable, or passed as an argument to a statement, such as the PRINT statement. For example:

```
Root = SQR(16)                          (Root = 4)

Number = 25
Result = Root * (SQR(Number) + 24)      (Result = 4 + 5 + 24 = 33)

PRINT "The answer is: "; SQR(16)        (The answer is:  4)

SQR(16)      (Produces a syntax error because the function is not assigned to a variable)
```

NUMERIC FUNCTIONS

Numeric functions accept a string or numeric argument and return a numeric value. Note that the mode independence feature of the expression processor performs automatic conversions if a numeric argument is used where a string argument is expected, and vice versa.

Abs(X)

Returns the absolute value of the argument X. For example:

```
PRINT ABS(-32.4)
PRINT ABS(17.2)
```

returns:

```
32.4
17.2
```

Asc(A)

Returns the ASCII decimal value of the first character of argument A. The argument may be either a string literal or string variable. For example:

```
A$ = "v"
B$ = 2
C$ = "2"
PRINT ASC("A"),ASC(A$)
PRINT ASC(B$),ASC(C$)
```

displays:

```
65          114
50          50
```

Exp(X)

Returns the constant e (2.7182818285) raised to the power X. e is the base of the system of natural logarithms.

Fact(X)

Returns the factorial of X. For example:

```
READY
PRINT FACT(7) [RETURN]
5040
```

Fix(X)

Returns the integer part of X (fractional part truncated). For example:

```
READY
PRINT FIX(50.4685) [RETURN]
50
```

Int(X)

Returns the largest integer less than or equal to the argument X. For example:

```
READY
Print Int(23.4) RETURN
23
```

where 23 is the largest integer less than or equal to the real number 23.4.

The INT( ) function differs from FIX( ) when dealing with negative numbers only. For example:

```
READY
Print Int(-23.4) RETURN
-24
```

```
READY
Print Fix(-23.4) RETURN
-23
```

Log(X)

Returns the natural (base e) logarithm of the argument X.

Log10(X)

Returns the decimal (base 10) logarithm of the argument X

Rnd(X)

Returns a random number between 0 and 1. The number returned is based on a previous value known as the "seed." The argument X controls the number to be returned. If X is negative, it is used as the seed to start a new sequence of numbers. If X is zero or positive, the next number in the sequence is returned, depending on the current value of the seed (this is the normal mode). The RANDOMIZE statement may be used to create a seed which is truly random and not based on a fixed beginning value set by the system.

If you want to generate a random number greater than or equal to number A and less than number B, you can use the expression:

```
(B - A) * RND(0) + A
```

The INT function can be used when generating random integer numbers to give you an integer number. For example, to generate a random integer greater than or equal to 5 and less than 31, use the expression:

```
INT(26 * RND(0) + 5)            ! B - A = 26
```

Sgn(X)

Tells you the sign of the number X. If X is a negative number, SGN(X) will generate a -1. If X is 0, SGN(X) will be 0. If X is a positive number, SGN(X) will be 1.

Sqr(X)

Returns the square root of the argument X. For example:

```
READY
PRINT SQR(25) RETURN
5
```

Val(A$)

Returns the numeric value of the string variable or string literal A$ converted to floating point under normal BASIC format rules. For example, VAL("123") returns 123.

TRIGONOMETRIC FUNCTIONS

The following trigonometric functions are implemented in full 11-digit accuracy:

SIN(X)	Sine of X
COS(X)	Cosine of X
TAN(X)	Tangent of X
ATN(X)	Arctangent of X
ASN(X)	Arcsine of X
ACS(X)	Arccosine of X
DATN(X,Y)	Double arctangent of X,Y

CONTROL FUNCTIONS

Control functions indicate the status of file input and output operations, and provide information on system operations.

Err(X)

Returns a status code which refers to program status during error trapping. For a complete list of these codes, and examples of using ERR(X), see Chapter 18. If X is 0, ERR returns the specific code of the error detected; if X is 1, ERR returns the number of the last program line encountered before the error occurred. If X is 2, ERR returns the file number of the last file accessed.

STRING FUNCTIONS

The following string functions accept numeric or string arguments, and return strings. Note that the mode independence feature of the expression processor performs automatic conversions if a numeric argument is used where a string argument is expected, and vice versa.

Asc(X)

Returns the ASCII decimal value of the first character in a string. For example:

```
A$ ="Zirconium's atomic number is:"
PRINT ASC(A$)
```

prints:

```
90
```

since 90 is the ASCII value (in base 10) of upper case Z. You may also use string literals:

```
PRINT ASC("Z")
```

Chr(X)

Returns a single character having the ASCII decimal value of X. Only one character is generated for each CHR function call. For example:

```
READY
PRINT CHR(90) RETURN
Z
```

Instr(X,A$,B$)

Performs a search for the substring B$ within the string A$, beginning at the Xth character position. It returns a value of zero if B$ is not in A$ at or after the Xth position, or the character position if B$ is found within A$. Character position is measured from the start of the string, with the first character position represented as one. Some direct statements will illustrate:

```
READY
A$ = "ELEPHANT" RETURN
B$ = "ANT" RETURN
PRINT INSTR(1,A$,B$) RETURN
6

READY
PRINT INSTR(8,"MEADOWLARK","LARK") RETURN
0
```

In the second example, the specified string "LARK" is not found in the string "ARK", which is the substring starting at the 8th position, therefore, a zero is returned.

Lcs(A$)

Returns a string which is similar to the argument string (A$), but with all characters translated to lower case. If:

```
A$ = "A is for Alpha"
```

then:

```
PRINT LCS(A$)
```

prints:

```
a is for alpha
```

Useful for checking input when the case is not certain. For example:

```
IF LCS(VAR$) = "yes" THEN GOTO START
```

is easier than using:

```
IF VAR$ = "YES" or VAR$ = "yes" THEN GOTO START
```

Left(A$,X)

Returns the leftmost X characters of a string expression. For example:

```
READY
A$ = "Now is the time" RETURN
PRINT LEFT(A$,7) RETURN
Now is
```

Note that the printed string includes the trailing blank after "is".

Len(A$)

Returns the number of characters in a string expression. For example:

```
STRSIZ 50
A$ = "Wherefore art thou, Romeo?"
PRINT LEN(A$)
```

produces:

```
26
```

This is helpful for checking for null strings. For example:

```
IF LEN(VAR$) = 0 THEN GOTO RE'INPUT
```

Mid(A$,X,Y)

Returns the substring composed of the characters of the string expression A$ starting at the Xth character and extending for Y characters. A null string is returned if X is greater than the length of A$. For example:

```
STRSIZ 60
A$ = "The quick brown fox jumped over the sleeping dog"
PRINT MID(A$,17,15)
```

produces:

```
fox jumped over
```

Right(A$,X)

Returns the rightmost X characters of the string expression A$. For example:

```
STRSIZ 60
A$ = "I THINK, THEREFORE I AM"
PRINT RIGHT(A$,4)
PRINT RIGHT(1234,2)
```

produces:

```
I AM
34
```

Remember that you can use numeric arguments for many string functions.

Space(X)

Returns a string of X number of blank spaces. X must be a positive number. The statement:

```
PRINT "COLUMN A"; : PRINT SPACE(10); : PRINT "COLUMN B"
```

prints the following:

```
COLUMN A          COLUMN B
```

where the 10 spaces between the first and second strings are the result of the SPACE(10) function. SPACE is especially handy for padding strings to a fixed length. For example:

```
STRSIZ 25
INPUT "Name?",NAME$
IF LEN(NAME$) < 25 &
   THEN NAME$ = NAME$ + SPACE(25 - LEN(NAME$))
```

Str(X)

Returns a string which is the character representation of the numeric expression X. No leading space is returned for positive numbers. For example:

```
A$ = STR(45)
PRINT A$
A$ = "1" + A$
PRINT A$
```

produces:

```
45
145
```

Ucs(A$)

Returns a string which is similar to the argument string (A$), except that all characters are translated to upper case. For example:

```
STRSIZ 20
A$ = "M is for Micro"
PRINT UCS(A$)
```

produces:

```
M IS FOR MICRO
```

This is useful for checking input when the case is not certain. For example:

```
IF UCS(VAR$) = "YES" THEN GOTO START
```

is easier than using:

```
IF VAR$ = "YES" or VAR$ = "yes" THEN GOTO START
```

Chapter 12 - System Functions

AlphaBASIC supports a unique group of operators called system functions, which give you the ability to:

- Access the computer's input/output ports
- Access physical memory
- Access various system parameters

System functions use a format similar to that used by standard functions, with the reserved word representing the desired function followed by optional arguments enclosed within parentheses.

The major difference is that the reserved word of a system function may appear on the left side of an assignment statement. When a system function is used on the left side of the equal sign in a statement, it outputs the result of that function to a specified place.

System functions used within expressions on the right side of an assignment statement perform an input or read operation and deliver back a result to be used in the expression evaluation.

BYTE(X) AND WORD(X)

The BYTE and WORD system functions allow you to inspect and alter any memory locations within the memory addressing range of the computer. These operations are often called PEEK and POKE statements in other versions of BASIC.

The BYTE function deals with 8 bits of data in the range of 0 to 255, and the WORD function deals with 16 bits of data on an even address boundary (which means that the "X" you specify will be rounded down to the next even number) in the range of 0 to 65535, inclusive. Any unused bits are ignored, with no error message.

These commands are not protected; it is possible to cause severe damage to the operating system in memory if you use the commands improperly.

```
BYTE(X) = <expr>          ! Writes the low byte of <expr>
                          ! into decimal memory location X
WORD(X) = <expr>          ! Writes the low word of <expr>
                          ! into decimal memory location X
      A = BYTE(X)         ! Reads decimal memory location
                          ! X and places the byte into A
      A = WORD(X)         ! Reads decimal memory location
                          ! X and places the word into A
```

DATE

The DATE system function returns the two-word system date. You cannot set the DATE function. The format is:

```
A = DATE              ! Places the system date into variable A
```

You can use the external subroutine (XCALL) ODTIM to convert this number into a date and/or time display formatted any way you like. See the *AlphaBASIC XCALL Subroutine User's Manual* that applies to your system for more information. The following statements:

```
STRSIZ 50                   ! DAY$ must be > 44
XCALL ODTIM,DAY$,0,0,-1
PRINT DAY$
```

print the date and time like this:

```
Wednesday, January 15, 1986 01:07:07 AM
```

IO(X)

The IO system function allows the input/output ports to be selectively read from or written to. In both cases only one byte is considered, and an output expression greater than 511 merely ignores the unused bits. The range of ports available is 0 to 511.

```
IO(X) = <expr>        ! Writes low byte of <expr> to port X
A = IO(X)             ! Reads decimal port X into A
```

If (X) is 0-255, the function accesses the external I/O ports (the addresses are FFFF00-FFFFFF). If (X) is 256-511, it accesses the internal (on-board) I/O ports (addresses FFFE00-FFFEFF.)

MEM(X)

Returns a positive integer value which specifies the decimal number of bytes currently in use for various memory areas used by the compiler system.

The most common use of MEM(X) is to return the number of free bytes left in the user memory partition. The MEM(0) call duplicates the action performed by the FRE(X) function in some other versions of BASIC.

Other values of X return memory allocations which pertain to various areas in use by the compiler, and may or may not be of use to you. The byte counts returned for the various values of X are:

0 - Free memory space remaining in current user partition
1 - Total size of current user partition
2 - Size of source code text area
3 - Size of user label tree
4 - Size of user symbol tree (variable/function names)
5 - Size of compiled object code area
6 - Size of data pool (compiled DATA statements)
7 - Size of array index area (dynamic links to arrays)
8 - Size of variable storage area (excluding arrays)
9 - Size of dummy data termination field (always zero)
10 - Size of file I/O linkage and buffer area
11 - Size of variable array storage area (dynamically allocated at run-time)

The statement PRINT MEM is equivalent to PRINT MEM(0).

TIME

The TIME system function requires no argument and is used to retrieve the time of day as stored in the system monitor communications area. The time is stored as a two-word integer representing the number of seconds since midnight. You cannot set the system time from within AlphaBASIC.

```
A = TIME              ! Returns time-of-day into A
```

You can use the external subroutine (XCALL) ODTIM to convert this number into a date and/or time display formatted any way you like. See the *AlphaBASIC XCALL Subroutine User's Manual* that applies to your system for more information. See DATE above for an example.

Chapter 13 - Formatting Output (PRINT USING and Expanded Tabs)

Most BASIC business applications programs spend a great deal of effort in generating reports and printouts in which data must be neatly and clearly presented. In other words, correctly formatting output is usually a major concern of the BASIC programmer.

AlphaBASIC provides several important features that help you to format data. This chapter discusses:

- How to employ the USING modifier to format data
- The formatting characters you can use
- How to use the expanded tab functions that control the output of data on the terminal screen

THE USING MODIFIER

The USING modifier allows you to format numeric or string data by specifying a format string (sometimes called an "editing mask"). Although you can use the USING modifier to store the formatted data in a string variable, you may also use it in combination with the PRINT statement to send the formatted data to a terminal display or to a file. For information on PRINT, see Chapter 10.

By "formatting" data, we mean the process of adjusting the appearance of data so that it fits the pattern of a specific format string. It might help to think of the format string as a template or pattern with which you are going to control the format of your data. The USING modifier allows you to apply the format string to your data. Using format strings and USING, you can do such things as:

- Line columns of numbers up by their decimal points
- Insert dollar signs and commas into numeric data to represent dollar amounts
- Line up numeric and string data within specified fields
- Generate and print leading zeros for numeric data
- Print asterisks instead of leading spaces
- Print numeric data in exponential form

And many other things, too.

The sections below talk about the special formatting characters within the format string that allow you to perform such adjustments. The statements in which you use the USING modifier take these forms:

```
variable = expression USING format-string

PRINT expression USING format-string

PRINT USING format-string, expression-list
```

where ***expression*** is usually a numeric or string constant, or a numeric or string variable.

For example, if you want to format the number 2345.678 with the format string "$$####.##", you could say:

```
NUMBER = 2345.678 USING "$$####.##"
```

or:

```
PRINT 2345.678 USING "$$####.##"
```

or:

```
PRINT USING "$$####.##",2345.678
```

You may use the first and second formats only for numeric data; you may use the third format for string and numeric data. Also, remember that USING has the lowest precedence of all operators. Therefore, all other operations in expressions surrounding the USING operator are performed before formatting is done. For example:

```
READY
PRINT 23 + 4 USING "###" + ".#" RETURN
27.0
```

The format string may be a string expression (for example, MID$(A$,4,5), a string constant (for example, "###.##"), or a string variable (for example, MASK$).

If you use the third PRINT USING variant above, you may supply a list of expressions to be formatted, separating the expressions with commas as with the regular PRINT statement (for example: PRINT USING "#####.##",A,B,C,D,E). If you supply more expressions than the format string is meant to handle, AlphaBASIC re-uses the format string until each of the elements in the expression list has been formatted. If you supply fewer expressions than the format string is meant to handle, AlphaBASIC ignores the unused portion of the format string.

You may also send formatted data to a file by specifying a file-channel number after the PRINT keyword. For example:

```
PRINT #1 USING format-string, expression-list
```

For information on sending data to files, see Chapter 16.

HOW TO SPECIFY FORMAT STRINGS

The sections below discuss the special characters that make up the format string; these special characters control the output of your data. Characters other than these special formatting characters which appear in a format string are output literally as part of your data.

Formatting a Numeric Field (#)

The # symbol in a format string always indicates that you want to format numeric data. Each # symbol in a format string represents one numeric digit. The simplest numeric format string would consist of just # symbols. For example:

```
PRINT C USING "####"
```

The statement above tells AlphaBASIC to format the numeric variable C into a field of four digits, with no fractional part. If the format string causes AlphaBASIC to remove the fractional part of a number, AlphaBASIC rounds the number to the next integer, rather than truncating it. For example:

```
PRINT 2367.88 USING "####" RETURN
2368
```

If the numeric field is too small to contain the specified number (for example, if we had specified the number 650456.56 with the format string "####"), AlphaBASIC prints the number in standard BASIC format preceded by a % symbol, indicating overflow. For example:

```
PRINT 150450 USING "####" RETURN
%150450
```

If the numeric field is larger than the specified number, AlphaBASIC right justifies the number in the field, inserting leading blanks into the digit positions not needed. For example:

```
PRINT USING "######",23 RETURN
    23
```

As you can see, the computer prints out four blanks before the number 23. Note that other formatting characters discussed below (for instance, the $$ and ** symbols) also define digit positions as well as perform special formatting functions.

You cannot format string data with a numeric field format string. If you try to do so, AlphaBASIC just prints the format string, indicating that it was unable to format the data. For example:

```
PRINT USING "#####","Hi there" RETURN
#####
```

Formatting a String Field (\)

You may specify fields for string data by using the backslash symbol (\). Two backslashes define a string field whose size equals the number of characters enclosed in the backslashes plus the backslashes themselves.

Although the usual practice is to enclose blanks in the string field (for example, "\ \"), AlphaBASIC permits the use of any characters. Since these characters are never printed, but simply define the size of the field by which a string is to be formatted, non-blank characters serve only as a comment.

However, when using several string fields within a single format string, it can be useful to visually separate them from the spaces between the fields by using non-blanks within the backslashes. For example:

```
"\---field1----\     \-----field2-----\   \-field3-\"
```

String fields allow you to define the placement and size of string data. For example:

```
A$ = "Now is the time."
PRINT USING "As he once said, '\--------------\'",A$
```

produces:

```
As he once said, 'Now is the time.'
```

If the string to be formatted is larger than the string field, AlphaBASIC truncates the extra characters. If the string to be formatted is smaller than the string field, AlphaBASIC adds trailing blanks to the string to make it the same size as the field, and thus left justifies it in the field.

You may combine string fields and numeric fields within a single format string. For example:

```
STRSIZ 25
MAP1MASK,S,42,"\-10char-\ ####.## \---15 char---\"
C$ = "(in millions)"
PRINT USING MASK,"YEAR 1979",234.556,C$, &
   "YEAR 1980",5678.456,C$
```

produces:

```
YEAR 1979       234.56     (in millions)
YEAR 1980       5678.46    (in millions)
```

Remember that the default string size is 10 characters, so you will want to explicitly define any strings over 10 characters by using MAP statements, or include a STRSIZ statement in your program to adjust the default string size.

One-Character String Fields (!)

The exclamation mark identifies a one-character string field. AlphaBASIC replaces the exclamation mark with a corresponding string. If the string constant or string variable contains more than one character, AlphaBASIC ignores any characters past the first. For example:

```
STRSIZ 40
MASK$ = "The temperature is: ###!=##!"
PRINT USING MASK$,50,"F",10,"C",68,"F",20,"C", &
86,"F",30,"C",104,"F",40,"C"
```

prints:

```
The temperature is:   50F  =  10C
The temperature is:   68F  =  20C
The temperature is:   86F  =  30C
The temperature is:  104F  =  40C
```

If no string is available to be substituted for the ! symbol, AlphaBASIC simply prints the ! symbol instead. For example, if we took our sample program above and removed the first "F" from the PRINT USING expression list, the first line of our display would look like this:

```
The temperature is: 50!=10C
```

Using Decimal Points in Numeric Fields (.)

You may include one period within a numeric field to specify where a decimal point is to appear in the formatted number. For example:

```
PRINT USING "####.##",2345.502,1100.657,200
```

produces:

```
2345.50
1100.66
200.00
```

If the number specified contains more digits to the right of the decimal point than the format string, AlphaBASIC rounds the number so that it contains the right number of digits in the fractional part.

If the format string contains more digits to the right of the decimal point than the specified number, AlphaBASIC fills in the unused digit positions with zeros (as in the case of the number 200, above).

If the format string specifies any digits in front of the decimal point, AlphaBASIC prints at least one digit in front of the decimal point for each number, even if that digit is a zero.

Dollar Signs and Numeric Fields ($$)

Two dollar signs ($$) at the front of a numeric field format string tell AlphaBASIC to insert a dollar sign at the front of the formatted number. The double dollar sign symbol defines two digit positions, one of which is taken up by the dollar sign itself. For example:

```
PRINT USING "$$#####.##",17500.66,100,345.2
```

produces:

```
$17500.66
  $100.00
  $345.20
```

Notice the difference between using the double dollar sign to produce a floating dollar sign, and simply using the single non-formatting character "$" in the format string:

```
PRINT USING "$#####.##",17500.66,100,345.2
```

produces:

```
$17500.66
$100.00
$345.20
```

Because you will use $$ to format data that represents money amounts, you may want to use the floating comma symbol in combination with $$. See the section below for information on this formatting character.

Remember that you can include non-formatting characters in a format string. In the case above, a single dollar sign is not a formatting character, and so AlphaBASIC simply prints it as part of the formatted data. As another example:

```
PRINT USING "###%",23.45,56.78,99.84
```

produces:

```
 23%
 57%
100%
```

In the example above, the "%" symbol is *not* a special formatting character. Another example:

```
PRINT USING "Telephone: (###) ### ####",714,555,1212
```

produces:

```
Telephone: (714) 555 1212
```

Putting a Comma Every Three Digits (,)

By including a comma to the left of the decimal point in your format string, you tell AlphaBASIC to insert a comma every three digits to the left of the decimal point. The comma defines one digit position. For example:

```
PRINT 6507501.89 USING "########,.##"
```

produces:

```
6,507,501.89
```

Fill Leading Blanks With Asterisks (**)

By including two asterisks at the front of your format string, you tell AlphaBASIC to replace any leading blanks that would normally be output in front of a number with asterisks. This is especially useful when printing checks. The double asterisk defines two digit positions. For example:

```
PRINT 231.69 USING "**###########.##"
```

produces:

```
**********231.69
```

You will probably use asterisk-fill formatting when printing dollar amounts; remember that you may include a double-dollar sign symbol in the format string. For example:

```
PRINT 231.69 USING "**$$###########.##"
```

prints:

```
*******$231.69
```

Fill Leading Blanks With Zeros (Z)

To generate leading zeros, include the Z symbol within your format string. The format string must begin with one # symbol followed by a series of Zs. The total size of the formatted string is the number of Zs plus the one # symbol. For example:

```
PRINT 123 USING "#ZZZZZ"
```

produces:

```
000123
```

Add a Trailing Minus Sign to a Number (-)

You may cause the sign of a number to be printed following the number by ending a numeric field in a format string with a minus sign. If the number is positive, AlphaBASIC prints a blank after the number; if it is negative, AlphaBASIC prints a minus sign after the number. For example:

```
MAP1 MASK,S,26,"\--7--\         $$#####.##-"
C$ = "Credit:"
D$ = "Debit:"
PRINT USING MASK,C$,35.67,D$,-57.89,C$,10.89,D$,-356.33
```

produces:

```
Credit:         $35.67
Debit:          $57.89-
Credit:         $10.89
Debit:          $356.33-
```

Printing Numbers as Exponents (^^^^)

You may specify exponential format by following the numeric field in a format string with four carets (^^^^). These symbols define the spaces taken up by the "E nn" exponent characters. AlphaBASIC left justifies the significant digits, adjusting the exponent as necessary. As with other numeric formats, AlphaBASIC allows any decimal point arrangement. For example:

```
PRINT USING ".#####^^^^",100,2345.66,5000,.0004
```

prints:

```
.10000E+03
.23457E+04
.50000E+04
.40000E-03
```

FORMATTING EXAMPLES AND HINTS

All of our examples above used the PRINT statement to print formatted data. Remember that you may also format a numeric value without displaying it by using the USING modifier without the PRINT statement. For example:

```
A$ = B USING C$
```

The statement above formats the number in B using the format string in C$, and leaves a string result in A$.

This format of the USING modifier is only for formatting numeric data. Also note that even though we are formatting numeric data, the result is always a string.

This type of format allows you to create headings and image lines that you use more than once, and to inspect and manipulate formatted data before printing it.

You may not use the USING modifier recursively. That is, you may not use a format string that is itself the result of a USING modifier. For example, if you have specified:

```
C$ = B USING "###.##"
```

you may not say:

```
N$ = D USING C$
```

When using the PRINT USING format, remember that PRINT USING differs from the regular PRINT statement in that the use of semicolons to separate the elements of the print list has no effect on the spacing of those formatted elements.

EXPANDED TAB FUNCTIONS

The TAB function in AlphaBASIC has been expanded beyond its normal use. Besides being able to use TAB to indent text, you can also use it to control various functions of your terminal.

The TAB function is only used in a PRINT statement. It operates in the traditional manner when supplied with only a single numeric argument such as TAB(X). In this case the function causes the cursor to be positioned on the "X + 1" column on the current line. For instance, TAB(5) would cause 5 spaces to be printed, and the following characters would begin in column six.

When supplied with two arguments such as TAB(R,C), however, the TAB function performs special CRT operations.

If the value of R is positive, the R,C arguments are treated as row and column coordinates for positioning the cursor on the terminal screen. The specified characters are then printed beginning in that position. As in other functions, the R and C arguments may be expressions. Terminals are assumed to begin with row 1 (top of screen) and column 1 (left end of each row).

Special Screen Handling Codes

If the value of R is -1, the function is interpreted as a special terminal command and the C argument then specifies the code of the screen function you wish to use. The codes are transmitted to the terminal driver (.TDV file in DSK0:[1,6]), which does the actual interpretation and performs the special function for your terminal.

You may be able (depending on the capabilities of your terminal) to do such things as display in reverse video, draw lines on the screen, display special characters, etc. See Appendix D for the standard decimal codes in use for the terminal drivers supplied by Alpha Micro.

Chapter 14 - Scaled Arithmetic

Whenever a mathematical operation with long decimal numbers is performed, the number of significant digits used becomes important.

AlphaBASIC uses a floating point format which gives an accuracy of 11 significant digits. Unfortunately, this accuracy is absolute only when dealing with numbers that are total integers (with no numbers to the right of the decimal point). This is because of the conversions that are required from ASCII decimal input to the floating point format used to perform arithmetic.

For most business people, the range of numbers that are usually important are those numbers in "money" format—with two digits to the right of the decimal and up to nine digits to the left of the decimal point. When the fractional part of the number is converted between decimal and floating point formats, a small but significant error is sometimes introduced which may show up when you want to see accurate dollars-and-cents values.

As an example of the kinds of inaccuracies that can occur, take a look at the following statements:

```
SIGNIFICANCE 11
PRINT 26.4 - INT(26.4)
```

Instead of the expected answer of .4, we see the answer:

```
.39999999999
```

This is NOT an error in AlphaBASIC, but simply represents the side effects of converting a decimal fraction to floating point representation and back again. Some decimal fractions cannot be exactly expressed as a floating point fraction in a finite number of digits, and so a round-off error occurs.

The error was only visible because our program set the number of significant digits to 11 (the usual number of significant digits is six). Such errors can accumulate and present themselves when you do a large number of multiplications and divisions using decimal fractions.

AlphaBASIC incorporates a scaling feature which helps to alleviate this problem by storing all floating point numbers with a scale offset. This offset tells AlphaBASIC where the 11 absolute accuracy digits are located in relation to the decimal point. AlphaBASIC does this by multiplying every input number by the scaling factor and then dividing it out again before printing. This is a simplified explanation, and many other checks and conversions are done internally to scaled numbers. If you are interested in more detail on scaling numbers, see the section at the end of this chapter.

THE SCALE STATEMENT

Scaled arithmetic is normally entered at the start of a program and continues in effect throughout the program. The statement for setting the program into scaled mode is:

```
SCALE n
```

The scaling factor "n" must be a decimal digit in the range of -30 to +30. It may not be a variable, since scaling is done at compile time for constant values as well as at run-time for input and output conversions.

Negative scaling moves the 11-digit window to the left. A negative scaling factor takes care of those cases where your numbers are too large, rather than too small.

A few words of caution are in order here. Once AlphaBASIC detects the SCALE statement during compilation, AlphaBASIC scales all constant values that follow by the scaling factor so that they are stored properly. In addition, a run-time command is generated in the executable program which causes the actual scaling to be performed on INPUT and PRINT values when the program is running.

If two or more different SCALE statements are executed in the same program, some very strange results may come out unless you are totally familiar with what is happening with compile-time and run-time conversions.

Let's add a SCALE argument to our previous program:

```
SCALE 2
SIGNIFICANCE 11
PRINT 26.4 - INT(26.4)
```

now it prints:

```
.4
```

If you are using a positive scaling factor to adjust real numbers, note that using SCALE does nothing to prevent inaccuracies if the scale factor you use is not large enough to cause AlphaBASIC to handle your data as integers.

For example, if you want to handle numbers that have three digits to the right of the decimal point, a scaling factor of 2 will leave one digit to the right of the decimal point, and scaling error can still occur. So, if you will be using numbers with a fractional part of two digits, use a scaling factor of 2; if the fractional part will be three digits, use a scaling factor of 3; and so on.

Floating point numbers that are stored in files by the sequential output PRINT statement are unscaled and output in ASCII with no problems. Floating point numbers that are written to random access files by using the WRITE statement are not unscaled first; any program that reads this file as input must either be operating in the same scaling mode in which the data was written, or else must apply the scale factor explicitly to all values from the file. Binary and string values, of course, are never modified, regardless of the scaling factor currently in use.

HOW THE SCALING FACTOR WORKS

All of the conversions we discuss in this section are done automatically by AlphaBASIC. This section merely talks about the background theory of scaling numbers.

The scaling factor represents the number of decimal places that the 11-digit "window" is effectively shifted to the right in any floating point number. For example, the most common application is in a business environment where the scaling factor of 2 would be used to give absolute 11 place accuracy to numbers which extend 2 places to the right of the decimal point.

This means that the value of 50.12 is multiplied by the scaling factor of 2 digits (100) and stored as the floating point value of 5012. Since this value is an integer, it has absolute accuracy. Just before printing, AlphaBASIC divides this number by the scaling factor to reduce it to its intended value of 50.12.

Other conversions have been included into the system to take care of all the little subtle effects of storing scaled numbers. For example, when converting scaled numbers to integer or floating point format, AlphaBASIC must unscale the number first before converting it.

When AlphaBASIC multiplies two scaled numbers together, the result is a number which must be unscaled once, while division of two scaled numbers creates exactly the opposite problem. Dealing with scaled numbers for exponential, logarithmic and trigonometric functions creates even more exotic problems—but AlphaBASIC handles them all for you.

Chapter 15 - Mapping Variables

This chapter discusses the following subjects:

- What variable mapping is
- How to define how variables are grouped
- The format of MAP statements
- How to use MAP statements
- Locating variables during debugging
- How memory is allocated by MAP statements

WHAT IS VARIABLE MAPPING?

Mapping variables is a method of grouping variables together. The ability to group variables together is useful when those variables contain information that is related. Variable mapping is a powerful tool that is similar to COBOL data description techniques and Pascal record definitions.

Why should you use mapped variables? Well, mapped variables offer the following advantages:

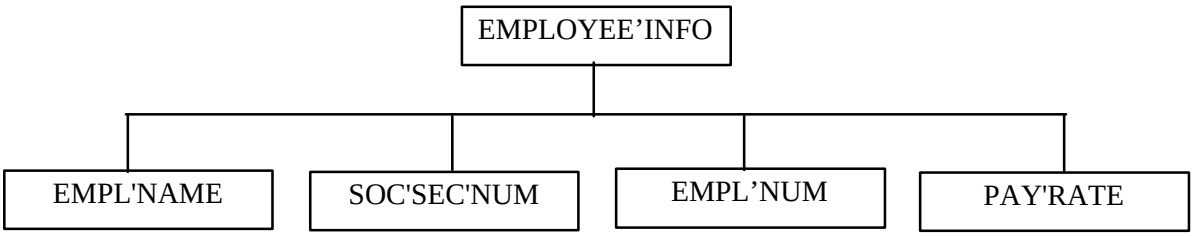
- They can save memory.
- They make variables easier to refer to.
- They can make it easier to access data in random data files.
- They can improve the logic and readability of your program. If all variables are mapped at the start of the program, it is easy to see what variables are used and how they are related. Defining all variables used is good programming practice.
- They allow you to define binary variables (which can save memory when using flags).

Let us take a look at an example to show you how mapped variables can help you. Say that you want to write a program that keeps a record for each of your employees. You want to know the employee's name, social security number, employee number, and rate of pay. In most cases where you are dealing with the variable that contains the employee's name, you will also want to work with one or more of the other variables too.

Using regular AlphaBASIC variables, you might have EMPL'NAME$, SOC'SEC'NUM$, EMPL'NUM, and PAY'RATE$. If you wanted to read one employee's records in from a file, you would need a statement like this:

```
READ #1,EMPL'NAME$,SOC'SEC'NUM$,EMPL'NUM,PAY'RATE$
```

If you use variable mapping, however, you could set up the variables so that they all "connect" to a variable (let's call it EMPLOYEE'INFO). Here is a visual representation of this idea:



Now, when you refer to the variable EMPLOYEE'INFO (for instance, if you say READ EMPLOYEE'INFO), you also get the other four variables (the total information about the employee would be brought in from the data file).

You may have noticed that none of the variable names in the above diagram have a $ after them, which is AlphaBASIC's usual way of denoting string variables. This is because you specify what type of variable it is when you map a variable. AlphaBASIC knows that EMPL'NAME is a string, and knows what maximum size it can be because you also specify that in setting up the variable.

You can also form a variable "structure" like the one above using different types of variables. In the above example, EMPL'NAME would be a string variable, but EMPL'NUM might be a numeric variable. The variables can also be different sizes. You can refer to a single element of the group or to the group as a whole.

THE FORMAT OF MAP STATEMENTS

Now that you have an idea what mapped variables are like, let us take a look at the statement that sets up mapped variables. The MAP statement has the form:

```
MAPn variable{(Dim)} {{{{Type},Size},Value},Origin}
```

where n gives the level of the MAP statement. The rest of the elements after the variable name are optional, depending on the kind of variable you are defining. For example, if you are defining an array variable, you will include the optional "Dim" (short for "Dimensions") in the MAP statement. We'll discuss each of these optional elements in detail in the following sections.

If you "skip" an element in the MAP statement (for example, you want to specify the "value" but not the "size"), you must keep the comma that shows where the missing "size" would be. For example:

```
MAP1 NEW'VARIABLE,F,,23
```

The MAP statement above defines the variable NEW'VARIABLE, assigns it the data type F (for floating point), does NOT specify a size (which becomes, by default, 6), and assigns it the initial value of 23. Without the extra comma, BASIC would think that you were trying to assign a size of 23 bytes to NEW'VARIABLE—an illegal operation for a floating point variable.

MAP Level

The number that follows the MAP statement (MAPn) represents the level of the mapped variable. It must be within the range of MAP1 through MAP16. MAP statements are hierarchical in nature. For example, a variable mapped with a MAP1 statement may consist of several sub-variables mapped by using MAP2 statements. Each of those variables may in turn consist of several variables mapped by using MAP3 statements. And so on, down to MAP16.

MAP16 represents the lowest-level (or innermost) variable; MAP1 represents the highest level variable. You do not need to map levels in strict numeric sequence—for example, a MAP5 statement may follow a MAP3 statement without an intervening MAP4 statement.

You may refer to variables at any level of the hierarchy. An example may help to clarify this idea:

```
MAP1                        PATIENT'INFO

MAP2             NAME             ADDRESS       INSURANCE

MAP3       LAST    FIRST   STREET   NUM  CITY
```

The diagram above shows three levels of variables that have been mapped with MAP1, MAP2, and MAP3 statements. You may refer to the level 1 variable PATIENT'INFO as a whole, or may refer to one of the variables on levels 2 and 3 that represent sub-groups of the variable PATIENT'INFO, such as NAME, ADDRESS, or STREET.

When referring to any unformatted variable in the group, you automatically get the information in any of the variables below that variable in the hierarchy.

For example, when you refer to NAME you get the information in the variables LAST and FIRST. As BASIC allocates the variables NAME and ADDRESS, it automatically includes them (and their sub-variables) within the variable PATIENT'INFO.

The MAP statements for the variable group above might be:

```
MAP1  PATIENT'INFO
      MAP2  NAME                  ! Patient's name
            MAP3  FIRST,S,15
            MAP3  LAST,S,20
      MAP2  ADDRESS               ! Patient address
            MAP3  STREET,S,30
            MAP3  NUM,S,10
            MAP3  CITY,S,30
      MAP2  INSURANCE,B,1         ! Flag if patient has insurance
```

Notice that the variables PATIENT'INFO, NAME, and ADDRESS are unformatted variables (since nothing is specified after them, they default to unformatted type). An unformatted variable is like a storage area for all the variables that are mapped below it (in the case of NAME, the variables LAST and FIRST). This allows these "groups" of information to be moved as a unit, simply by using the unformatted variable name. For example:

```
READ #1, PATIENT'INFO
WRITE #3, NAME
HOLD'IT = PATIENT'INFO
```

Variable Names in MAP Statements

The variable name is the name that your program uses to refer to the mapped variable; it must follow the rules for AlphaBASIC variable names. However, since you may specify the type, you do not need to follow a string variable with a dollar sign.

If the variable name is followed by a set of subscripts within parentheses, the variable is assigned as an array with the dimensions specified by the subscripts, just as if a DIM statement had been used. For example, the statement:

```
MAP1 NUMBER,F,6
```

assigns a single floating point variable called "NUMBER," but the statement:

```
MAP1 NUMBER(5,10),F,6
```

assigns a floating point array with 50 elements in it (5 x 10), just as if the statement DIM NUMBER(5,10) had been executed.

Since these mapped arrays are assigned memory at compile time and not at run-time, the subscripts must be decimal numbers instead of variables.

Type Code

The type code is a single character code which specifies the type of variable to be mapped:

X - unformatted absolute data variable
S - string variable
F - floating point variable
B - binary unsigned numeric variable

If no type code is entered, AlphaBASIC assumes an unformatted variable.

Unformatted Data

Unformatted variables are used to "group" together any variables that are mapped below them. It might help to think of an unformatted variable as the "file folder" that holds all the information that is stored in the variables below it. You usually define an unformatted variable so that you can refer to a group of other variables as one unit.

The contents of unformatted data variables should only be moved to other unformatted data variables.

String Data

If the string contained within a string variable is less than the allocated size of that variable, it is terminated by a null character. If you move a string into a shorter variable, any characters that do not fit into the new variable are truncated.

String variables default to a length of 0.

Floating Point Data

Floating point numbers default to 6 bytes long. At this time, 6 bytes is the only size allowed for floating point numbers. If you specify a length, you must specify 6.

Binary Data

Binary variables may range in size from 1 to 5 bytes, giving from 8 to 39 bits of binary unsigned numeric data or 40 bits of binary signed data. This is handy for the storage of small integer data in a single byte, such as flags, or for the storage of memory references as word values with a range of up to 65535 in two bytes. All numbers are stored as two's-complement values.

Since AlphaBASIC converts all binary variables to floating point format before performing any arithmetic calculations, binary arithmetic is actually slower than normal floating point arithmetic. It is used mainly for compacting data into files and arrays where the floating point size of six bytes is inefficient.

When conversions from floating point to binary are done, any data that does not fit within the defined size of the target variable is merely lost with no error message given. As a programmer, it is your responsibility to make range checks before moving a floating point number to a binary variable area. The best way to understand this is to play with a few examples in interactive mode.

Binary numeric variables are not allowed in some instances. You cannot use a binary variable as the control variable in a FOR-NEXT loops may not use a binary variable as the control variable, although you may use them in the expressions designating the initial and terminating values of the control variable, as well as in the STEP expression.

Size

The size parameter in the MAP statement is optional but, if it is used, it must be a decimal number specifying the number of bytes to be used in the variable. If it is omitted, it defaults to 0 for unformatted and string types, 6 for floating point types, and 2 for binary types. The size parameter of floating point variables MUST be 6.

Value

An initial value may be given to any mapped variable (except an array variable) by including any valid expression in the value parameter. This value may be a numeric constant, a string constant or a complete expression including variables.

Remember, however, that the expression is resolved when the MAP statement is executed at run-time, and the current value of any variable within the value expression is the one used to calculate the assignment result. MAP statements may be executed more than once if you desire to reload the initial values.

If you omit the size parameter (such as for floating point variables), but you use the value parameter, there must be an extra comma to indicate the missing size parameter:

```
MAP1 PI,F,6,3.14159265359
MAP1 HOLIDAY,S,9,"CHRISTMAS"
```

The first example preloads the value 3.14159265359 into the floating point variable called PI. The second example preloads the letters CHRISTMAS into the string variable called HOLIDAY.

Origin

In some instances, it may be desirable to redefine records or array areas of different formats so that they occupy the same memory area. For instance, a file may contain several different record formats with the first byte of the record containing a type code for that specific record format.

The origin parameter allows you to redefine the record area in the different formats to be expected. When the record is read into the area, the type code in the first byte can be used to execute the proper routine for the record type.

Each different routine can access the record in a different format by the different variable names in that format. All record formats actually occupy the same area in memory. This feature is much like the REDEFINES verb in the COBOL language data division. Using the origin parameter can save large amounts of memory.

For instance, suppose you have three very large variables of 256 bytes each that define logical records, and that you never use these variables at the same time. By defining the variables so that they occupy the same area of memory, your program only uses 256 bytes for the variables instead of 768 bytes.

Normally, a MAP statement causes allocation of memory to begin at the point where the last variable with the same level number left off. The origin parameter allows this to be modified so that allocation begins back at the base of some previously defined variable, and therefore overlays the same memory area. If the new variable is smaller than the previous one (or the exact same size), it is totally contained within the previous one.

If it is larger than the previous one, it spills over into newly allocated memory or possibly into another variable area of the same level depending on whether there were more variables following it.

The origin parameter must be the last parameter on the line. It takes this form: an @ symbol followed by the name of the previously mapped variable whose area you wish to overlay. This variable must be on the same level as the variable you are presently allocating. If size and value parameters are not included in this statement, you may omit them with no dummy commas. For example:

```
MAP1  CUSTOMER'ID
      MAP2  NAME,S,13
      MAP2  ID'NUM,F
      MAP2  SEX,B,1
MAP1  PRODUCT'INVENTORY,@CUSTOMER'ID
      MAP2  BRAND,S,13
      MAP2  PARTNO,F,6
      MAP2  RESALE,B,1
```

The MAP statements above allocate the variable CUSTOMER'ID which takes up a total of 20 bytes. Then it allocates the variable PRODUCT'INVENTORY (also taking up 20 bytes), and specifies, by using the @CUSTOMER'ID origin parameter, that PRODUCT'INVENTORY will occupy the same space in memory as CUSTOMER'ID.

The following statements define three areas which all occupy the same 48-byte memory area, but which may be referred to in three different ways:

```
100   MAP1  ARRAY
110         MAP2  INDEX(8),F,6
200   MAP1  ADDRESS,@ARRAY
210         MAP2  STREET,S,24
220         MAP2  CITY,S,14
230         MAP2  STATE,S,4
300  MAP1  DOUBLE'ARRAY,@ARRAY
310         MAP2  UNIT(6)
320               MAP3  CODE,B,2
330               MAP3  RESULT,F,6
```

Statements 100 and 110 define an array with 8 floating point elements: a total of 48 bytes in memory. Statements 200-230 define an area with three string variables in it, for a total of 42 bytes. Normally, this area would follow the 48-byte ARRAY area in memory, but the origin parameter in statement 200 causes it to overlay the first 42 bytes of the ARRAY area instead.

Statements 300-330 define another array area of a different format with 6 elements, each element being composed of one 2-byte binary variable (CODE) and one floating point variable (RESULT). The origin parameter in statement 300 also causes this area to overlay the ARRAY area exactly.

The above scheme allows variables to be referred to in a different format than when they were entered into memory. If you load the 8 elements INDEX(1) through INDEX(8) with floating point values, and then refer to the variable STREET as a string, you get the first four floating point variables, INDEX(1) through INDEX(4), which look very strange in string format!

USING MAP STATEMENTS

MAP statements can be used as direct statements in interactive mode as a learning tool to see how variable mapping works. Interactive mode also gives you immediate feedback if you enter the MAP statement incorrectly, making it easier to learn the MAP syntax.

MAP statements are not designed to be practical in the interactive mode, however, and are best used by putting them into a program file and compiling the program.

MAP statements should come at the beginning of the program, and must come before any references to the variables being mapped. If you refer to a variable before it is mapped (such as A = 5.8), the variable is set up in the standard way. When the MAP statement is reached, it then returns an error, since the variable is already defined.

Remember that the COMPIL program offers a /M switch that reports unmapped variables in your program—useful if you want all your variables to be mapped.

Examples

Let us take a mapped variable group:

```
MAP1  ADDRESS
      MAP2  STREET,S,20
      MAP2  CITY,S,20
      MAP2  STATE,S,20
      MAP2  ZIP'CODE,S,9
```

With this group, we can do a number of things. We could print the whole group to the terminal screen:

```
PRINT ADDRESS
```

or write it to a file with a simple statement:

```
WRITE #1,ADDRESS
```

And we could sort the information in useful ways by sorting the CITY, STATE, or ZIP'CODE variables.

Another use of MAP statements is to set up arrays. The following two statements produce identical arrays:

```
MAP1  ARRAY(10),F,6
DIM ARRAY(10)
```

Both statements produce arrays containing ten floating point variables, referred to as ARRAY(1) through ARRAY(10). The first statement, however, defines its placement in memory in relation to other mapped variables.

Similarly, the statements:

```
MAP1  ARRAY(5)
      MAP2  ARRAY(20),F,6
```

produce the same two-dimensional array as:

```
DIM ARRAY(5,20)
```

As another example, these statements:

```
DIM WIDTH(10)
DIM HEIGHT(10)
```

Produce two arrays, each with ten variables. But these:

```
MAP1 AREA(10)
     MAP2 WIDTH,F,6
     MAP2 HEIGHT,F,6
```

Produce one array with twenty variables in it. The variables are still referred to as WIDTH(1) through WIDTH(10) and HEIGHT(1) through HEIGHT(10), but their placement in memory is quite different. The WIDTH variables are interlaced with the HEIGHT variables, giving WIDTH(1), HEIGHT(1), WIDTH(2), HEIGHT(2)... WIDTH(10), HEIGHT(10).

There are also ten unformatted variables AREA(1) through AREA(10), which each contain the respective pairs of WIDTH-HEIGHT variables in tandem. Referencing one of these AREA variables references an unformatted item composed of the WIDTH-HEIGHT pair of the same subscript.

When using MAP variables, keep these points in mind:

- Remember to use an unformatted variable to group more than one mapped variable together.
- If printing a string or unformatted variable, any floating point or binary variable mapped beneath it will not display properly.
- When using string variables, do not assign them a size that is much larger than you need for what that variable will hold.
- Use only the variables you need. For example, if you want an address, do not break down the address into partial strings unless you need to refer to that part of the address. The more variables you define, the more memory you use.

USING MAPPED DATA WITH DATA FILES

You may often use MAP statements to define groups of information that will be transferred in and out of random disk files. Because of the structure of sequential files, it is difficult to input and output mapped variables as "groups." The information will only transfer correctly if all of the variable information is carefully "padded" to the proper lengths.

Therefore, it is best to use random files when your program uses mapped variables that are grouped.

Let us take a look at an example that defines a logical record. Our program probably uses a file that contains a large number of logical records in this format, each record containing information about a single check.

In effect, MAP statements give us a way to form a template in memory into which we can read information from the random file and transfer information from the program to the file. This allows us to quickly and efficiently read in an entire group of information whose elements may be of different types and sizes, and to access information in that group flexibly and simply. For example:

```
MAP1  CHECK'INFO
      MAP2  CHECK'NUMBER,F,6
      MAP2  DATE,S,6
      MAP2  AMOUNT,F,6
      MAP2  TAX'DEDUCTABLE,B,1
      MAP2  PAYEE,S,20
      MAP2  CATEGORY,S,20
      MAP2  BANK'ACCOUNTS
            MAP3  SAVINGS,S,20
            MAP3  CHECKING,S,20
            MAP3  TERM,S,20
! Define a file that contains an account balance:
      MAP2  ACCOUNT'BALANCE,S,22,"DSK1:BALANC.DAT[200,1]"
```

Once these MAP statements have been executed, we can access the group of variables as a whole by specifying CHECK'INFO, or we can access specific sub-fields in the record (for example, BANK'ACCOUNT or CHECKING). For example, once the program has filled the variables with the specific data for a record, we can write it to the file by saying:

```
WRITE #1,CHECK'INFO
```

Another section of the program might want to read in a record of data. By using a simple statement:

```
READ #1,CHECK'INFO
```

We can bring in all the information related to that check. This is much easier than using 11 statements to read in the 11 variables contained in CHECK'INFO!

We can also use the individual items. For example:

```
IF AMOUNT > 100 GOTO VERIFY'TRANSACTION
```

HOW VARIABLES ARE ALLOCATED IN MEMORY

This section goes into the details of how memory is mapped. If you are not interested in this, you may want to skip on to the next chapter.

During compilation, AlphaBASIC allocates memory storage for all defined variables in an area that is contiguous and predictable. The compiled program references all variables through an indexing scheme.

Each variable in the working storage area has a representative item in the index area which contains all the information needed to define and locate that variable. The working storage area therefore contains only the pure variables themselves without any associated or intervening descriptive information. The index area is a separate entity, physically located before the working storage area in memory.

AlphaBASIC normally allocates variable storage as it encounters each variable during compilation. Since this scheme is not easily followed by human beings, a different method must be derived which can override normal allocation processes if you wish to have the variables allocated in a predetermined manner. Also, the disk I/O system requires that variables used be in a specific relationship to each other when used in some of the more sophisticated programs.

The MAP statement has been included in AlphaBASIC for the purpose of allocating variables in a specific manner. MAP statements are non-executable at run-time, but merely direct the compiler in the definition and allocation of the referenced variables.

Each MAP statement contains a unique variable name to which the statement applies. When the compiler encounters this statement, it allocates the next contiguous space in working storage as required and assigns it to that variable. All variables not defined in a MAP statement are then automatically assigned storage in sequence, for total compatibility with existing standards.

To eliminate potential allocation problems, AlphaBASIC forces all MAP1 level variables to begin on an even memory address. This ensures that certain binary and floating point variables will begin on word boundaries for assembly language subroutine processing. The instruction set performs most efficiently when word data is aligned on word boundaries.

In interactive mode, if an error occurs in the syntax of the statement, the variable will already have been added to the tree in memory. This is the reason MAP statements are not useful in interactive mode (except for learning and debugging).

LOCATING VARIABLES DURING DEBUGGING

A command has been implemented which can assist you in locating the mapped variables and in understanding the allocation techniques used by the AlphaBASIC variable mapping system. It is valid only as a BASIC interactive command and has no meaning if used within a program text. The command has the general format of an at sign (@) followed by a variable name.

The system searches for the requested variable and prints out all parameters about the variable for you on the terminal. This may actually be two definitions, since the variable "A" may actually be two different variables; one would be a single floating point number and the other would be a subscripted array.

The information returned about the variable is:

- The type of variable (string, binary, etc.)
- The dimensions of the array if the variable is an array
- The size of the variable in bytes
- The offset to the variable from the base of the memory area which is used to allocate all variables

If you enter a reserved word (such as @PRINT) the system tells you that the name is a reserved word.

The general format of the definition line returned by the system is:

```
memory-type var-type {dimensions}, size n, location
```

For actual examples of the definition line, see the Examples section below. Memory-type means the method of memory allocation used when defining the variable. The memory-type may be MAPn (where n is a number from 1 to 16), FIXED or DYNAMIC variables. FIXED variables are not defined by a MAP statement and are allocated automatically when the compiler finds references to them in the program. This is the normal method used to allocate variables.

DYNAMIC variable arrays are allocated by a DIM statement or by a default reference to a subscripted variable.

If the array is dynamic and has not been allocated yet, the subscript values are replaced by the letter "X" to indicate that they are not known at this point. Remember that any variable defined in a MAP statement which is in a lower level relative to another variable inherits all subscripts from that higher level variable.

The size of the variables are given in decimal bytes. In the case of arrays, the size represents the size of each single element within the array.

The location of the variable is a little tricky to explain, since it is actually an offset to the base of a storage area that is set aside for the allocation of user variables.

As each new variable or array is allocated, it is assigned a location which is relative to the base of this storage area. The location information given here is an example to help you understand the relative placement of the variables in the mapping system, and does not represent the actual memory locations which they occupy.

There are two distinct areas in use for variables, and thus the offsets of the variables are to one of these two areas. All FIXED and MAP1 through MAP16 variables are allocated in the fixed storage area, while all DYNAMIC arrays are allocated in the dynamic array storage area.

As dynamic arrays are dimensioned, their positions may shift relative to one another and relative to the dynamic storage area base. Variables in the fixed storage area never change position relative to each other or to the storage area base.

Array location information that is given is only pertinent to the base of the array itself, which is the location of the first element within the array. The actual range of locations used by the array may or may not be contiguous in memory depending on whether overlapped dimensioning techniques are being used in the MAP statements. Simple (non-array) variables are defined as a location range which tells exactly where the entire variable lies within the storage area.

Keep in mind that this "@" command is to assist you in following the allocation of variables, particularly in more complex mapping schemes. A few minutes at the terminal with direct MAP statements followed by "@" commands will help you see how the mapping scheme works.

Examples

Given the sample MAP statements below:

```
MAP1  CUSTOMER'ID
      MAP2  NAME
            MAP3  FIRST,S,15
            MAP3  LAST,S,15
      MAP2  ADDRESS
            MAP3  STREET,S,15
            MAP3  CITY,S,10
            MAP3  STATE,S,2
```

```
MAP2  PHONE
      MAP3  HOME,B,3
      MAP3  BUSINESS,B,3
MAP2  TRANSACTIONS(12)
      MAP3  BALANCE,F,6
      MAP3  CREDIT,F,6
      MAP3  YTD,F
```

Here are the results of using the @ command in interactive mode to determine the locations of several of the variables above:

```
READY
@CUSTOMER'ID RETURN
MAP1 Unformatted, size 279, located at 0-278

@TRANSACTIONS RETURN
MAP2 Unformatted Array (12), size 18, base located at 63

@CITY RETURN
MAP3 String, size 10, located at 45-54

@HOME RETURN
MAP3 Binary, size 3, located at 57-59

@CREDIT RETURN
MAP3 Floating point Array (12), size 6, base located at 69
```

We can also use the @ command to locate unmapped variables. For example:

```
READY
DIM A(2,3) RETURN
@A RETURN
Dynamic Floating point Array (2,3), size 6, base located at 103

A = 15 RETURN
@A RETURN
Fixed Floating point, size 6, located at 72-77
Dynamic Floating point Array (2,3), size 6, base located at 103
```

Note that we allocated two different variables: a fixed floating point variable, A, and a dynamic floating point array variable, A(2,3).

Chapter 16 - AlphaBASIC File Input/Output System

This chapter contains information on creating and using disk files from within your AlphaBASIC programs. The following subjects are discussed:

- How to identify files
- Input/output statements
- Sequential files
- Random files
- How to lock files
- File statements

We also provide a sample program that uses file input/output.

WHAT IS A DISK FILE?

A disk file is a group of data items stored on a hard disk device. AlphaBASIC programs are usually stored on the disk as a disk file. But AlphaBASIC can use other disk files to access and store data. In many applications, using disk files is much more efficient and versatile than using user input or DATA statements to obtain data. And it is the only efficient way to store data for later reference.

AlphaBASIC supports two kinds of data files—sequential access and random access disk files. You may write data either in ASCII characters, or in packed binary formats.

Files that AlphaBASIC programs create are compatible with all other system programs, and AlphaBASIC files may be interchanged with files from other languages. That is, AlphaBASIC data files can be read and manipulated by programs written in other languages. Conversely, files created by other languages and system utilities may be read and manipulated by programs written in AlphaBASIC.

Note the sample program at the end of the chapter—it demonstrates the use of random files. In it, you can see how to define a logical record; compute the logical record blocking factor; allocate, open and close a random file; search for a file; and write and read data to and from a file.

Since the I/O processes differ somewhat depending on whether you want to use sequential or random data files, we discuss sequential and random files generally before getting into the specific commands you can use to work with these files.

FILE CHANNELS

All references to a file are made by specifying a file channel number, which may be any integer value from 0 to 65535. You might think of the file channel number as a pipeline between your AlphaBASIC program and the data file, through which data can be transferred.

For example, when you first create a file, you will assign it a file channel number (say, #1). Then, whenever you refer to that file, you use that number to tell AlphaBASIC which file you are talking about (say, READ #1). This allows you to have many files ready for use at the same time.

Once you close a file, the file channel is no longer associated with it, and you may open another file on that file channel. You may never have two files open at the same time that have the same file channel number. The file channel always follows the verb (such as OPEN) in any file input/output statement, and may be any numeric expression which is preceded by a pound sign (#). For example:

```
OPEN #1,"PAYROL.DAT",INPUT
```

or:

```
OPEN #file'channel'num,"PAYROL.DAT",INPUT
```

or:

```
OPEN #(number'needed - num'in'stock),"INVEN.DAT",OUTPUT
```

File channel #0 is defined as your terminal. You can use this in sequential file statements to write to your terminal at run-time if you wish. Random files cannot use file channel zero.

There is no absolute limit to the number of files that may be open at any given time in a program, but since each file requires a certain amount of memory, there is a practical limit to this number based on how much memory is available in your user memory partition.

INPUT/OUTPUT STATEMENTS

The following table shows the AlphaBASIC program statements that you use to input and output data to and from files:

ACTION	PROGRAM STATEMENTS	
	Random Files	Sequential files
Read Data	READ	INPUT, INPUT LINE
Write Data	WRITE	PRINT
Read Data (Locking record)	READL	
Write Data (Locking record)	WRITEL	

SEQUENTIAL FILES

Sequential disk files are the easiest to understand and to implement in AlphaBASIC. AlphaBASIC writes data to a sequential file in ASCII format, and stores numeric data as ASCII string values, if possible. A sequential data file usually has the extension .DAT, and AlphaBASIC uses .DAT as the default extension.

The typical sequential data files are normal ASCII files in all respects, and you may edit them by using AlphaVUE, the printer spooler, or any of the other system utilities. To open a sequential file, use the OPEN statement (see below), specifying INPUT, OUTPUT or APPEND mode.

Use the PRINT statement (followed by a non-zero file channel number) to write data to sequential files. The PRINT statement automatically appends a carriage return/linefeed to your data in the same manner that it does when sending data to a terminal display. PRINT also converts floating point and binary data to printable ASCII form.

Use the INPUT or INPUT LINE statements (followed by a non-zero file channel number) to read data from a sequential file. See the examples below.

The following sections contain step by step instructions for using sequential files in the various modes (the commands used are discussed in detail later in this chapter):

Using Sequential Files for Output

Use the LOOKUP command to see if the file already exists.

1. When you output to a sequential file, you are creating a brand new file. If a file of the same name and extension already exists in the account you are writing to, AlphaBASIC automatically deletes the old file for you before it opens the new output file. Therefore, if you don't want AlphaBASIC to delete an existing file, be sure to use the LOOKUP command before you open a file for output to make sure that such a file does not already exist. If the file already exists, you can go ahead and open it (if you want AlphaBASIC to delete the existing file for you) or you can choose another file name and use the LOOKUP command again to see if that file already exists.

2. Use the OPEN statement to open the file for output. If your computer uses the LOKSER system, your file is open for exclusive use of the file when you open it for output. See the section on LOKSER below for more information.
3. Use PRINT statements (specifying the file channel number associated with the file by the OPEN statement) to write data to the file.
4. When finished, use the CLOSE statement to close the file.

Using Sequential Files for Input

1. You will use a file for input only if that file already exists and contains data. You may wish to use a LOOKUP statement to make sure the file exists before trying to open it.
2. Use the OPEN statement to open the file for INPUT. If your computer uses the LOKSER system, your file will be OPEN for shared use of the file.
3. Use INPUT LINE or INPUT statements to read data from the file (specifying the file channel number associated with the file by the OPEN statement).
4. Check the EOF function after each input to make sure you haven't read beyond the end of the file.
5. When finished, use the CLOSE statement to close the file.

Using Sequential Files in Append Mode

1. Use the LOOKUP command to see if the file exists. If it does not, check your file specification.You will use a file in append mode only if that file already exists.
2. Use the OPEN statement to open the file for append. If you use LOKSER, your file will be open for exclusive use of the file.
3. Use the PRINT statement (specifying the file channel number associated with the file by the OPEN statement) to write data to the end of the file.
4. When finished, use the CLOSE statement to close the file.

RANDOM FILES

Random access, or direct access, files are more complex than sequential files, but offer a much more flexible method for storing and retrieving data in different formats. Random files are written in "unformatted" or packed data mode. Random file disk blocks are contiguously allocated on the disk. The major advantages of random files over sequential files are the speed and flexibility with which you may access data in a random file.

You may only open a sequential file for input OR output, but you may open a random file for input and output simultaneously. Accessing data in a sequential file requires that you step through the file record by record.

In the case of a random file, however, you may access any record without referring to any other record in that file. In addition, random files can contain data in any format supported by AlphaBASIC (unlike sequential files, which may only contain ASCII data). Random files also have a default extension of .DAT.

Logical Records

All program accesses to random files are made by using the "logical record" approach. A logical record is defined as a fixed number of bytes whose format is explicitly under control of the program performing the access. Physical blocks on the disk are each 512 bytes long, and each random file must be pre-allocated as some given number of these 512-byte blocks.

Logical records may be any length from 1 byte to 512 bytes (logical records can never overlap physical disk blocks). AlphaBASIC automatically computes the number of logical records that fit into one disk block, and performs the blocking and unblocking functions for you.

For example, if your logical record size is defined as 100 bytes, then each block on the disk contains 5 logical records with the last 12 bytes of each block being unused.

The most efficient use of random files comes when the logical record size is a number that divides evenly into 512 (256, 128, 64, etc.).

Figuring Out How Many Blocks Your File Needs

Random access files are pre-allocated once, using the ALLOCATE statement, which gives the number of physical 512-byte blocks to allocate. It is up to you to calculate the maximum number of logical records required in the file, and then to calculate how many disk blocks are required to completely contain the number of logical records you desire.

For instance, assume the logical record size is 100 and you need a maximum of 252 logical records in the file. Each disk block is 512 bytes, and therefore contains 5 logical records. This number, 5, is called the "blocking factor," because it is the number of records that will fit into each block of disk space. You need 252 logical records, so dividing 252 by 5 gives 50 full disk blocks plus 2 logical records remaining.

Since the file must be allocated in whole disk blocks, you need 51 blocks, which gives you a maximum of 255 logical records. These logical records are referred to in your program as records 0 through 254, since the first record of any random file is record 0, unless you have used FILEBASE. See "FILEBASE" below.

When figuring out how many blocks you need to have, use this formula:

```
BLOCKS = RECORDS'NEEDED/(INT(512/RECORD'SIZE))
```

And round the result (BLOCKS) up to the next integer value. RECORDS’NEEDED is the number of records your program will need to handle, and RECORD'SIZE is the size (in bytes) of each individual record.

When your record size does not divide evenly into 512 bytes, it is a good idea to consider expanding it so that it does. This is for two reasons:

1. This leaves you room for future expansion of the data in the record.
2. You will be using the same number of physical disk blocks whether or not you expand the record size, so you do not save anything by not doing so.

When you are opening a random file, you must specify the logical record size in the OPEN statement (also specifying RANDOM or RANDOM'FORCED mode); it is possible to get things fouled up if you do not have the record size correct. No logical record size is maintained within the file structure itself.

This fact does make it nice in one respect—a file which is accessed by many programs can have its record size expanded without recompiling all the accessing programs. Here is how it works:

Assume (as an example) that you have a file which is considered the parameter descriptor file for all other files in the entire system. This file gives the record size as 100 bytes for the vendor name and address file.

All programs which refer to the vendor file first read this parameter file to get the size of the vendor file logical record. The programs then set the size into a variable and use this variable in the OPEN statement for the record size.

Each READ, READL, WRITE or WRITEL statement then manipulates the 100 bytes of data by reading or writing to or from variables whose size totals 100 bytes. Let's say you now want to expand the file record size to 120 bytes and that most of the programs do not have to make use of the extra 20 bytes until some time in the future.

You write a program which copies the 100-byte record file into a new 120-byte record file. Then you update the main parameter file to indicate that the new record size for the vendor file is 120 bytes instead of 100.

Each program now opens the file using the new 120-byte record size (since it is read in from the parameter file at run-time), but only READs or WRITEs the first 100 bytes of each record due to the variables used by the READ, READL, WRITE and WRITEL calls.

Accessing Random Files

The following sections show you the normal procedure for using Random files for each type of access:

USING RANDOM FILES IN RANDOM MODE

1. Use the LOOKUP command to see if the file already exists. If it does, skip down to step #3.
2. If the file doesn't exist, you must create it. First, decide what size the logical records will be (in decimal bytes). Then compute the blocking factor as discussed above. Use the ALLOCATE command to create the file with the number of disk blocks needed.
3. Use the OPEN statement to open the file for RANDOM processing. Specify the size of the logical records in the file, and the record-number variable that will hold the number of the logical record you are currently accessing.

If you are using the LOKSER system (explained below), your file will be OPEN for exclusive access. You may wish to use WAIT'FILE, in case the file is in use when you try to open it.

4. Use READ and WRITE statements (specifying the file channel number associated with the file by the OPEN statement) to read and write data in the file. Remember to change the record-number variable to the correct record number before performing each read or write operation so that you access the logical record you want. Make sure that the record-number variable contains a valid record number before performing the file I/O.
5. When you are finished reading and writing the file, use the CLOSE statement to close the file.

USING RANDOM FILES IN RANDOM'FORCED MODE

1. When a random file is open in this mode, a READ operation forces a disk access, even if the requested block is already in memory. Likewise, if a WRITE is performed, the block is reread, modified and forced out to disk, even if the buffer is not yet full. This mode makes it possible for users to share random files.

If you are using the LOKSER system (explained below), RANDOM'FORCED causes the file to be OPENed for shared access. You may want to specify WAIT'FILE and/or WAIT'RECORD also, in case the file or record is in use when you try to access it.

2. If you want to only read the contents of a record, use the READ statement.
3. If you want to read in and then update, use the READL statement. Then use either a WRITE statement to write the record back to the file, or use an UNLOKR statement to unlock the record again.
4. If you want to write to a new or blank record, use the WRITEL statement.
5. Close the file.

ISAM FILES

A special type of random file, opened in INDEXED and INDEXED'EXCLUSIVE modes. See Chapter 20 for information.

USING FILE LOCKING (LOKSER)

LOKSER is an AMOS file and record locking system that allows you to protect your files against the type of problems that can arise when more than one user accesses the same file at the same time.

See the *LOKSER User's Manual* that applies to your system for an in-depth discussion of file and record locking.

Installation of the LOKSER system is optional—if you have the LOKSER system installed, there will be a few extra commands and additions to commands that you will need to know in order to use files. All of these new commands are explained in this section. The following table shows how the file is OPENed for each type of access:

Type of File	Mode	File Status
Sequential	INPUT OUTPUT APPEND	Shared Exclusive Exclusive
Random	RANDOM RANDOM'FORCED INDEXED INDEXED'EXCLUSIVE	Exclusive Shared Shared Exclusive

File Locking for Sequential Files

With sequential files, the file locking is done automatically by the monitor if you use the LOKSER system. If you open a sequential file for output or append, the monitor will automatically open the file for your exclusive use. Since you are capable of modifying the file in these modes, you have to have exclusive use of the file. If you open the file for Input, the monitor will automatically open the file for shared use, since you are not going to be changing the file.

You can specify WAIT'FILE when you open sequential files. This causes your program to wait if the file you want to access is being used when you try to open it. If you do not use this wait clause, if you try to access a file and the resource you need is already locked, LOKSER will prevent you from accessing it and return an error code.

What happens then depends on your program—if your program has error trapping enabled, the actions specified by your error handling routine will be performed; if your program does not have error trapping enabled, your program will be interrupted and you will be returned to monitor command level. For information on using error trapping to deal with the usual system errors see Chapter 18.

If you specify WAIT'FILE, LOKSER will not report an error if the program tries to access a file that is already locked, but will instead put the job using the program to sleep until the file is available.

File Locking for Random Files

With random files, you have a choice of how you would like to open the files. If you open a file using RANDOM mode, the monitor will open the file for your exclusive use. If you specify RANDOM'FORCED mode, the file is opened for shared use. The RANDOM'FORCED mode is useful because it allows many users to access the same file at the same time without any updating problems.

When a file is opened for shared access, the LOKSER system will make certain that any records you are working with are locked before they are changed. LOKSER will recognize if you are trying to WRITE to an unlocked record, and halt your program, displaying a "Record not locked" error message—unless you handle the error with an error trapping routine.

When locking specific records, LOKSER must lock an entire physical block on the disk. This may mean that it will lock more than one record, depending on how your logical records are defined, since multiple records may appear in one physical block.

You can also use ISAM (Indexed) files in the same manner. If you specify INDEXED when you open the file, AlphaBASIC will open it for shared use. Or, you can specify INDEXED'EXCLUSIVE when you open the file, which opens the file for your exclusive use. See Chapter 20 for information on ISAM files.

The WAIT'FILE option applies to random files, also. Random files opened in RANDOM'FORCED and INDEXED modes have another option called WAIT'RECORD, which works just like WAIT'FILE, except that it applies to records inside the file.

FILE STATEMENTS

AlphaBASIC automatically closes all open files when the program exits or when a CHAIN statement is executed, if the files have not already been explicitly closed by use of a CLOSE statement.

If your program should be terminated abnormally (for example, by something like a system crash or power failure), and a file was not closed with a CLOSE statement, AlphaBASIC may not have updated the last record. However, the file will be unlocked if it was locked.

All file statements are valid as direct statements, but AlphaBASIC closes any open files before it executes another RUN command. This prevents statements in an executing program from reading or writing to files which were opened by a direct statement. Each open file requires about 580 bytes of free memory for buffers and control blocks.

The following sections in this chapter show you the general format of each of the file statements and give detailed examples of their uses.

ALLOCATE

The ALLOCATE statement prepares a blank random file on the disk, which you may then open for random processing. An attempt to allocate a file which already exists results in an error message. A random file need only be allocated once and may then be opened for random read/write operations as many times as desired. The statement format is:

```
ALLOCATE filespec,number-of-blocks
```

As in the OPEN statement, the filespec is any string expression which evaluates to a legal file description. The number-of-blocks is a floating point expression which represents the number of physical 512-byte disk blocks to be allocated to the file. For example:

```
ALLOCATE FILE$, BLOCKS
ALLOCATE "NEW.DAT",20
```

Random files can also be created at command level using the command CREATE (see the CREATE reference sheet in the *System Commands Reference Manual* that applies to your system).

CLOSE

The CLOSE statement ends the transfer of data to or from a file. Once a file has been closed, no further references are allowed to that file until another OPEN statement for that file is executed. Any files that are still open when the program exits are closed automatically. Closing a file that is locked by the LOKSER system unlocks that file. The format of the CLOSE statement is:

```
CLOSE #file-channel
```

where ***#file-channel*** specifies the file channel number associated with the file you want to close. For example, if you have previously opened a file called VENDOR.DAT:

```
OPEN #3,"VENDOR.DAT[200,1]",INPUT
```

to close that file, use the statement:

```
CLOSE #3
```

EOF(X)

The EOF function returns a value giving the status of a sequential file whose file channel number is X. The file is assumed to be open for INPUT. The values returned by the EOF function are:

-1	if the file is not open or the file channel number X is zero. A file channel number of zero indicates the terminal is being used as the file.
0	if the file is not yet at end-of-file
1	if the file has reached the end-of-file condition

Due to the method used by the operating system for processing files, the end-of-file status is not achieved until after an INPUT statement has been executed.

Any INPUT statements which reach end-of-file return numeric zero or null string values forever more. This means the normal sequence for processing sequential input files would be to INPUT the data into the variables and then test the EOF(X) status before actually using the data in those variables, since if an end-of-file has been reached that data will be no good.

End-of-file should only be tested for sequential input files. Files open for output or for random processing always return a zero value.

FILEBASE

During normal operation, AlphaBASIC refers to the first record in a random file as record number zero (i.e., you set the record number variable to zero to access the first record in the file). In some applications you may want AlphaBASIC to refer to this first record by some number other than zero: for instance, to allow you to use zero to flag some special condition, such as a deleted record. The FILEBASE command allows you to set the number used to refer to the first record. For example:

```
FILEBASE 1
```

tells AlphaBASIC that the first record in the file is record number one, not record number zero. You may use any integer argument with FILEBASE.

Note that FILEBASE does not associate its value with a file, but only takes effect when you execute the program it is in. If one program uses a FILEBASE command when referring to a file, all other programs which refer to that file should also use a FILEBASE command with the same value to be able to access the same record with the same record number.

INPUT

Once a sequential file has been opened for input, you may use a special form of the standard AlphaBASIC INPUT statement to read data from the file. The INPUT statement uses a file channel number corresponding to the file channel assigned in the OPEN statement.

The variables in the list may be either numeric or string variables, but should follow the format of the data in the file being read. You can INPUT numeric data into string variables, but an attempt to INPUT a string variable into a numeric variable will set that variable to 0. The general format is:

```
INPUT #file-channel,var1{,var2,...varN}
```

During the reading of the input data into the variable list, all leading spaces are bypassed unless they are enclosed within quotes, just as in the normal form of the INPUT statement.

Also, all carriage returns and linefeeds are bypassed, allowing the file created by the PRINT statements to contain formatted line data if desired. Commas, spaces and end-of-line characters all terminate numeric data and then are bypassed.

As with the non-file version of the INPUT statement described in Chapter 10, the data being input must be in the proper format. In the file version of the INPUT statement, that means you must properly separate the data when you first write it out to the file using the PRINT statement. Here are the rules to follow when writing data to a file:

1. Separate all floating point data with spaces or commas.
2. Separate all string data with commas.
3. If you have both floating point and string data in the same statement, separate them with commas.

Keep in mind the characteristics of the PRINT statement when writing data to a file, so that you do not conflict with the rules above. See Chapter 10 for details on PRINT.

Using PRINT to send data to a file formats that data in exactly the same way that it would if you were to use PRINT to send data to the terminal screen. Remember that PRINT does not separate the data with commas for you. For example, the following statement:

```
PRINT #100 "HELLO","AGE","DATE"
```

sends the following line to the file:

```
HELLO      AGE        DATE
```

If you try to use INPUT to read that data in the file into three different string variables, the first variable will contain:

```
HELLO
```

or:

```
HELLO      AGE        DATE
```

and the other two string variables will contain null data. In the first case above, we are assuming that no STRSIZ command was set, so the first string variable only picked up the first ten characters of the string. If STRSIZ was set to, say, 40, you would see the second display.

To read the data above as three separate pieces of string data, you must remember to separate the data by explicitly placing commas into the file. For example:

```
PRINT #100 "HELLO,AGE,DATE"
```

sends the following line to the file:

```
HELLO,AGE,DATE
```

which will be input correctly by the following statement:

```
INPUT #100 A$,B$,C$
```

Note that the statement:

```
PRINT #100 "HELLO", ",", "AGE", ",", "DATE"
```

will improperly format the data in the file because the unquoted commas above will cause PRINT to separate the data with spaces as well as commas:

```
HELLO      ,     AGE        ,     DATE
```

Using semi-colons instead of the unquoted commas will solve this problem. For example:

```
PRINT #100 "HELLO";",";"AGE";",";"DATE"
```

INPUT LINE

After a sequential file has been opened for input, the data can be read from the file by a special form of the INPUT LINE statement which uses a file channel number corresponding to the file channel assigned in the OPEN statement.

The variables in the list may be either numeric or string variables, but should follow the format of the data in the file being read. String variables will accept numeric data, but if you try to read string data into a numeric variable, the numeric variable will equal 0.

The general format is:

```
INPUT LINE #file-channel,variable
```

The INPUT LINE statement operation is identical to that of the INPUT statement with the exception that input into a string variable accepts the entire line up to but not including the carriage return and linefeed that ends the line. This allows commas, quotes, blanks and other special characters to be input. Also, INPUT LINE accepts blank lines as input.

You will usually use INPUT LINE specifying one string variable to read in one line of the file at a time.

KILL

The KILL statement erases one file from the disk. It does not need a file channel number and no OPEN or CLOSE need be performed to KILL a file. The format for the KILL statement is:

```
KILL filespec
```

For example:

```
KILL "NEWDAT.DAT"
```

As in the OPEN statement, the ***filespec*** is any string expression which evaluates to a legal file description. KILL assumes an extension of .DAT. If you try to erase a file that does not exist, you see the error message:

```
?File not found
```

You may not erase a file that exists in an account outside of the project you are logged into. For example, if you are logged into account [110,2] and the program you are running tries to kill a file in account [200,1], you see:

```
?Protection violation
```

If you have LOKSER on your system, and you try to delete a protected file, you will get an error message.

LOOKUP

The LOOKUP statement looks for a file on the disk and returns a value which tells you if the file was found and, if so, how many disk blocks it contains. The format for the statement is:

```
LOOKUP filespec, result-variable
```

As in the OPEN statement, the ***filespec*** is any string expression which evaluates to a legal file description (the default extension is .DAT). The file specification can be up to 48 characters in length. The ***result-variable*** is any legal floating point variable which receives the result of the search. The LOOKUP result-variable may return:

0	File was not found
Positive #n	File was found; it is a sequential file, and contains n disk blocks.
Negative #n	File was found; it is a random file, and contains n disk blocks.
0.5	File was found and contains zero blocks.

Remember that the number returned by LOOKUP is the number of physical disk blocks used by the file. You must multiply this number of 512-byte blocks by the file's blocking factor to find out how many logical records the file contains. For example, after you execute:

```
LOOKUP "PAYROL.DAT",BLOCKS
```

the variable BLOCKS contains the number of disk blocks in the file PAYROL.DAT, or a 0 if the file does not exist. For example, say that BLOCKS = 40, and the blocking factor for the file is 4.Multiplying 40 by 4 gives you 160 logical records.

OPEN

You must open a file before you can transfer data to or from it. The OPEN statement assigns a unique file channel number to a file and also specifies the name that is either to be given to an output file, or to be used in locating an input file. The format is:

```
OPEN #file-channel,filespec,mode{,record-size}
     {,record#-variable}{,WAIT'FILE}{,WAIT'RECORD}
```

The parts of this OPEN command are defined as follows:

file-channel	Any numeric expression which evaluates to an integer from 0-65535 (0 is defined as the user terminal and treated as such). Random files cannot use #0 as the file channel.
filespec	Any string expression up to 48 characters long which evaluates to a legal file description. May be a string variable or string literal. String literals must be enclosed in quotation marks.
mode	The mode for opening the file. See below.

Record-size	An expression which dynamically specifies at run-time the logical record size for read/write operations on the file. Not for Sequential files.
Record#-variable	A non-subscripted numeric variable which must contain the record number of the desired random access for READ or WRITE statements when they are executed. It must be a floating point variable—not a literal number. Not used in sequential files.
WAIT'FILE	Optional clause that causes your program to wait until the specified file is available for use. Used only with LOKSER.
WAIT'RECORD	Optional clause that causes your program to wait until the specified record is available for use. Used only with LOKSER.

If your program tries to read or write to a file which has not been opened, you will see:

```
IO to unopened file in line nnn_
```

and the program will be aborted. If your program tries to read or write to a file which is locked, your program will end and a message will be printed that informs you that the file is in use. Of course, if you use an error trapping routine, you can prevent your program from being aborted.

The filespec string may be as brief as the name of the file, in which case it is assumed to have an extension of .DAT and to reside in the disk account you are logged into. Or it may be a complete file specification, if you desire, giving the explicit location of the file, which may be in another account or even on another disk drive. Some examples are:

```
OPEN #1,"DATFIL",INPUT
OPEN #15,"PAYROL.TMP",OUTPUT
OPEN #A,C$,OUTPUT
OPEN #3,"DSK1:OFILE.ASC[200,20]",OUTPUT
OPEN #1,"VENDOR.DAT",RANDOM,100,RECNUM,WAIT'FILE
OPEN #1 + X,MID$(A$,2,3),OUTPUT
OPEN #25,"MASTER",INDEXED,80,RELKEY
```

The OPEN statement references the file by its actual ASCII filespec in the standard operating system format. Most of the other file commands refer to the file channel number which is assigned in the OPEN statement.

OPEN Modes

The mode specifies how the file is to be opened:

INPUT	Opens an existing sequential file for input operations. For LOKSER, access is shared.
OUTPUT	Opens an existing sequential file for output operations. For LOKSER, access is exclusive.
APPEND	Opens an existing sequential file so that you can add data to the end of the file. LOKSER access is exclusive.

RANDOM	Opens an existing random file for random read/write. LOKSER access is exclusive.
RANDOM'FORCED	Opens an existing random file for random read/write. LOKSER access is shared.
INDEXED	Opens an ISAM data file and primary index file. LOKSER access is shared.
INDEXED'EXCLUSIVE	Opens an ISAM data file and primary index file. LOKSER access is exclusive.

PRINT

Once you have opened a sequential file for output, you will use a special form of the PRINT statement using the file channel assigned in the OPEN statement to write data to it.

All the techniques available to you when you use the normal form of the PRINT statement (which outputs to the terminal) are also available for sending data to a file, including PRINT USING for formatted data.

PRINT writes data to the file in the same format as it would appear if you used PRINT to send the data to a terminal display (i.e., if you left off the file channel number). Here is the format for PRINT, and some examples:

```
PRINT #file-channel, expression-list

PRINT #1 A; B; C
PRINT #4 USING A$, A, SQR(A)
PRINT #Q1 USING "###.##", A1(10);
PRINT #1 "THIS IS A SINGLE LINE"
PRINT #file'number "WRITE TO","PRINT ZONES",
```

See INPUT, above, for the required format for data when reading data from a file. For more information on PRINT, see Chapter 10.

READ

The READ statement reads a selected logical record from an open random file. The logical record which is transferred by the system I/O is the one whose record number is currently in the record-number variable mentioned in the OPEN statement. The format of the READ statement is:

```
READ #file-channel,var1{,var2...,varN}
```

For example, if your OPEN statement is:

```
OPEN #1,"STOCK.DAT",INPUT,100,5
```

and your READ statement is:

```
READ #1,PART'NUMBER
```

Then PART'NUMBER will be filled with the data in the 5th record of the file STOCK.DAT.

The variables in the READ statement may be any format, but they obviously should match that of the designated record format. The data is read into the variables as unformatted bytes, without regard to variable type. Of course, if it is transferred into the wrong type of variable, you will get garbage and/or errors when you use that variable. The data is transferred into each variable until the variable has been completely filled. Then the next variable in the list is filled, and so on.

If the record is longer than the variable list specifies, all excess data in the record will not be transferred. If you try to transfer more data than is in the logical record size, you will get an error message.

The most efficient use of random files comes when the variable or variables used are mapped by a MAP statement to the exact picture of the record format in use. See Chapter 15 for information on MAP statements.

Also see the sample program at the end of this chapter for a demonstration of creating and reading a random file.

READL

READL is a form of the READ statement that can be used in RANDOM'FORCED, INDEXED, or INDEXED'EXCLUSIVE modes. It locks the record that you are reading for your exclusive use. If you do not have LOKSER installed on your system, or if you are in RANDOM mode, READL works the same as READ (that is, it does not lock the record).The format is:

```
READL #file-channel,var1{,var2...,varN}
```

Once you have locked the record with READL, you use WRITE to rewrite the record to the file. WRITE will automatically unlock the record once it is finished. If you use READL without later using WRITE, you must use the UNLOKR statement to unlock the record.

UNLOKR

If your system uses LOKSER, UNLOKR allows you to unlock a record that was locked with a READL statement. The format is:

```
UNLOKR #file-channel
```

If you do not use a WRITE statement to unlock records after their use, you must use the UNLOKR statement, so that other users can access those records.

WRITE

The WRITE statement is used to write a selected logical record to an open random file. The logical record which is transferred is the one whose record number is currently in the record-number variable mentioned in the OPEN statement.

If your system is using LOKSER, WRITE unlocks the record automatically if you are sharing the file (RANDOM'FORCED or INDEXED modes). The format of the WRITE statement is:

```
WRITE #file-channel,expression-list
```

For example, if your OPEN statement is:

```
OPEN #1,"STOCK.DAT",OUTPUT,100,5
```

and your WRITE statement is:

```
WRITE #1,PART'NUMBER
```

Then the data in PART'NUMBER will be written as the 5th record of the file STOCK.DAT.

The variables in the list may be in any format, but they obviously should match that of the designated record format. The data is written into the logical record from the user variables as unformatted bytes, without regard to variable type. The data is transferred from each variable until the variable has been completely emptied. Then the next variable in the list is used, and so on.

If the record is longer than the variable list specifies, all excess data in the record will not be modified. If you try to transfer more data than is in the logical record size, you will get an error message.

The most efficient use of random files comes when the variable or variables used are mapped by a MAP statement to the exact picture of the record format in use.

WRITEL

WRITEL is a special form of the WRITE statement used with the LOKSER system that allows you to write a new record if you are in RANDOM'FORCED, INDEXED or INDEXED'EXCLUSIVE modes. Normally, you would use a combination of READL and WRITE to modify a record in the file, but if it is a new record, you cannot READL it because it does not yet exist.

Thus, to write to the record, you want to lock it for exclusive use for protection. If you do not have LOKSER installed on your system, or if you are in RANDOM mode, WRITEL functions exactly like WRITE. The format is:

```
WRITEL #file-channel,expression-list
```

SAMPLE PROGRAM

The program below gives a simple demonstration of data manipulation. Notice that you could easily write modules that would expand its functions to include deleting customer records, changing data in existing customer records, adding more customer records to a partially filled file, and so on.

Some of the file handling commands demonstrated by the program are: LOOKUP, ALLOCATE, OPEN, CLOSE, READ, and WRITE. Notice that we also use the extended TAB functions to clear the screen and position the cursor, and use the MAP statement to define all logical records and variables used in the program.

```
! SAMPLE PROGRAM TO CREATE AND ACCESS A RANDOM FILE
!
! This program shows a very simple information management system.
! Define Control record that contains info about file:

MAP1   HEADER'RECORD
       MAP2   TOTAL'RECS,F                     ! Total number of records
       MAP2   IN'USE,F                         ! Number of records in use
       MAP2   FILLER,S,116                     ! Filler bytes needed to
                                               ! pad record to 128 bytes

! Define logical record (128 decimal bytes):
MAP1   CUSTOMER'INFO
       MAP2   NAME,S,20                        ! Name and address
       MAP2   STREET,S,25
       MAP2   CITY,S,15
       MAP2   STATE,S,2
       MAP2   ID'NUM,F                         ! Customer ID number
       MAP2   CAR'INFO
              MAP3   MODEL,S,15                ! Information about car
              MAP3   YEAR,S,4
              MAP3   INSURANCE,B,1             ! Does owner have insurance?

! Miscellaneous variables used by the program:
MAP1   BLOCKS,F                          ! # of disk blocks used
MAP1   BYTES,F                           ! # of bytes used
MAP1   REC'SIZE,F,6,128                  ! # of bytes in record
MAP1   REC'NUM,F                         ! Current record number
MAP1   RESULT,F                          ! LOOKUP result variable
MAP1   QUERY,S,3,""                      ! Scratch var (init to null)
MAP1   NVAL,F                            ! Scratch var for user input

                     ! BEGIN MAIN PROGRAM

!Use LOOKUP to see if the file already exists.  If it does, go to the routine
!that will read information from the file;  otherwise, create the file.
!First, ask the user for the total number of records we can write to the
!file.  Then, see how many bytes this requires (128 * TOTAL'RECS).  We can fit
!exactly 4 records per disk block (512 = 4 * 128).  If we can't fit an even
!number of records per block, we will allocate one extra block.  When we know
!how many disk blocks to allocate, we can ALLOCATE and OPEN the file.

START:
       LOOKUP "CUSTMR.DAT",RESULT
       IF !RESULT <> 0 GOTO READ'FILE
       PRINT TAB(-1,0);                              ! Clear screen PRINT
       TAB(10,1);                                    ! Position cursor
       INPUT "Enter the total number of file records: ",TOTAL'RECS
```

```
        BYTES = TOTAL'RECS * REC'SIZE
        BLOCKS = BYTES/512
        IF BLOCKS <> INT(BYTES/512) &
              THEN BLOCKS = FIX(BLOCKS) + 1

        ALLOCATE "CUSTMR.DAT",BLOCKS
        OPEN #2,"CUSTMR.DAT",RANDOM,REC'SIZE,REC'NUM

        ! Write initial file header to Record 0.  File header is control
        ! record that tells us how many records are in file (TOTAL'RECS)
        ! and, of those, how many are in use (IN'USE).

        REC'NUM = 0 : IN'USE = 0
        WRITE #2, HEADER'RECORD
        REC'NUM = 1                       ! Ready to write to next record
                                          ! Clear screen and position cursor:

        PRINT TAB(-1,0);"Please enter the information for the";TOTAL'RECS;
        PRINT "records.  Press the RETURN key at the"
        PRINT "Customer Name to stop."; TAB(10,1)
        GOSUB GET'INFO                    ! Get info and write it to the file

              ! READ INFORMATION FROM THE EXISTING FILE:

! Open the file for input.  Get the control record to see how many records are
! in use.  Ask if user wants to read from the file.  If not, exit. Check
! to see if the existing file is empty. If so, exit.  Tell the user what
! customers we have information about.  Ask which customer the user wants
! information about. Check to be sure customer number is valid.
! Display the desired information.  If the user presses the return key, stop.

READ'FILE:
        OPEN #3,"CUSTMR.DAT",RANDOM,REC'SIZE,REC'NUM,WAIT'FILE
        REC'NUM = 0
        READ #3,HEADER'RECORD : PRINT
        INPUT "Do you want to read the file (Y or N)? ",QUERY
        QUERY = UCS(QUERY)
        IF (QUERY = "N") GOTO READ'EXIT
              ! Clear screen and position cursor:
        PRINT TAB(-1,0); "Reading the file..."; TAB(10,1)
        IF (IN'USE = 0) THEN &
              PRINT "The file is empty" : GOTO READ'EXIT
              ! Show the user what customers we have info on:
        ? "Here is a list of the customers for whom we have information:"
        FOR REC'NUM = 1 TO IN'USE
              READ #3, CUSTOMER'INFO : PRINT
              PRINT "CUSTOMER #:";ID'NUM;"   CUSTOMER NAME: "; NAME
        NEXT REC'NUM

GET'NUM:                                  ! Find out info to display
        QUERY= "" : PRINT                 ! Set initial choice to null
        PRINT "Enter the ID number of the customer whose information ";
        PRINT "you want to see.  Press"
        PRINT "the RETURN key to end the program.  Enter ALL to see ";
        PRINT "information for all of"
        INPUT "the customers: ", QUERY
        IF (QUERY="") GOTO READ'EXIT
        QUERY = UCS(QUERY)
        IF (QUERY = "ALL") THEN GOTO DISPLAY'ALL
              ! Check to see that customer number is valid.  Convert
              ! string to number for comparison:
        NQUERY = VAL(QUERY)
        IF (NQUERY < 1 OR NQUERY > IN'USE) THEN &
              PRINT : PRINT "Invalid number." : GOTO GET'NUM
              ! Read desired record (set REC'NUM to customer number).
        REC'NUM = NQUERY
        READ #3,CUSTOMER'INFO
```

```
        GOSUB DISPLAY'INFO              ! Display record
        GOTO GET'NUM                    ! Return to query

DISPLAY'ALL:                            ! Display all customer records
        FOR REC'NUM = 1 TO IN'USE
              READ #3,CUSTOMER'INFO     ! Get next record
              GOSUB DISPLAY'INFO
        NEXT REC'NUM
        GOTO GET'NUM                    ! Return to query
        READ'EXIT:                      ! Exit the program
        PRINT : PRINT "Closing the display file..."
        CLOSE #3
        END

DISPLAY'INFO:                           ! Display record info subroutine
        PRINT : PRINT "CUSTOMER #:";ID'NUM;"--"; NAME
        PRINT "Street address: "; STREET
        PRINT "City: "; CITY;”          State: "; STATE
        PRINT "Car model : "; MODEL;" Car year: "; YEAR
        PRINT " ";
        IF (INSURANCE = 0) PRINT "The car does not have insurance."&
        ELSE PRINT "The car is insured."
        RETURN

        ! Subroutine to get information and write it to the file:

GET'INFO: PRINT
              ! Make sure we're not trying to add data to a full file:
        IF (IN'USE = TOTAL'RECS) &
              THEN PRINT "The file is full..." : GOTO EXIT
        NAME = ""                               ! Clear NAME
              ! Start entering data.  Pad it to proper length with spaces:
        PRINT "Customer number”;  IN’USE + 1": " : PRINT
        INPUT "Customer name: ", NAME
        IF NAME = "" GOTO EXIT
        NAME = NAME + SPACE(20 - LEN(NAME))
        INPUT "Street address: ", STREET
        STREET = STREET + SPACE(25 - LEN(STREET))
        INPUT "City: ", CITY
        CITY = CITY + SPACE(15 - LEN(CITY))
        INPUT "State: ", STATE
        STATE = UCS(STATE + SPACE(2 - LEN(STATE)))
        INPUT "Car model: ", MODEL
        MODEL = MODEL + SPACE(15 - LEN(MODEL))
        INPUT "Car year: ", YEAR
        YEAR = YEAR + SPACE(4 - LEN(YEAR))
        INPUT "Car insurance? (Y or N): ", QUERY
        QUERY = UCS(QUERY)
        IF (QUERY = "Y") INSURANCE = 1 &
              ELSE INSURANCE = 0
        ID'NUM = REC'NUM                        ! Customer number = record #
        ! Write whole record; increment records-in-use counter and add 1 to
        ! REC'NUM so we are ready to write to the next record:
        WRITE #2,CUSTOMER'INFO
        IN'USE = IN'USE + 1
        REC'NUM = REC'NUM + 1
        PRINT "The Customer ID Number is: "; ID’NUM
        GOTO GET'INFO

EXIT:                                           ! Stop entering data
        PRINT "Now closing the output file."
        REC'NUM = 0
        WRITE #2, HEADER'RECORD
        PRINT "The total number of records in the file: "; TOTAL'RECS
        PRINT "Number of records in use: "; IN'USE
        CLOSE #2
        RETURN
```

Chapter 17 - Chaining to AlphaBASIC and System Programs

The CHAIN statement stops the run of the current program and causes another program or system function to run. The new program to be executed must be named in the CHAIN statement itself; that name may be a full file specification.

The file named in the statement may be another AlphaBASIC program (compiled only), or it may be a system command or command file. This allows your program to execute a command file and invoke system commands as well as execute other AlphaBASIC programs.

CHAINING TO ANOTHER AlphaBASIC PROGRAM

CHAIN assumes a default extension of .RUN. If you specify another extension, AlphaBASIC will assume that you have specified a command file or system command. Therefore, if you wish to use CHAIN to run another AlphaBASIC program, the file you specify must be compiled, and it must have a .RUN extension.

If you do not specify a device and account, AlphaBASIC follows the search pattern outlined in Chapter 2 when looking for .RUN files. If you do specify a device and account, AlphaBASIC looks in the specified area.

All variables in the new program are first cleared to zero prior to the run. Also, all variables in the current program are set to zero (or null, if strings). Some examples of legal CHAIN statements are:

```
CHAIN "PAYROL"
CHAIN "PAYROL.RUN"
CHAIN "DSK1:PAYROL[101,13]"
```

Due to the fact that programs are compiled and not interpreted, there is no way to execute a program at any entry point other than its physical beginning. There is also no internal method for passing parameters between programs, but you can accomplish this function for yourself by using the AlphaBASIC assembly language subroutine COMMON to store data in a common memory area.

COMMON allows you to store information either in system memory (where programs run by all users on the system can access the information) or in your memory partition (where only programs run by you can access the information). For details on using COMMON, see the *AlphaBASIC XCALL Subroutine User's Manual* that is appropriate for your system.

In addition to sharing information, you can use the common area to pass parameters to the chained program. For example, the current program can use the COMMON to pass a parameter to the new

program. The new program uses the parameter in an ON-GOTO statement to begin execution at a location based on the value passed in the parameter.

Another way to make sure that chained programs can share information is the use of disk files. The current AlphaBASIC program can open a data file, write the variables it wants to share into that file, and then close the file. When the new file is chained in, it can open the file and read the necessary information.

CHAINING TO SYSTEM FUNCTIONS

It is sometimes desirable to transfer execution to a system function or a command file from an AlphaBASIC program. If the extension of the file in the CHAIN statement is not .RUN, AlphaBASIC assumes that the file is a system command program or system command file (a .LIT, .DO or .CMD file). In this case, the AlphaBASIC run-time package creates a dummy command file at the top of the current user partition and transfers control to the monitor command processor.

The monitor then interprets this dummy command file as a direct command and executes it. Note that the dummy command file created by the run-time package is merely the one line name specified in the CHAIN statement. It is not the command file itself, which is the target function desired. You can also include parameters for that program or command file. Some valid examples are:

```
CHAIN "SYSTAT.LIT[1,4]"
CHAIN "TEST1.CMD"
CHAIN "DSK0:BACKUP.CMD[2,2]"
CHAIN "SEE.DO[110,0] MEMO.TXT"
CHAIN "LOG.LIT[1,4] DSK9:[200,3]
CHAIN "DIR.LIT[1,4]/W"
```

Note that if the device and account are not specified, the action taken is the same as if you had entered the command directly from your keyboard. That is, if you omit device and account specifications, the monitor command processor searches for command files or programs in the following order:

1. System memory
2. User memory
3. The account and device you are logged into.

To load a file into your user memory partition, use the monitor level LOAD command. To load the file into system memory (where it may be accessed by all users on the system), the System Operator must add the appropriate SYSTEM command line to the system initialization command file.

Note also that when you chain to a monitor command, after the command has finished executing, it returns you to monitor level, rather than to your program. This means that if you wish to automatically return to some AlphaBASIC program, you have to execute a command file whose final command is a RUN command which specifies that original AlphaBASIC program.

Chapter 18 - Error Trapping

This chapter discusses:

- What is error trapping?
- The ON ERROR GOTO statement
- The ERR(X) function
- The error codes returned by ERR(X)
- The RESUME statement
- Control-C trapping

In addition, we include two sample programs that illustrate how to use error trapping.

WHAT IS ERROR TRAPPING?

Normally, when an error occurs during the run of an AlphaBASIC program, an error message is printed, and the program run is halted. Using a technique called "error trapping," you can catch the error before the AlphaBASIC processor stops your program, giving you a chance to correct the error. This has the advantage of allowing your program to continue running, and gives you more control over what is happening in your program. Many of the errors you encounter can be handled this way.

THE ON ERROR GOTO STATEMENT

Error trapping is enabled and disabled by using the ON ERROR GOTO statement in one of two forms. The first form specifies a line number (or label) within the program. When the program encounters this ON ERROR statement, it stores the line number and turns error trapping on.

If an error occurs any time after this, AlphaBASIC transfers control to the line number or label you specified. Here is what such a statement looks like:

```
ON ERROR GOTO 500
ON ERROR GOTO TRAP'ROUTINE
```

The error routine must then take appropriate action based on the type of error.

The second form of the statement turns off error trapping by giving a line number of zero or leaving the line number off completely:

```
ON ERROR GOTO 0
ON ERROR GOTO
```

After executing a statement like the ones above, error trapping is off, and errors will be handled in the normal way. It is recommended that all error trapping routines execute the ON ERROR GOTO 0 statement for all errors which have no special recovery processing.

If an error occurs within the error trapping routine itself, that error is processed and the error message:

```
?Error in error trapping
```

is printed. There is no method to trap errors within an error recovery routine.

ERR(X) FUNCTION

The ERR(X) function returns the following data based on conditions at the time of the error:

ERR(0)	The number of the error that was detected
ERR(1)	The number of the last line encountered prior to the error
ERR(2)	The number of the last file accessed (only relevant for file errors)

Using any or all of these functions will give you the information you need to decide what your error recovery routine is going to do.

Error Codes Returned By ERR(X)

Code	Meaning	Code	Meaning
1	Control-C interrupt	2	System error
3	Out of memory	4	Out of data
5	NEXT without FOR	6	RETURN without GOSUB
7	RESUME without ERROR	8	Subscript out of range
9	Floating point overflow	10	Divide by zero
11	Illegal function value	12	XCALL subroutine not found
13	File already open	14	IO to unopened file
15	Record size overflow	16	File specification error
17	File not found	18	Device not ready
19	Device full	20	Device error
21	Device in use	22	Illegal user code
23	Protection violation	24	Write protected
25	File type mismatch	26	Device does not exist
27	Bitmap kaput	28	Disk not mounted
29	File already exists	30	Redimensioned array
31	Illegal record number	32	Invalid filename
33	Stack overflow	34	Invalid syntax code
35	Unsupported Function	36	Invalid subroutine version
37	File in use	38	Record in use
39	Deadly embrace	40	File cannot be deleted
41	File cannot be renamed	42	Record not locked
43	Multiple link translation	44	LOKSER queue is full
45	Device not file structured	46	Illegal ISAM sequence
47	#0 illegal for random files	48	File not open
49	File already open	50	Remote not responding
51	First logical unit not mounted		

THE RESUME STATEMENT

The RESUME statement is used to resume execution of the program after the error recovery procedure has been performed. It also turns Control-C detection back on (a Control-C interrupt is not allowed while AlphaBASIC processes the error trapping routine).

The RESUME statement takes on two forms similar to the forms of the ON ERROR GOTO statement.

The first form specifies a line number (or label) within the program where the execution is to be resumed:

```
RESUME 410
RESUME TRY'AGAIN
```

The second form specifies a line number of zero, or no line number at all, and causes the execution to be resumed at the statement which caused the error to occur:

```
RESUME 0
RESUME
```

Both forms cause the error condition to be cleared and turns on error trapping again.

You should never use the GOTO statement to exit from an error trapping routine. Use RESUME. This is because RESUME clears the area of memory used by the error routine. GOTO does not, which could cause memory errors.

CONTROL-C TRAPPING

When you use Control-C during the run of an AlphaBASIC program, the program stops at the next statement. What happens next depends on whether error trapping is on or off.

If error trapping is off, the program stops and the appropriate message is printed on the terminal.

If error trapping is on, the error trapping routine is entered with the code in ERR(0) being set to 1. You can then decide what your program is going to do when a Control-C is encountered. If you simply let the program RESUME, you have effectively turned off Control-C interrupting (which may not be a good idea—if your program gets into an endless loop you couldn't stop the run, except by resetting your system).

This Control-C trapping feature allows you to prevent users of your program from exiting the program during a critical time, such as during a file update. The Control-C is detected immediately.

During an error recovery routine, no Control-C use is allowed, so that the program cannot be stopped during this important phase.

A SAMPLE ERROR RECOVERY ROUTINE

In this first sample of an error recovery routine, we'll take a case where the user of your program is asked to input two numbers. The program then divides the first number by the second number. But, what if

the user enters a zero as the second number? Normally, BASIC would halt the run at that point and print a `Divide by zero` error message.

By using an ON ERROR GOTO statement and an error recovery routine, the program can handle the error itself:

```
ON ERROR GOTO DIVIDE'BY'ZERO
INPUT "Enter two numbers: ",A,B
PRINT "A divided by B equals: "; A/B
END

DIVIDE'BY'ZERO:
   ! If error is not "divide by zero" exit the program:
   IF (ERR(0) <> 10) THEN &
      PRINT "System Error Detected.  Exiting..." : END
   PRINT
   PRINT "Division by zero does not work!"
   INPUT "Please re-enter the second number: ",B
   RESUME                  ! Go back to the line where the
                           ! problem occurred.
```

Two sample runs of the program look like this:

```
Enter two numbers: 2,3 [RETURN]
A divided by B equals: .666667

Enter two numbers: 25,0 [RETURN]
A divided by B equals:
Division by zero does not work!
Please re-enter the second number: 5 [RETURN]
A divided by B equals: 5
```

Note that a successful error trapping routine must either resolve the error or exit the program.

For example, if the program above had merely printed an error message and then RESUMEd back to the line where the error occurred, the "divide by zero" error would still exist, AlphaBASIC would again transfer control to the error trapping routine, and we would be in an eternal loop.

Instead, the program resolves the error by having the user change the value of the second variable, then resuming program execution.

Another Sample Routine

The following program shows an error trapping routine that handles Control-C. Notice that we turn on the error trapping routine CATCH'CTRLC: just before the user enters input.

Directly afterward, we turn off our routine and return to the regular AlphaBASIC error trapping by using the ON ERROR GOTO 0 statement. This way, we catch any errors (besides Control-C) that might occur in the rest of the program.

```
!                ERROR TRAPPING SAMPLE PROGRAM
! Define error code for Control-C, and various string variables.

MAP1 CONTROL'C,F,,1
MAP1 SCRATCH,S,16," "
MAP1 ANSWER,S,16," "
MAP1 QUERY,S,1

!                     Begin Main Program
START:
   ? "This program converts positive decimal numbers to binary."

GET'NUMBER:                       ! Ask the user for a number.
   ! Turn on error trap to catch Control-C on input:
   ON ERROR GOTO CATCH'CTRLC
   INPUT "Enter a number between 1 and 65535: ",NUMBER

   ! If the user used Control-C, we already caught it, so turn
   ! off error trapping and turn regular BASIC error trapping
   ! back on in case other error occurs:
   ON ERROR GOTO 0
   IF NUMBER < 0 GOTO      GET'NUMBER
   CURRENT = NUMBER
CALCULATE:                            ! Now figure the answer.
   IF (CURRENT/2 = FIX(CURRENT/2)) THEN &
      SCRATCH = SCRATCH + "0" &
         ELSE SCRATCH = SCRATCH + "1"
   IF FIX((CURRENT/2) = 0) THEN &     ! If done, go to display
      GOTO DISPLAY
   CURRENT = FIX(CURRENT/2            !Get rid of the remainder.
   GOTO CALCULATE
   DISPLAY:                           !Display routine. Reverses
                                      ! string so the answer
   FOR I = 1 TO LEN(SCRATCH)          ! is in the proper order.
      ANSWER = ANSWER + SCRATCH[-I;1]
   NEXT I
   PRINT "The number ";NUMBER;" is ";ANSWER;" in binary."
   SCRATCH = " " : ANSWER = " " ! Initialize answer to null.
   GOTO GET'NUMBER

CATCH'CTRLC:                      ! Error trap.  Looks for
   IF (ERR(0) <> CONTROL'C) &     ! Control-C.  Gives the user
      THEN RESUME                 ! a chance to quit or resume.
   INPUT "Do you wish to quit? (Y or N): ",QUERY
   QUERY = UCS(QUERY)
   IF (QUERY = "N") THEN RESUME GO'AHEAD
   PRINT : PRINT "So long..." : PRINT
   END
GO'AHEAD:                             ! User wants to resume
   PRINT : PRINT "Resuming..." : PRINT
   GOTO GET'NUMBER
```

A sample run of the program above looks like this:

```
RUN CNVRT [RETURN]

This program converts positive decimal numbers to binary.
Enter a number between 1 and 65535: 24 [RETURN]
The decimal number 24 is 11000 in binary.
Enter a number between 1 and 65535: [CTRL] /C
Do you wish to quit? (Y or N): Y [RETURN]

So long... .
```

Chapter 19 - External Assembly Language Subroutines

This chapter discusses:

- Why use external subroutines?
- Automatic subroutine loading

AlphaBASIC supports the use of external assembly language subroutine programs that can be called from your AlphaBASIC programs. We discuss the details of writing these subroutines in Appendix E. For information about assembly language, see the *Assembly Language Programmer's Manual* and the *Monitor Calls Manual* appropriate to your system.

WHY USE ASSEMBLY LANGUAGE SUBROUTINES?

There are several good reasons why you might want to use an assembly language program to carry out a function rather than using another AlphaBASIC program or subroutine.

Assembly language programs are generally much smaller and faster than equivalent BASIC programs—when speed and size are important factors, you may want to code parts of your program into assembly language. Yet another reason for using assembly language programs is simply that some tasks are too awkward (or even impossible) to do from within a higher level language.

Assembly language programs are uniquely suitable for applications that require that you work more closely with the hardware or operating system than is convenient or possible in BASIC.

Although you may want to write your own assembly language subroutines (see Appendix E for further information if so), note that we do provide a set of existing assembly language subroutines in the AlphaBASIC Library Account, DSK0:[7,6]. For information on these subroutines, see the *AlphaBASIC XCALL Subroutine User's Manual* appropriate to your system.

In addition, a set of business-oriented assembly language subroutines is available from your dealer.

To call an assembly language subroutine from an AlphaBASIC program, use the XCALL statement. The syntax for this statement is:

```
XCALL routine{argument1{,argument2,...argumentN}}
```

The routine to be called is an assembly language program which has been assembled using the machine language assembler.

When the XCALL statement is executed by the AlphaBASIC run-time system, the named subroutine is located in memory and then called.

AUTOMATIC SUBROUTINE LOADING

When an AlphaBASIC program calls a subroutine by using an XCALL statement, AlphaBASIC attempts to locate the subroutine in user or system memory. If it cannot, it attempts to load the subroutine from the disk, following the search pattern outlined in Chapter 2.

If an AlphaBASIC program fetches a subroutine from the disk, AlphaBASIC loads the subroutine into memory only for the duration of its execution. Once the subroutine has completed its execution, it is removed from memory.

Therefore, if a subroutine is to be called a large number of times, it is wise to load it into memory (using the monitor LOAD command) to avoid the overhead of fetching the subroutine from disk.

Subroutines loaded into memory by use of the monitor LOAD command remain in memory until you reset the system or until you use the monitor command DEL to delete them.

Chapter 20 - Using ISAM Files

This chapter discusses:

- What ISAM is
- The ISAM file structure
- Opening indexed files
- File locking for ISAM
- Closing indexed files
- Error processing

WHAT IS ISAM?

The ISAM program is a tool for organizing and retrieving data. The name stands for:

Indexed
Sequential
Access
Method

and refers to the manner in which the data is organized.

Even though the word "sequential" is part of the ISAM name, ISAM files are RANDOM files. It is the access method which is sequential, not the file.

AlphaBASIC has the ability to process indexed sequential files by linking to the ISAM assembly language package (which must reside either in system memory or in individual user memory). ISAM supports multiple index files by using some elementary ISAM statements that allow the direct control of index file and data file items.

AMOS/L versions 1 .1 and later support the LOKSER file locking system. Although use of the LOKSER system is optional, we strongly recommend that you use the new AlphaBASIC command statements designed for the LOKSER system—even if you do not use LOKSER. Use of these will help to clarify your code, and will enhance compatibility with other systems and future Alpha Micro software.

It is important when reading the following sections that you be familiar with opening and using random data files. If you are not, refer first to Chapter 16. It is also a good idea to be familiar with error trapping, explained in Chapter 18, because error trapping is a necessity for ISAM programs.

This chapter also assumes that you are familiar with the Alpha Micro ISAM system, which is fully explained in the *ISAM System User's Guide* for your system. This chapter is intended as a quick reference guide to the ISAM commands used within AlphaBASIC.

ISAM FILE STRUCTURE

ISAM files are more complex in structure than sequential or normal random files. This allows faster and more precise access to large data files.

Both the INDEXED and INDEXED'EXCLUSIVE modes of ISAM require that the ISAM program be able to write to the disk containing the index files - even if you do not plan to do anything more than read from the disk. Therefore, make sure that the disk containing the index files is not write-protected.

All indexed sequential files must be created by the ISMBLD program prior to access by any AlphaBASIC program. There is no method for the creation of a new indexed file within the AlphaBASIC language since this would require a prohibitive amount of seldom used code.

You may, however, create indexed files by using the feature that allows an AlphaBASIC program to create and then execute a command file. This command file could set up parameters and then call the ISMBLD program to perform the actual creation of the files.

For compatibility with existing structures, the data file must have an extension of .IDA and all index files must have an extension of .IDX. There must be at least one index file which is called the primary index file.

There may also be additional index files called secondary index files which also link to the primary data file. The primary index file must always be opened in any program in order to gain access to the data file.

This is true even if you only intend to access the data file through one of the secondary index files in the current program. For information on file structures and operating the ISMBLD program, refer to the *ISAM System User's Guide* that is appropriate for your system.

SYMBOLIC AND RELATIVE KEYS

Indexed files are accessed by one of two specific types of keys—relative keys and symbolic keys.

Relative keys are probably already familiar to you since they are the same type of key used to access normal random files. The relative key is the record-number variable you specified when you opened the file. It contains the number of the logical record to be accessed. When used with an indexed file, a relative key is used only to access a specific record in a data file.

Symbolic keys are used only with indexed files. Symbolic keys are ASCII strings of variable lengths and are used to access the index file (primary or secondary). They are specified in the ISAM statements when accessing the index file, and are used to retrieve the relative key of the associated data record in the data file.

The concept of symbolic versus relative keys and their different uses is an important one, and misuse of them can cause the ISAM system to malfunction in a number of ways.

Symbolic keys are used with the ISAM statement; relative keys are used with the OPEN statement so that READ, READL, WRITE and WRITEL statements can be successfully performed. In most instances, you will not be working with the relative key—it will be merely a device automatically set up and referenced by the above calls.

THE ISAM STATEMENT

You access indexed files by a special statement in AlphaBASIC called the ISAM statement. This statement has the general form:

```
ISAM #file-channel,code,symbolic-key
```

All ISAM statements follow the above format, using a different numeric value in "code" to indicate the specific function to be performed by the ISAM package. All ISAM statements directly translate into a specific type of call to the assembly language ISAM program.

A symbolic key must always be specified, even for those functions which do not require the use of one (this simplifies syntax checking and execution coding). You may use a dummy string variable if you desire. The following codes are used by the ISAM statement:

1 Finds a record in the data file by symbolic key (i.e., returns the relative record number in the variable specified by the OPEN statement that opens the data file/primary index file).

2 Finds the next data record (by the order in which the symbolic keys appears in the index file). Returns the relative record number in the variable specified by the OPEN statement.

3 Adds a symbolic key to an index file.

4 Deletes a symbolic key from an index file.

5 Locates the next free data record in the data file (returning the relative record number in the variable specified in the OPEN statement).

6 Deletes a record from a data file, and returns that record to the free list.

7 Performs code 2 (explained above) and returns the symbolic key.

An error results if an ISAM statement is executed with the value of "code" not equal to one of the above numbers. The "code" may be any legal numeric expression which is resolved at run-time.

OPENING AN INDEXED FILE

As with other types of files, an indexed file must be opened with a specific file-channel number prior to any references to the file by other statements. The OPEN statement follows the same format as that used by the normal random files except that you specify INDEXED or INDEXED'EXCLUSIVE for the mode:

```
OPEN #file-channel,filespec,mode,record-size,
     relative-record-number{,WAIT'FILE}{,WAIT'RECORD}
```

#file-channel Any numeric expression that evaluates to an integer from 0-65535 (0 is defined as the user terminal).

filespec	Any string expression that evaluates to a legal Alpha Micro file specification (optionally including account and device specifications). Specifies the data file/primary index file or the secondary index file. (The primary index file always has the same name as the data file, but has the .IDX extension; the data file has the .IDA extension.)
INDEXED	Specifies indexed sequential mode. If you use LOKSER, the file is opened for shared use.
INDEXED'EXCLUSIVE	Specifies indexed exclusive mode. (See below for more information). Opens the file for exclusive access, even if LOKSER is not installed on the system.
record-size	Expression that specifies the logical record size for the data file.
relative-record-number	Floating point variable that contains the record number of the logical record you want to access.
{,WAIT'FILE}	If you have LOKSER installed on your system, you may specify WAIT'FILE. This causes your program to wait if the file you want to OPEN is in use.
{,WAIT'RECORD}	If you have LOKSER installed on your system, you may specify WAIT'RECORD. This causes your program to wait if the record you want to access is in use.

The filespec must refer to the name given to the index file when the file was created using ISMBLD. If this is a call to open a secondary index file, you must have previously opened the corresponding primary index file on another file number so that the data file may be accessed.

As an example, assume that an indexed file structure consists of the primary index and data files named MASTER.IDX and MASTER.IDA respectively. The structure also has secondary index files named ADRESS.IDX and PAYROL.IDX which access the MASTER.IDA file in different sequences. If you desire to process the file structure by using the sequence used by the ADRESS.IDX index file, the following two statements are required:

```
OPEN #1, "MASTER", INDEXED, RECSIZ, RELKEY
OPEN #2, "ADRESS", INDEXED, RECSIZ, RELKEY
```

The first statement opens both the data file and the primary index file. Remember that there are now three files opened:

1. The data file, MASTER.IDA
2. The primary index file, MASTER.IDX
3. The secondary index file, ADRESS.IDX.

When you are processing using secondary indexes, the LOKSER system has no way of knowing which primary index record is involved. You must use the LOCK statement to lock the proper primary index file to prevent record locking problems. See the *LOKSER User's Manual* appropriate to your system for more information.

Notice that the record size expression (RECSIZ) and the relative record number (RELKEY) are identical in both statements. This is important since they both refer to the same data file (MASTER.IDA).

ISAM statements may then be made referring to either index file (#1 or #2) but all READ and WRITE statements must be made to the data file (#1) which is associated with the primary index file. In other words, READ, READL, WRITE and WRITEL statements must not be made to file #2.

FILE LOCKING FOR ISAM FILES

When your program opens an indexed file in INDEXED mode, you must use file-locking procedures to protect your indexed file, since INDEXED mode opens files for shared access.

Because of their interconnected file structure, ISAM files are especially vulnerable to simultaneous update problems. The whole structure of your files could be destroyed if you do not use a file locking system. If you do not have LOKSER on your system, you will have to use the FLOCK and XLOCK programs.

Make sure you use the READL, WRITEL, and/or LOCK statements where appropriate to insure file security. For information on the file-locking system, LOKSER, and the FLOCK and XLOCK programs, see the *LOKSER User's Manual* appropriate to your system.

READ AND WRITE

The ISAM calls do not access the data records themselves but merely deliver back the relative key of the associated data record to be used. Normal READ and WRITE statements are then used to actually retrieve or write into the data record itself.

These READ and WRITE statements follow the same format used when accessing a normal random access data file in AlphaBASIC. The relative key associated with the primary file (as specified in the OPEN statement) must contain a valid relative key for the operation or an error results.

As mentioned before, the READ and WRITE statements must only be made using the primary index file-channel number. For example:

```
OPEN #3,"PAYROL",INDEXED,67,NUM'REC        ! Open the file
ISAM #3,1,NAME                             ! Get the record
READ #3,LABEL                              ! Read the record
```

READL AND WRITEL

READL locks the current record until you use the UNLOKR statement (see below) or until you use a WRITE statement to output the record.

WRITEL locks the current record until it finishes outputting the record, when it unlocks it.

If you are in INDEXED'EXCLUSIVE mode (or in either mode if LOKSER is not installed), READL and WRITEL work exactly like READ and WRITE. However, even if your system does not use LOKSER, we strongly recommend that you use READL and WRITEL, since this will promote program standardization and will make it simple for you to transfer your programs to a LOKSER system at a later date.

LOCK AND UNLOKR

This section discusses the various options open to you if you want to explicitly lock and unlock files and records.You can explicitly lock the primary index file by using the LOCK statement:

```
LOCK #File-channel
```

If you locate a record using the primary index file, LOKSER automatically locks the primary index file for you. However, if you locate a record using the secondary index file, you must use the LOCK statement to explicitly lock the primary index file, whether or not the WAIT'FILE clause appears in your OPEN statement—LOKSER will wait until the primary index file is free if it is already locked. The statement:

```
UNLOKR #File-channel
```

unlocks the primary index file and any record currently locked by using a READL statement.

CLOSING AN INDEXED FILE

To ensure that all data records have been rewritten to the data file and that all links in the index file have been properly updated and rewritten to the disk, all index files (primary and secondary) must be closed using the normal CLOSE statement, referencing the correct file-channel number. Failure to do so may result in destroying the link structure.

It makes no difference in which order you close the ISAM files—however, remember that you cannot access a secondary index file if you have already closed the primary index.

INDEXED'EXCLUSIVE MODE

When your program is the only program that needs to access an ISAM indexed file, you can specify INDEXED'EXCLUSIVE as the mode in which you open the file. It locks the entire file whether or not LOKSER is installed on your system. For example:

```
OPEN #5,"PAYROL",INDEXED'EXCLUSIVE,100,REC'NUM,WAIT'FILE
```

If LOKSER is installed on your system, ISAM still requires that you use READL and WRITEL statements where appropriate, even though you have exclusive use of the file. This is to ensure program standardization and proper coding technique.

The statement above opens the data file PAYROL.IDA and the primary index file PAYROL.IDX in exclusive mode. The main advantage of INDEXED'EXCLUSIVE mode is a large increase in the speed with which your programs can access the indexed file. It also prevents other users from accessing your indexed file until you close the file. Otherwise, it works in the same way as in INDEXED mode.

In INDEXED'EXCLUSIVE mode, ISAM knows that no other program is going to access the indexed file while your program is working with it. Therefore, ISAM can take full advantage of prior knowledge about the file for every access and can speed up your access time considerably.

When your program opens an indexed file in INDEXED'EXCLUSIVE mode, ISAM will not allow other user to access the specified indexed file; if they try to do so, they see a "file in use" message.

This means that you only have to worry about file-locking at the moment in which you are opening the indexed file. If your program tries to open a file that is being used, you will get the "file in use" message, unless you specified WAIT'FILE in your OPEN statement, in which case, your program will wait until the file becomes available.

The advantage of an indexed file opened in INDEXED'EXCLUSIVE mode is that NO other user can access the file while you are using it. If you need to have several programs access the file, use the INDEXED mode (Of course, if you use INDEXED mode, you will have to perform file locking to prevent simultaneous update problems).

One feature of the INDEXED'EXCLUSIVE mode is that it temporarily renames the .IDX file to an .IDY extension to prevent ISAM from letting other programs access the file. If something should go wrong (such as a system crash), ISAM may not be able to rename the file to its original .IDX extension, and you will have to do so yourself.

For more information on INDEXED'EXCLUSIVE mode, see the *ISAM System User's Guide* that is appropriate for your system.

ERROR PROCESSING

Any ISAM operation can result in some kind of error. If the error is a system error (for example, the disk is not mounted), AlphaBASIC interrupts your program and aborts to the monitor (or, if error trapping is enabled, AlphaBASIC transfers control to your error handling routine).

To check for hard system errors, use the ERR(X) function. Remember that ERR(0) returns the code of the hard error, and ERR(1) returns the number of the program line at which the error occurred.

Special ISAM errors can also occur as a result of an ISAM operation. These errors do not generate an error message or result in an error trap. It is therefore very important that your program check for these errors after every ISAM statement; otherwise, you have no way of knowing whether or not the ISAM function was performed successfully.

To do so, use the ERF(X) function, where X is the file channel number used by the preceding ISAM statement. The ERF(X) function operates in much the same way as the EOF(X) function.

If ERF(X) returns a zero, the preceding ISAM statement was successful. If ERF(X) returns a nonzero value, then an error was detected. If an error occurred, your program should correct the problem before going on to access the file. The nonzero value returned tells you which error occurred:

```
! If a "Record not found" error (#33) occurs,
! then go to the routine that asks for a new key.

IF ERF(2) = 33 THEN &
PRINT "RECORD NOT FOUND" : GOTO PROMPT
```

The current ISAM error codes are:

32 Illegal ISAM statement code.
33 The record was not found during the index file search.
34 A duplicate key found in the index file during the attempted key addition.
35 The link structure is smashed and must be re-created.
36 The index file is full.
37 The data file is full (the free list is empty).
38 The end of the file was reached while reading the key.
39 Illegal ISAM sequence.

The last message (39) occurs if LOKSER is installed on your system and you do not perform the ISAM file access procedures in the order discussed in the *ISAM System User's Guide* that is appropriate for your system.

Always check after performing an ISAM function to see if an error occurred. If you do detect an error, your program must fix it before continuing.

Appendix A - AlphaBASIC Messages

Below is a complete list of all messages output by the AlphaBASIC system (BASIC, RUN, and COMPIL), and a brief explanation of each message.

#####

Your program is using a PRINT USING statement, and it could not process the data given to it. Check the syntax of the line against the rules for PRINT USING in Chapter 12.

?Bitmap kaput

Your program attempted a file operation (OPEN, ALLOCATE, etc.) on a device with a bad bitmap.

Break at line n

The program reached the breakpoint that was set at line n.

Can't continue

You have attempted to continue a program which is not stopped at a breakpoint, or which has reached a point where it can go no further (for example, it has reached an END statement).

?Cannot find xxxxxxx

The program xxxxxxx was not found. Check the spelling of the program you specified, and/or check the directory to see if the file exists. Make sure the program specified is in the proper account.

?Channel #0 is illegal for random files

You may not specify file channel #0 when accessing a random file. Change the file channel number to a positive integer, or use a sequential file (channel #0 equals your terminal if you are using sequential files).

Copying from xxxxxx.xxx

The program you are compiling contains a ++INCLUDE command. This message is displayed as the file specified in the ++INCLUDE command is copied into your program.

?Deadly embrace possible

Your program has opened files out of sequence with their order on the LOKSER data base. This message only appears if the deadlock check indicators are set in the LOKSER data base.

DELETE what?

You have specified a DELETE command without specifying what line(s) are to be deleted.

?Device does not exist

The device you specified in a file operation (OPEN, LOOKUP, etc.) does not exist. Make sure your entry was typed correctly.

?Device driver must be loaded into user or system memory

If you are accessing a non-DSK device, the appropriate device driver must be loaded into user or system memory.

?Device error

An error has occurred on the referenced device. Check the documentation for that device to determine how to correct the error before rerunning the AlphaBASIC program.

?Device full

The specified device has run out of room during a WRITE, CLOSE, or ALLOCATE operation. Remember that an ALLOCATE requires contiguous disk space, so that this error may occur when there are still a number of non-contiguous blocks available.

?Device in use

The specified device is currently assigned to another user. You will have to wait until that user is finished before rerunning your program.

?Device is not file structured

The device you were trying to access is not an accessible device. Check your specification for spelling, or talk to your System Operator to find out what devices are accessible on your system.

?Device not ready

The specified disk is not ready for use. Check your device to see if the appropriate buttons are pushed, plugs are in, etc.

?Disk not mounted

The specified disk has not been mounted. Mount it by using the MOUNT monitor command or by using the XMOUNT subroutine.

?Divide by zero

Your program attempted to perform a division by zero. Check the values that went into the equation. You may want to install a check for zero values, or use an error trapping routine to catch zero divisions.

?Duplicate label

Your program has defined the same label name more than once. Search the .BAS file for the duplicate labels, and change one of them to a unique name.

***** End of Program *****

You have reached the end of the program during single-stepping in Interactive Mode. If you press the line feed key again, your program will restart at the beginning.

Enter <CR> to continue:

You have reached a STOP statement in your program. You may continue from the STOP statement by pressing the RETURN key, or you may abort the run by using Control-C.

?Error in Error Trapping

An error occurred while you were in the error trapping routine. Check your routine for incorrect statements or miss-spellings.

?File already exists

Your program tried to create a file which already exists. You may not need the section of code that creates the file, or you may need to install a check using LOOKUP to determine if the file exists before you try to create it.

?File already open

You have attempted to open a file that is already open on the same file number. You may not need that OPEN statement, or you may need to check (using LOOKUP) to see if the file is already open.

?File cannot be DELETEd

Your program tried to delete a file that has LOKSER DELETE protection. If you really do want to delete the file, you will have to change its status under LOKSER first.

?File cannot be RENAMEd

Your program tried to rename a file that has LOKSER RENAME protection. If you really do want to rename the file, you will have to change its status under LOKSER first.

?File in use

The file your program wants to access is already locked by another user. This only occurs if LOKSER is in use. You may want to use the WAIT'FILE option with the file access statement.

?File not found

BASIC was unable to locate the specified file. Check your specification, and your directory. Make sure the account is correct.

?File not open

Your program attempted to access a file that was not open. Have your program open the file before access is attempted.

?File spec error in <program name>

The file specification you gave in a file operation (OPEN, LOOKUP, etc.) is in error. All file specifications must conform to the system standard (Devn:filename.ext [p,pn]).

?File type mismatch

Your program tried to perform a sequential operation on a random file or vice-versa. Make sure you are using the correct files and the correct file statements.

?First logical unit not mounted

The first logical unit on the drive your program was trying to access was not mounted. Mount the unit and try your program again.

?Floating point overflow

A floating point overflow occurred during a calculation. This means that the calculation went beyond the capabilities of the computer in some way (usually the number was too large). Check your calculation.

?Illegal expression

The specified expression is not valid. Check your syntax and spelling.

?Illegal function value

The specified value is not valid for the particular function. See the instructions for that function to see what type and number of arguments it takes.

?Illegal GOTO or GOSUB

The format of the GOTO or GOSUB statement is invalid. Check the label or line number that it refers to.

?Illegal ISAM sequence

The sequence of commands used for the ISAM file were executed in the wrong order for the LOKSER system.

?Illegal line number- xxx

The specified line number is invalid (for example, not between 1 and 65534).

?Illegal NEXT variable

The variable in the NEXT statement is invalid—it does not equal the number in any previous FOR statement. Check your variable.

?Illegal or undefined variable in overlay

The variable specified in a MAP statement overlay (preceded by @) has not been previously defined, or is not a mapped variable. Check your spelling. You may want to define a MAP statement for the variable, or remove the @.

?Illegal PRINT USING format

The edit format used in a PRINT USING statement is invalid.

?Illegal PROGRAM argument

The argument in the PROGRAM statement is invalid. Make sure that you entered the argument correctly, and the numbers are within the parameters allowed by the PROGRAM statement.

?Illegal record number

The relative record number specified in a random file processing statement (READ or WRITE) is either less than the current FILEBASE or outside of the file.

?Illegal SCALE argument

The argument given in a SCALE statement is invalid. The argument must range between -30 and +30. Check your argument.

?Illegal size for variable type

The specified variable size is not valid for the particular variable type. Floating point variables must be size 6, and binary variables must have sizes 1 through 5.

?Illegal STRSIZ argument

The argument given in a STRSIZ statement is invalid. Check your syntax and the number you specified.

?Illegal subroutine name

The name specified as a subroutine is not valid. Names used must be a legal filename with a .SBR extension.

?Illegal subscript

The subscript expression is not valid. Subscripts must be positive, whole numbers.

?Illegal TAB format

Your program has incorrectly specified a TAB function. Check the syntax of the statement, and check to see that there are not more than two numbers specified within the parenthesis.

?Illegal type code

The variable type code used in a MAP statement is not valid. Check your number against the legal values allowed.

?Illegal user code

The specified PPN was not found on the specified device, or is not in a valid format. If you get this message while using LOKSER, it may mean that your program tried to access or lock a resource in a protected account.

?Insufficient memory to initialize RUN

Your memory is too full to run your program. You can see your System Operator about allocating more memory to your partition, or examine your program to cut down memory use. See Appendix C for tips on making your programs more memory efficient.

?Insufficient memory to load program xxxxxxx

The RUN program did not find enough free memory to be able to load the specified program. See your System Operator about increasing your memory, and/or see Appendix C for hints on making your programs more memory efficient.

?Internal stack error (xxxxxx, yyyyyy). Please submit an SPR.

Please submit a Software Problem Report (SPR), stating that you received this error message and specifying the xxxxxx, yyyyyy numbers given in the message. Describe, if you can, the AlphaBASIC code and the situation (including data provided to the program run) that caused the problem.

?Invalid filename

The specified filename was not a legal filename. Make sure that the filename contained six or less characters, and has a valid extension.

?Invalid subroutine version

The subroutine specified in the XCALL statement is not in the correct format for the processor you are running on. You may have to rewrite or re-assemble the subroutine.

[Invalid syntax code]

An internal error has occurred in BASIC. Please notify Alpha Micro of this error by submitting a Software Problem Report (SPR). Include, if you can, the code and situation (including data provided to the program run) that caused it.

?IO to unopened file

The program tried to write to or from a file that was not open. Check your code and make sure that the file is OPENed before you try to use it.

?Line number must be from 1-65534

The line number entered is not legal. Check to see if your line number is negative, too large, or has a fractional part.

?Line x not found

The specified line was not found for a DELETE, LIST, etc., operation.

?LOKSER queue is full

Your program attempted to lock a record or file, and there was no room in the LOKSER queue to hold it. You may want to either delete some of the records/files in the LOKSER queue, or increase the size of the queue. See the *LOKSER User's Manual* that is appropriate for your system.

?Multiple PROGRAM statements

Only one PROGRAM statement is allowed per program. Remove any extra PROGRAM statements.

?Nested ++INCLUDE files are not permitted

Your program contains a ++INCLUDE command that specifies a file which also contains a ++INCLUDE command. You will either have to remove the ++INCLUDE command from the second file, or change the way you access that code.

?NEXT without FOR

A NEXT statement was encountered without a matching FOR statement. Check your program for the extra NEXT statement.

?No breakpoints set

This message just informs you that there are currently no breakpoints set in your program.

?No source program in text buffer

You tried to compile when there was no program in memory. Load or create a program before trying to compile again.

?Operator interrupt

This message appears when you use Control-C to interrupt a program run.

?Out of data

Your program tried to READ data after the data in all of the DATA statements had been used. You have to add more data to your DATA statements, execute a RESTORE statement, or examine your program to determine why it is doing too many READ operations (if the amount of your data is correct).

?Out of memory

BASIC is telling you that it has run out of memory in which to execute your program. See your System Operator about allocating more memory to your job, and/or see Appendix C about making your program more memory efficient.

?Out of memory - Compilation aborted

COMPIL is telling you that it does not have enough free memory to finish compiling your program. See your System Operator about allocating more memory to your job, and/or check your memory partition to see if you have any programs that can be removed.

?PPN not found

Your program tried to access a file in an account that does not exist. Check the syntax of your file specification. Make sure the correct disk and account have been given.

Program name:

You tried to SAVE or LOAD a program without providing a filename. Enter the filename at this point.

?Protection violation

Your program tried to write into another account where you do not have write privileges. Write to a different account, or talk to your System Operator about getting access to the account you want.

?Record in use

The random file record your program wants to access is already locked by another user. Only for LOKSER systems. You may want to specify WAIT'RECORD in your file access statement.

?Record not locked

Your program tried to update a random file record that it had not first locked with a READL instruction. Only for LOKSER systems.

?Record size overflow

Your program tried to read a file record into a variable larger than the file record size. Re-adjust your variable size, or your record size.

?Redimensioned array

You tried to redimension an array. An array can only be dimensioned once in any one BASIC program. Remove any extra DIM statements for that array.

?Remote not responding

Your program tried to access a file on a remote system, and that system was not open for access. Establish a connection with the remote system and try your program again.

?RESUME without error

A RESUME statement was encountered, but no error has occurred. Your program is probably running into your error trapping routine by accident. Make sure a GOTO, END, or similar statement stops or redirects your program before the error trapping routine.

?RETURN without GOSUB

A RETURN statement was encountered, but no corresponding GOSUB has been executed. Your program is probably running into your subroutine by accident. Make sure a GOTO, END,or similar statement stops or redirects your program before the subroutine.

?Runtime package (RUN.LIT) not found

The monitor was unable to locate the run-time package, or did not have sufficient memory in which to load it. Check to make sure RUN.LIT is in DSK0:[1,4], and/or check your memory partition for extraneous files. If necessary, see your System Operator for help locating RUN.LIT or increasing your memory.

?RUN file is in an incompatible format.

The program file you tried to RUN is not a .RUN file, or was compiled using an incompatible older version of COMPIL. You may have to rewrite or edit the BASIC file and/or recompile it.

?Source line overflow

A line in the source program, including continuation lines, exceeds 500 characters. Check your code for the excessive line, and see if you can break it into multiple statements, or reduce its length.

?Stack overflow

BASIC's internal stack has overflowed. This may be caused by nesting GOSUBs too deep, or branching out of FOR-NEXT loops. Check your code.

?Subroutine not found

The specified subroutine could not be found. Check your spelling and your directory.

?Subscript out of range

The specified subscript is outside the range specified in the DIM or MAP statement for the subscripted variable. Check your subscript variable.

?Syntax error

The syntax of the specified line is invalid. Check your spelling.

? System commands are illegal within the source program

Interactive BASIC commands (LOAD, DELETE, LIST, etc.) or system commands (such as DIR, VUE, etc.) are not valid within an AlphaBASIC source program. Check your file and remove any interactive or system commands.

?System error

This is used as a catch-all error message indicating that BASIC cannot identify the exact problem during the execution of the specified line. For example, if BASIC encounters a "Buffer not INITed" error message, it displays " System error," because it doesn't know how to handle this condition. Determine which line of your code is causing the problem, and correct it.

Transfer Address out of range

If a GOTO or GOSUB statement is pointing to an address (line number or label) that is more than 64 Kilobytes away from it in the program source file, the compiler (which uses 64K of memory) can't find the line number or label the statement is transferring control to. You may want to use the /A switch with COMPIL. /A increases the memory for COMPIL, allowing it to track addresses across larger areas of memory.

?Unable to find ++INCLUDE file xxxxxx.xxx

One of your ++INCLUDE commands specifies a file—xxxxxx.xxx—which AlphaBASIC cannot locate. Make sure you gave it the right name, or make sure the proper file exists or is created.

?Undefined line number or label

The line number or label specified in a GOTO, GOSUB or ON ERROR GOTO statement is not defined within the program. Correct the reference, or add the proper line number or label.

?Unmapped variable

You COMPILed a program using the unmapped variable. MAP the variable, or do not use the /M option.

?Unsupported function

Your program attempted to use a function that is not supported by AlphaBASIC. Refer to this manual for the functions AlphaBASIC supports.

?Write protected

Your program tried to write to a write-protected device. Before running the program again, see that the device's write-protection is off.

?Wrong number of subscripts

The number of subscripts specified is not the same as the number defined in the DIM or MAP statement for the subscripted variable. For example, if you defined a variable like this:

```
DIM Stock$(10,10)
```

As having two subscripts, and then used the variable like this:

```
Stock$(1,3,i) = "Two cases of nuts"
```

using three subscripts, you will see this error. Examine your program, checking your DIM and MAP statements and the use of the variable they define.

Appendix B - Reserved Words

Below is a list of the reserved words used by the BASIC compiler. Some of these reserved words designate routines that have not been implemented at this time. However, you must not use ANY of these reserved words as variable names or labels. This restriction applies to string variables as well as numeric variables. For instance, END$ and END are both illegal variable names.

ABS	ACS	ALLOCATE	AND
APPEND	ASC	ASN	ATN
BREAK	BYE	BYTE	CALL
CHAIN	CHR	CLOSE	COMPILE
CONT	COS	DATA	DATE
DATN	DEF	DELETE	DIM
ELSE	END	EOF	EQV
ERF	ERR	ERROR	EXP
EXPAND	FACT	FILEBASE	FIX
FOR	GO	GOSUB	GOTO
IF	++INCLUDE	INDEX	INDEXED'EXCLUSIVE
INPUT	INPUT LINE	INSTR	INT
IO	ISAM	KILL	LCS
LEFT	LEN	LET	LINE
LIST	LOAD	LOCK	LOG
LOG10	LOOKUP	MAP	MAX
MEM	MID	MIN	NEW
NEXT	NOEXPAND	NOT	ON
OPEN	OR	OUTPUT	PRINT
PROGRAM	RANDOM'FORCED	RANDOM	RANDOMIZE
READ	READL	REM	RESTORE
RESUME	RETURN	RIGHT	RND
RUN	SAVE	SCALE	SGN
SIGNIFICANCE	SIN	SPACE	SQR
STEP	STOP	STR	STRSIZ
SUB	TAB	TAN	THEN
TIME	TO	UCS	UNLOKR
USING	VAL	WAIT'FILE	WAIT'RECORD
WORD	WRITE	WRITEL	XCALL
XOR			

Appendix C - Making Your Programs More Efficient

In this appendix, we will explore a few ways to make your programs more time and memory efficient. We will discuss:

- Managing memory
- Increasing execution speed
- Increasing "readability" of program source files

MANAGING MEMORY

AlphaBASIC uses memory to store the variables that you use within your program. It also uses memory space for calculations. Here are some hints for reducing your program's use of memory:

- When you allocate arrays and/or strings, give them only as much memory they need, and no more. Allocating an array to 50 when you only need 10 subscripts wastes a lot of memory.
- Use MAP statements if you have a large number of strings that vary in size. By doing this, you can allocate each string individually, rather than setting STRSIZ to the size of the longest variable and wasting space for all the shorter variables. MAP statements generally help reduce memory use.
- Use as few variables as possible. Using one "scratch" variable that can be reset as needed instead of a number of individual variables can help.
- Do not rewrite sections of code that are used more than once. If you have ten lines of code that get used in more than one place, make those lines a subroutine. The more lines in the program, the more memory used.
- Where possible, MAP variables into the same storage area using the origin parameter (see Chapter 15).
- If you are using line numbers, do not assign a line number to lines containing MAP statements or lines that have only comments on them.

INCREASING EXECUTION SPEED

Many of the suggestions in the section above will also improve execution speed as well as saving memory. Some other hints for speed:

- Use floating point variables in arithmetic operations. Binary variables must be converted to floating point first, which takes time.
- Use the XCALL subroutines if you can, rather than writing program code. Assembly language routines are faster.

MAKING YOUR SOURCE PROGRAMS EASIER TO READ

One of the considerations that you should take into account whenever you write a BASIC program is "what happens if somebody else has to update the program?" Someday, someone may have to go into your source file (the .BAS file) and make corrections and/or additions to it. If you are not around to explain your code, will that person be able to understand it? Will it be easy for that person to find the section he or she needs to work on?

These questions are becoming increasingly important in the computer world. A few minutes spent making your source file clear and easy to read may save many hours of work for someone at a later time, and will help to minimize errors. It is also easier for you, because the program will be easier to debug, and easier to modify later.

You should practice writing your programs in good program form, so that your programs are always easy to understand and follow. It may take a few extra minutes when writing, but it will save much more time over the lifetime of the program!

Let us look at a couple of examples. Which of the following program lists is easier to read?

```
11  INPUT "INPUT THREE NUMBERS: ",A,B,C
13  D=A+B+C/3
20  ?"ANSWER: ",D
33  ? :INPUT "Do you want to try again?",E$
40  If E$="Y" then goto 11
43  END
```

or:

```
! This program takes three numbers and averages them:

MAP1  ANSWER,F
MAP1  FIRST,F
MAP1  SECOND,F
MAP1  THIRD,F
MAP1  QUERY,S,1

INPUT'NUMBERS:

      INPUT "Input three numbers: ", FIRST, SECOND, THIRD

CALCULATE'ANSWER:

      ANSWER = (FIRST + SECOND + THIRD)/3
      PRINT
      PRINT "The average is: ", ANSWER
      PRINT

DO'IT'AGAIN:

      PRINT "Would you like to average ";
      INPUT "more numbers? (Y or N): ", QUERY
      IF UCS(QUERY) = "Y" THEN GOTO INPUT'NUMBERS

END
```

Here are some tips, things the second program used that the first did not:

- If possible, do not use line numbers. They make your program harder to read and update. If you want to insert lines, you may have to renumber the rest of the program. Without line numbers, you can also indent the program lines more clearly to show levels and structures.
- Use descriptive variable names (ANSWER instead of D, for example) and labels. This makes the program much more comprehensible to another reader.
- Do what you can to make the program easy on the user. The second program used the UCS function on QUERY$ before comparing it to "Y". In the first program, if the user answers the question with a lower case y, the program will end! Good programs will not allow a user to enter incorrect responses.
- Use spaces to make lines easier to read. In most cases, BASIC ignores blank spaces, so you can use them to make text more readable. For example:

  ```
  ANSWER = (FIRST + SECOND + THIRD)/3
  ```

  instead of:

  ```
  13 D=A+B+C/3
  ```

- Use MAP statements so that all variables are defined at the front of the program. This is especially useful in large programs.

The programs above were too simple to show the use of some ideas. We have tried to use examples throughout this book that are written in structured, clear code, both to make our examples easier to read and to encourage good program style. Here are a few more things you may want to keep in mind:

- Whenever possible, group code into sections by using subroutines (using GOSUB and RETURN). Your program then becomes a series of sub-programs that each accomplish a task. Organizing your program in this manner also makes it easier to design and write your program.
- Do not use more variables than you need. The fewer variable names you have, the easier it is to keep track of them.

Appendix D - Special Screen Handling Codes

Below is a list of the terminal handling codes defined when this manual was printed. Chapter 13 explains how these codes can be used by AlphaBASIC programs.

All TCRT codes are reserved for future expansion by Alpha Micro. If you have a specific need for a new TCRT function, contact the Technical Assistance Center at Alpha Micro to reserve a TCRT code number. By using this reservation process, problems of conflicting and incompatible TCRT functions are avoided.

Calls marked with an asterisk (*) are either obsolete calls, or calls that are for special purposes. We recommend that they not be used, as they may change in the future.

TCRT CODE	FUNCTION
0	Clear Screen and set normal intensity
1	Cursor Home (move to 1,1)
2	Cursor Return (move to column 1 without linefeed)
3	Cursor Up one row
4	Cursor Down one row
5	Cursor Left one column
6	Cursor Right one column
7	Lock Keyboard
8	Unlock Keyboard
9	Erase to End of Line
10	Erase to End of Screen
11	Enter Background Display Mode (reduced intensity)
12	Enter Foreground Display Mode (normal intensity)
13	Enable Protected Fields
14	Disable Protected Fields
15	Delete Line
16	Insert Line
17	Delete Character
18	Insert Character
19	Read Cursor Address
20	Read Character at Current Cursor Address
21	Start Blinking Field
22	End Blinking Field
23	Start Line Drawing Mode (enable alternate character set)
24	End Line Drawing Mode (disable alternate character set)
25	Set Horizontal Position
26	Set Vertical Position
27	Set Terminal Attributes

TCRT CODE	FUNCTION
28	Cursor on
29	Cursor off
30	Start Underscore
31	End Underscore
32	Start Reverse Video
33	End Reverse Video
34	Start Reverse Blink
35	End Reverse Blink
36	Turn Off Screen Display
37	Turn On Screen Display
38	Top Left Corner
39	Top Right Corner
40	Bottom Left Corner
41	Bottom Right Corner
42	Top Intersect
43	Right Intersect
44	Left Intersect
45	Bottom Intersect
46	Horizontal Line
47	Vertical Line
48	Intersection
49	Solid Block
50	Slant Block
51	Cross-Hatch Block
52	Double Line Horizontal
53	Double Line Vertical
54	Send Message to Function Key Line
55	Send Message to Shifted Function Key Line
56	Set Normal Display Format
57	Set Horizontal Split (Follow with Row Code)
58	Set Vertical Split (39 Character Columns)
59	Set Vertical Split (40 Character Columns)
60	Set Vertical Split Column to Next Character
61	Activate Split Segment 0
62	Activate Split Segment 1
63	Send Message to Host Message Field
64	Up-Arrow
65	Down-Arrow
66	Raised Dot
67	End of Line Marker
68	Horizontal Tab Symbol
69	Paragraph
70	Dagger
71	Section
72	Cent Sign
73	One-Quarter
74	One-Half
75	Degree

TCRT CODE	FUNCTION
76	Trademark
77	Copyright
78	Registered
79	Print screen
80	Set to wide (132 column) mode
81	Set to normal (80 column) mode
82	Enter transparent print mode
83	Exit transparent print mode
84	Begin writing to alternate page
85	End writing to alternate page
86	Toggle page
87	Copy to alternate page
88	Insert column
89	Delete column
90	Block fill with attribute
91	Block fill with character
92	Draw a box
93	Scroll box up one line
94	Scroll box down one line
95	Select jump scroll
96	Select fast smooth scroll
97	Select medium-fast smooth scroll
98	Select medium-slow smooth scroll
99	Select slow smooth scroll
100	Start underscored, blinking field
101	End underscored, blinking field
102	Start underscored, reverse field
103	End underscored, reverse field
104	Start underscored, reverse, blinking field
105	End underscored, reverse, blinking field
106	Start underscored text without space
107	End underscored text without space
108	Start reverse text without space
109	End reverse text without space
110	Start reverse blinking text without space
111	End reverse blinking text without space
112	Start underscored blinking text without space
113	End underscored blinking text without space
114	Start underscored reverse text without space
115	End underscored reverse text without space
116	Start underscored reverse blinking text without space
117	End underscored reverse blinking text without space
118	Start blink without space
119	End blink without space
120	Set cursor to blinking block
121	Set cursor to steady block
122	Set cursor to blinking underline

TCRT CODE	FUNCTION
123	Set cursor to steady underline
124	Reserved
125	Reserved
126	Reserved
127	Reserved
128	Select top status line without address
129	End status line (all kinds)
130	Select unshifted status line without address
131	Select shifted status line without address
132	Select black text
133	Select white text
134	Select blue text
135	Select magenta text
136	Select red text
137	Select yellow text
138	Select green text
139	Select cyan text
140	Select black reverse text
141	Select white reverse text
142	Select blue reverse text
143	Select magenta reverse text
144	Select red reverse text
145	Select yellow reverse text
146	Select green reverse text
147	Select cyan reverse text
148	Save a rectangular area
149	Restore a rectangular screen area
150	Enter full graphics mode
151	Exit full graphics mode
152	Draw a box with rounded corners
153	Draw a window style box
154	Draw a box with double lines
155	Enable proportionally spaced text
156	Disable proportionally spaced text
157	Select color palette by RGB value
158	Enable graphics cursor
159	Disable graphics cursor
160	Select graphics cursor shape
161	Include graphics cursor location
162	Define graphics cursor regions
163	Output form feed character
164	Output line feed character
165	Output new line character
166	Output vertical tab character
167	Output plus-or-minus character
168	Output greater-than-or-equal character
169	Output less-than-or-equal character
170	Output not-equal character

TCRT CODE	FUNCTION
171	Output British pound character
172	Output Pi character
173	Enter bi-directional print mode
174	Exit bi-directional print mode
175	Set terminal time
176	Set terminal date
177	Select color palette by HLS value
178	Select PC character set
179	Select default character set
180	Select PC terminal emulation
181	Select ASCII terminal operation
182	Select AUX port host
183	Select main port host
184	Select toggle host ports
185	Select 8-bit character display
186	Select 7-bit character display
187	Select 8-bit keyboard mode
188	Select 7-bit keyboard mode
189	Select primary printer port
190	Select secondary printer port
191	Select 161 column display mode
192-255	Reserved for future use

The actual routines that perform the screen controls are in the specific terminal drivers and not in AlphaBASIC itself. Not all terminal drivers have all of the functions above simply because not all terminals are able to perform all of these functions.

Appendix E - Writing Assembly Language Subroutines

This appendix gives details on assembly language programming for those who want to write assembly language subroutines for AlphaBASIC programs. For further information about assembly language, see the *Assembly Language Programmer's Manual* and the *Monitor Calls Manual* appropriate to your system.

You write assembly language subroutines in the same manner that you write assembly language programs. The resulting .LIT program file must then be renamed to give it the extension .SBR, indicating that it is a subroutine.

When an XCALL statement is executed by AlphaBASIC, the named subroutine is located in memory and then executed.

AlphaBASIC first saves all registers, then sets certain parameters into those registers for use by the external subroutine. The addresses of the arguments are calculated and entered into an argument list in memory along with their sizes and type codes. The base address of this list is then passed to the user routine in register A3.

The arguments may be one of two basic forms:

- A variable name, in which case the argument entry in the list references the selected variable within the user impure area. This variable is available to the called subroutine for both inspection and modification.
- An expression (numeric or string), in which case the expression is evaluated and the result is placed on the arithmetic stack (referenced by A5).
- This result, instead of a single variable, is then referenced in the argument list entry. It is only available for inspection, since the stack is cleared when the subroutine exits.
- You are free to write a routine that uses and modifies all of the general work registers (A0-A6 and D0-D7), and your program may use the stack for work space as required. When your subroutine has completed its execution, it must return to the run-time system by executing a RTN subroutine return instruction.

REGISTER PARAMETERS

The following registers are set up by the run-time system to be used as required by the external subroutine. They may be modified, if desired, since they have been saved before the subroutine was called:

A0 Indexes the user impure variable area. A0 is used throughout the run-time system to reference all user variables. Details on the format of this area are not available at this time. A0 may be used as a work register.

A3 Points to the base of the argument list. A3 may be used to scan the argument list for retrieval of the argument parameters.

A4 Points to the base of the free memory area that may be used by the external subroutine as work space. This is actually the address of the first word following the argument list in memory, and, if desired, may be used to store a terminator word to stop the scanning of the argument list.

A5 This is the arithmetic stack index used by the run-time system. The stack is built at the top of the user partition and grows downward as items are added to it. When the external subroutine is called, A5 points to the current stack base. Since the arithmetic stack may contain valid data, the external subroutine must not use the word indexed by A5 or any words above it.

ARGUMENT LIST FORMAT

The list of arguments specified in the XCALL statement may range from no arguments at all to a number limited only by the space on the command line. To pass these arguments to the external subroutine, an argument list is built in memory which describes each variable named in the list and tells where it can be located in the user impure area.

The variables themselves are not actually passed to the subroutine, but rather their absolute locations in memory are. In this way, the subroutine may inspect them and modify them directly in their respective locations. This does not apply to expressions which are built on the stack as described previously.

A3 points to the first word of the argument list, which is a binary count of how many arguments were contained in the XCALL statement. Following this count word comes one 5-word descriptor block for each argument specified. If there are no arguments in the XCALL statement, the argument list consists only of the single count word containing the value of zero.

The format of each 10-byte block describing one argument is:

Entry 1 One word containing a variable type code. Bits 0-2 contain the type code for the specific variable:

Value	Bits Set	Type
0	None	unformatted
2	1 only	string
4	2 only	floating point
6	1 and 2	binary

Bit 3 is reserved for internal use.

Bit 4 is set if the variable is an array.

Bits 5 through 15 are currently unassigned.

Entry 2 One longword containing an absolute address of a variable in a user impure area. This address is the first byte of the variable no matter what its type or size might be.

Entry 3 One longword containing the size of the variable in bytes.

Note that the above descriptions also apply to the expression arguments, except that the results are located above the address specified by A5 instead of below it.

The argument list is built in free memory directly above the currently allocated user impure area. A4 points to the word immediately following the last word in the argument list. You may scan the argument list and determine its end either by decrementing the count word at the base of the list or by scanning until the scan index reaches the address in A4.

CONVERTING ARGUMENTS TO BINARY FORMAT

A standard subroutine, $GTARG, is provided in the system subroutine library SYSLIB.LIB to assist in converting floating point and string arguments to binary format for processing within an AlphaBASIC subroutine.

For further information, see the *Monitor Calls Manual* appropriate to your system.

FREE MEMORY USAGE

When the subroutine is called, indexes A4 and A5 mark the beginning and end of the free memory that is currently available for use as work space. This area is not preserved by the run-time system, and the subroutine must not count on its security between XCALL statements.

The word at @A4 may be used as the first word, but the word at @A5 is the base of the arithmetic stack and must not be destroyed. The last word of actually free memory is at -2(A5).

The run-time system has its own internal memory management system and does not conform to the operating system memory management method.

Therefore, the external subroutine must not use the GETMEM monitor calls to generate a block of work space in memory. Also, if any file calls are to be done they must be done with internal buffers, since the INIT call sets up a buffer by using the GETMEM monitor call.

LOCATING OPEN FILES

A standard library routine, $FLSET, is provided in the system subroutine library, SYSLIB.LIB, to locate the DDB associated with a file that has been opened by an AlphaBASIC program.

For further information, see the *Monitor Calls Manual* appropriate for your system.

PROGRAM HEADERS

All AlphaBASIC subroutines must contain a program header at the start of the subroutine. Program headers are defined by using the PHDR macro, discussed in the *Monitor Calls Manual* appropriate for your system.

Appendix F - The ASCII Character Set

The next few pages contain charts that list the complete ASCII character set. We provide the octal, decimal and hexadecimal representations of the ASCII values. Note that the first 32 characters are non-printing control characters

THE CONTROL CHARACTERS

CHARACTER	OCTAL	DECIMAL	HEX	MEANING
NULL	000	0	00	Null (fill character)
SOH	001	1	01	Start of Heading
STX	002	2	02	Start of Text
ETX	003	3	03	End of Text
ECT	004	4	04	End of Transmission
ENQ	005	5	05	Enquery
ACK	006	6	06	Acknowledge
BEL	007	7	07	Bell code
BS	010	8	08	Back Space
HT	011	9	09	Horizontal Tab
LF	012	10	0A	Line Feed
VT	013	11	0B	Vertical Tab
FF	014	12	0C	Form Feed
CR	015	13	0D	Carriage Return
SO	016	14	0E	Shift Out
SI	017	15	0F	Shift In
DLE	020	16	10	Data Link Escape
DC1	021	17	11	Device Control 1
DC2	022	18	12	Device Control 2
DC3	023	19	13	Device Control 3
DC4	024	20	14	Device Control 4
NAK	025	21	15	Negative Acknowledge
SYN	026	22	16	Synchronous Idle
ETB	027	23	17	End of Transmission Blocks
CAN	030	24	18	Cancel
EM	031	25	19	End of Medium

PRINTING CHARACTERS

CHARACTER	OCTAL	DECIMAL	HEX	MEANING
SS	032	6	1A	Special Sequence
ESC	033	27	1B	Escape
FS	034	28	1C	File Separator
GS	035	29	1D	Group Separator
RS	036	30	1E	Record Separator
US	037	31	1F	Unit Separator
SP	040	32	20	Space
!	041	33	21	Exclamation Mark
"	042	34	22	Quotation Mark
#	043	35	23	Number Sign
$	044	36	24	Dollar Sign
%	045	37	25	Percent Sign
&	046	38	26	Ampersand
‘	047	39	27	Apostrophe
(	050	40	28	Opening Parenthesis
)	051	41	29	Closing Parenthesis
*	052	42	2A	Asterisk
+	053	43	2B	Plus
,	054	44	2C	Comma
-	055	45	2D	Hyphen or Minus
.	056	46	2E	Period
/	057	47	2F	Slash
0	060	48	30	Zero
1	061	49	31	One
2	062	50	32	Two
3	063	51	33	Three
4	064	52	34	Four
5	065	53	35	Five
6	066	54	36	Six
7	067	55	37	Seven
8	070	56	38	Eight
9	071	57	39	Nine
:	072	58	3A	Colon
;	073	59	3B	Semicolon
<	074	60	3C	Less Than
=	075	61	3D	Equal Sign
>	076	62	3E	Greater Than
?	077	63	3F	Question Mark
@	100	64	40	Commercial At
A	101	65	41	Upper Case Letter
B	102	66	42	Upper Case Letter
C	103	67	43	Upper Case Letter
D	104	68	44	Upper Case Letter
E	105	69	45	Upper Case Letter
F	106	70	46	Upper Case Letter

CHARACTER	OCTAL	DECIMAL	HEX	MEANING
G	107	71	47	Upper Case Letter
H	110	72	48	Upper Case Letter
I	111	73	49	Upper Case Letter
J	112	74	4A	Upper Case Letter
K	113	75	4B	Upper Case Letter
L	114	76	4C	Upper Case Letter
M	115	77	4D	Upper Case Letter
N	116	78	4E	Upper Case Letter
O	117	79	4F	Upper Case Letter
P	120	80	50	Upper Case Letter
Q	121	81	51	Upper Case Letter
R	122	82	52	Upper Case Letter
S	123	83	53	Upper Case Letter
T	124	84	54	Upper Case Letter
U	125	85	55	Upper Case Letter
V	126	86	56	Upper Case Letter
W	127	87	57	Upper Case Letter
X	130	88	58	Upper Case Letter
Y	131	89	59	Upper Case Letter
Z	132	90	5A	Upper Case Letter
[	133	91	5B	Opening Bracket
\	134	92	5C	Back Slash
]	135	93	5D	Closing Bracket
^	136	94	5E	Circumflex
_	137	95	5F	Underline
`	140	96	60	Grave Accent
a	141	97	61	Lower Case Letter
b	142	98	62	Lower Case Letter
c	143	99	63	Lower Case Letter
d	144	100	64	Lower Case Letter
e	145	101	65	Lower Case Letter
f	146	102	66	Lower Case Letter
g	147	103	67	Lower Case Letter
h	150	104	68	Lower Case Letter
I	151	105	69	Lower Case Letter
j	152	106	6A	Lower Case Letter
k	153	107	6B	Lower Case Letter
l	154	108	6C	Lower Case Letter
m	155	109	6D	Lower Case Letter
n	156	110	6E	Lower Case Letter
o	157	111	6F	Lower Case Letter
p	160	112	70	Lower Case Letter
q	161	113	71	Lower Case Letter
r	162	114	72	Lower Case Letter
s	163	115	73	Lower Case Letter
t	164	116	4	Lower Case Letter
u	165	117	75	Lower Case Letter

CHARACTER	OCTAL	DECIMAL	HEX	MEANING
v	166	118	76	Lower Case Letter
w	167	119	77	Lower Case Letter
x	170	120	78	Lower Case Letter
y	171	121	79	Lower Case Letter
z	172	122	7A	Lower Case Letter
{	173	123	7B	Opening Brace
|	174	124	7C	Vertical Line
}	175	125	7D	Closing Brace
~	176	126	7E	Tilde
DEL	177	127	7F	Delete

Appendix G - Sample Program

This appendix contains a sample AlphaBASIC program that converts a number from one number base to another. You may convert numbers from the binary through hexadecimal (2-16) number bases to another number base in the same range. For example, you can translate an octal number to its hexadecimal form, or vice versa. Below is a sample run of the program:

```
"CONVRT"—CONVERT BETWEEN NUMBER BASES
   Enter positive numbers, any base from 2 to 16
   (Enter a zero for the FROM BASE? question to end
   the program)

FROM BASE? 10 [RETURN]
TO BASE? 2 [RETURN]
BASE 10 NUMBER? 364 [RETURN]
BASE 2 NUMBER = 101101100 [RETURN]

FROM BASE? 0 [RETURN]
.
```

The program:

```
   ! Program to convert between number bases:

   MAP1   IN'VARIABLES                          ! input base variables
      MAP2   IN'NUMBER,S,50                     ! input number string
      MAP2   IN'BASE,F                          ! base of input number,
                                                ! 2 through 16 valid
   MAP1   OUT'VARIABLES                         ! output base variables
      MAP2   OUT'NUMBER,S,50                    ! output number string
      MAP2   OUT'BASE,F                         ! base of output number,
                                                ! 2 through 16 valid
   MAP1   VALID'DIGIT,S,16,"0123456789ABCDEF"
                                                ! valid digits (base x
                                                ! contains x leftmost)
   MAP1   MISC'VARIABLES                        ! miscellaneous variables
      MAP2   BASE10'NUMBER,F                    ! input string in base 10
      MAP2   ERROR'FLAG,F                       ! flag set if invalid digit
      MAP2   LEADING'BLANK,F                    ! flag reset when first
                                                ! non- blank character found
      MAP2   LOOP'1,F                           ! FOR-NEXT index #1
      MAP2   LOOP'2,F                           ! FOR-NEXT index #2
      MAP2   WORK'1,F                           ! scratch variable

START'OF'PROGRAM:

   DISPLAY'BANNER:
      PRINT CHR(34);
      PRINT "CONVRT";
      PRINT CHR(34);
      PRINT "-- CONVERT BETWEEN NUMBER BASES"
      PRINT TAB(10);"Enter positive numbers, any base from 2 to 16"
      PRINT TAB(10);"(Enter a zero for the FROM BASE? question to end"
      PRINT TAB(10);"the program.)"

   ENTER'IN'BASE:                                   ! enter input base
      PRINT                                         ! set to 0 in case of
      IN'BASE = 0                                   ! carriage return
```

```
    INPUT "FROM BASE? ",IN'BASE                    ! enter input base
    IF IN'BASE = 0 THEN END
    IF IN'BASE >= 2 AND &
       IN'BASE <= 16 GOTO ENTER'OUT'BASE
    PRINT CHR(7);"INVALID BASE!"                   ! bases 2 to 16 only
    GOTO ENTER'IN'BASE                             ! re-enter base

ENTER'OUT'BASE:                                    ! enter output base
    OUT'BASE = 0                                   ! set to 0 in case of CR
    INPUT " TO BASE? ",OUT'BASE                    ! enter output base
    IF OUT'BASE = 0 GOTO ENTER'IN'BASE             ! if 0/CR, re-enter
    IF OUT'BASE >= 2 AND &
       OUT'BASE <= 16 GOTO ENTER'IN'NUMBER
    PRINT CHR(7);"INVALID BASE!"                   ! bases 2 to 16 only
    GOTO ENTER'OUT'BASE                            ! re-enter output base

ENTER'IN'NUMBER:                                   ! enter input number
    IN'NUMBER = ""                                 ! set to null in case of CR
    PRINT "BASE";IN'BASE;"NUMBER? ";
    INPUT LINE "",IN'NUMBER                        ! enter input number

VALIDATE'NUMBER:                                   ! check/convert input number
    LEADING'BLANK = 1
    ERROR'FLAG = 0                                 ! initialize flags
    BASE10'NUMBER = 0                              ! initialize base 10 number
    FOR LOOP'1 = 1 TO LEN(IN'NUMBER)               ! check one char/time
       IF IN'NUMBER[LOOP'1;1] <> " " &
          GOTO NON'BLANK                           ! skip if non-blank
       IF LEADING'BLANK = 0 THEN &
       LOOP'1 = LEN(IN'NUMBER)                     ! end if not leading blank
    GOTO END'LOOP'1                                ! conversion of input number;
                                                   ! otherwise skip it

NON'BLANK:                                         ! process non-blank chars
    LEADING'BLANK = 0                              ! reset leading blank flag,
    ERROR'FLAG = 1                                 ! assume invalid character
                                                   ! until valid one found
    FOR LOOP'2 = 1 TO IN'BASE                      ! check digit using
                                                   ! valid character list
    IF IN'NUMBER[LOOP'1;1] &
       <> VALID'DIGIT[LOOP'2;1] THEN &
       GOTO END'LOOP'2                             ! invalid character, try next
    BASE10'NUMBER = &
       BASE10'NUMBER * IN'BASE + LOOP'2 - 1        ! convert and shift
    ERROR'FLAG = 0                                 ! reset—valid found: no
    LOOP'2 = IN'BASE                               ! need to check more digits
END'LOOP'2:
    NEXT LOOP'2                                    ! next valid or end
    IF ERROR'FLAG <> 0 THEN &
       LOOP'1 = LEN(IN'NUMBER)                     ! if bad character found,
                                                   ! check no further
END'LOOP'1:                                        ! next character in input
    NEXT LOOP'1                                    ! string or end
    IF ERROR'FLAG = 0 THEN &
       GOTO CALCULATE'OUT'NUMBER
    PRINT CHR(7);
    PRINT "INVALID BASE";IN'BASE;"NUMBER!"         ! bad character found
    GOTO ENTER'IN'BASE                             ! start over
CALCULATE'OUT'NUMBER:                              ! convert to output base
    OUT'NUMBER = ""                                ! start with null string
CONTINUE'CALCULATION:
    WORK'1 = INT(BASE10'NUMBER/OUT'BASE)           ! remainder of number/base
                                                   ! is base 10 of next digit
    OUT'NUMBER = &
       VALID'DIGIT[1 + BASE10'NUMBER - &
       WORK'1 * OUT'BASE;1] + OUT'NUMBER           ! 1+ to adjust for position
                                                   ! in valid digits string
```

```
    BASE10'NUMBER = WORK'1                          ! new number is integer part
                                                    ! of number/base
    IF BASE10'NUMBER <> 0 THEN &
       GOTO CONTINUE'CALCULATION                    ! done when new number = 0

PRINT'OUT'NUMBER:                                   ! print output number
    PRINT "BASE";OUT'BASE;
    PRINT "NUMBER = ";OUT'NUMBER
GET'NEXT:
    GOTO ENTER'IN'BASE                              ! start over from the top
    END
```

Glossary

ACCOUNT

This is an area on the disk that can contain a group of files. It is identified by an account number such as [100,2]

ALLOCATE

The process of setting aside an amount of memory for a specific purpose.

ARRAY

A structure of data such that each piece of data (each variable) follows each other piece in a logical, accessible order. The data is ordered sequentially, i.e. one, two, three.

BASIC

Beginners All-purpose Symbolic Instruction Code.

BINARY

Data that is in the form of a series of 0's and 1's. This is how data is represented at the core level of the computer.

BOOLEAN

An operation that results in a TRUE or FALSE condition. Thus, a Boolean operator would compare two expressions and return a value of TRUE or FALSE depending on how they compared.

CHAINING

Moving from an AlphaBASIC program to another AlphaBASIC program or to a command file.

COMPILE

The process of changing a BASIC source file into an executable RUN file. Because BASIC is a high-level language, it must be converted into a lower level language by the compiler before it can execute.

CONVERSION

The process of changing one type of data into another. For example, if you have a string expression, "45," you can convert it to a numeric value by using the VAL function.

DEADLOCK

A situation where two programs are each trying to access a file that is locked by the other. Neither program can proceed, and the "deadlock" cannot be broken, except by a manual reset of the system.

DEBUGGING

The process of removing the errors, or "bugs," from a program. Most programs, when first written, do not work quite the way they are supposed to, and so a period of debugging is needed to fix the problems.

DELIMITER

An alphanumeric character that marks a division between two pieces of data. For example, in the date display 12/04/86, the slash "/" is a delimiter. A dash "-" is also a common delimiter.

DISKFILE

An area of memory on a hard disk that contains a program, text, or data file. Unless such a file is erased, it resides permanently on the disk.

DEVN:

A device specification. This abbreviation represents a logical unit of a physical device. Such a specification usually refers to a disk, but it can represent any valid system device (such as a magnetic tape drive or a printer for which a special driver program is required). For example:

DSK2: MIN0: VCR0:

ECHO

A response by your terminal to what you press on the keyboard. If you press a key (say the "a"), and you see a response on your terminal screen (an "a" appears), then your terminal is in ECHO mode (the normal way for it to be). Sometimes—for instance—when you are entering a password—the terminal echoing can be turned off, so that what you type goes to the computer, but is not displayed on the screen.

EXPONENT

If a number has more digits than BASIC usually displays, the number is then displayed as an exponent, which is a number expressed as a factor of ten. For example, 5,000,000 would be expressed as 5E10^6. The E indicates the number is an exponent, and the 10^6 indicates how many zeroes must follow the 5.

EXPRESSIONS

A portion of BASIC syntax that can contain variables, constant values, operator symbols, functions, or combinations of these things. Expressions are not complete BASIC statements, but make up parts of BASIC statements. For example:

```
X = SIN(SQR(Number))
```

Everything to the right of the equal sign is an expression, and the expression combines with the X= to form a BASIC statement.

FILE LOCKING

A process by which a file that is being used by one program is "locked", so that any other program that tries to access that file will be told that the file is in use. This prevents problems that can occur if two programs were to work on the same file at the same time.

FILESPEC

A file specification. Such a specification identifies a file. It usually has these elements:

```
Devn:filename.extension[p,pn]
```

Where *filename.extension* is the name of the file and its identifying extension (i.e., PAYROL.TXT), and [p,pn] is the account number (see above).

FLOATING POINT#

A "real" number. That is, a number that may contain a decimal part. For example, 4.175. Called "floating point" because the number of digits both right and left of the decimal point may vary, so the decimal point has no fixed position.

INCREMENT

The process of increasing the value of a variable by a constant amount. For example, the statement:

```
NUMBER = NUMBER + 1
```

Increments the variable NUMBER by one.

INDEXED FILE

An ISAM file (see ISAM, below).

INPUT

Any data coming into the program or computer. Input may come from the user at the keyboard, from a data file, or from some other external device.

INTEGER#

A "whole" number. That is, a number that does not have a decimal part. For example, 3.

INTERACTIVE

Mode of BASIC operating in which the computer and the person writing or running a program interact. The computer responds directly to every input by the operator.

ISAM

Indexed Sequential Access Method. An advanced type of random file that handles sophisticated data structures.

LABEL

A name that marks a place in a program, so that other lines in the program can transfer control to the spot. For example, if you had a subroutine that added up three numbers, you might label it ADD'THREE'NUMS:. The label may not contain spaces, and must have a colon ":" after it (though you do not use the colon when referring to the label).

LIBRARY

In AlphaBASIC use, a group of subroutines and/or programs in a special account. These subroutines/programs can be called or incorporated into your AlphaBASIC programs.

LIST

The process of displaying the contents of a BASIC program, so that you can see the lines of code that make up the program.

LITERAL

An actual value, rather than a variable. For example, in the statement:

```
A$ = "This is a literal string."
```

the quoted material to the right of the equal sign is a literal string, whereas A$ is a string variable.

LOGICAL OPERATORS

BASIC words that perform tests of logic on expressions. Often used in IF-THEN statements, so that the program can do different things depending on how the "logic test" turns out. For example, the statement:

```
IF NUMBER < 1 OR NUMBER > 100 &
  THEN GOTO ILLEGAL'NUMBER
```

Performs two tests on the variable NUMBER, to see if it is less than 1 or greater than 100. The OR is the logical operator, and instructs BASIC to GOTO ILLEGAL'NUMBER if either of the two tests are true.

LOOP

A repetition of the same section of code. For example:

```
FOR I = 1 TO 10
  PRINT I
 NEXT I
```

The above lines cause the PRINT I statement to be executed 10 straight times, and each time, the variable I is incremented by 1.

MAPPED VARIABLE

A variable defined at the start of the program by a MAP statement. Mapped variables can be grouped into data structures.

MOUNT

The process of preparing a disk device for use.

NULL VALUE

A variable expression that has no value. For a number, the value would be zero (0). For a string variable, it would be empty quotation marks ("").

OUTPUT

Any data that comes out from an AlphaBASIC program. Output is displayed on the terminal or on paper, or is placed in a data file.

PRECEDENCE

What comes first. In BASIC, certain expressions could be evaluated in more than one way, so a hierarchy of precedence is built into BASIC to determine which operations are done first. For example:

```
NUMBER = 1 + 4 * 6
```

will make NUMBER equal to 25, because multiplication has more precedence than addition (4 * 6 will be done before 1 is added). If addition was higher in precedence, then NUMBER would equal 30 (1 + 4 would be done, then * 6).

QUEUE

A "waiting list". For example, when printing files, a queue is used so that the files sent to the printer do not print all at the same time, but rather in an orderly way—first come, first served.

RANDOM

A method of storing data where the data is placed at random in memory, and pointers keep track of where the data record is. This type of data storage makes data retrieval faster.

RECORD

A piece of data. For instance, if you think of a data file as a bookcase, each "record" would be like a book—a separate unit of data contained within the file.

SAVE

The process of transferring a file from user memory (which is temporary) to a disk (which is permanent).

SCALING

A way of storing variables by multiplying them by an offset so that they have no decimal fractions. This is done so that mathematical operations are more accurate. Useful if you use large or long numbers which may be subject to "round off" errors.

SEQUENTIAL

A method of file storage in which each item of data follows the previous item of data in order. Sequential data files are slower for data retrieval than random files, but they can be edited using AlphaVUE.

SIGNIFICANCE

How many digits will be used to represent a number. Normally, numbers are given in 6 digit significance, but can be set from 1 to 11 by the SIGNIFICANCE statement.

SOURCE

A file containing the original text or code. In AlphaBASIC, the source file is the file containing the AlphaBASIC code (the file usually has a .BAS extension).

SPOOLING

A method of sending a file to a printer queue, so that it can await printing.

STRING

An ASCII character or a group of ASCII characters. ASCII characters include letters, numbers, punctuation marks, and special symbols (see Appendix C for a complete list).

STRING LITERAL

A string that is enclosed in quotation marks to indicate that the string is as it appears (as opposed to being a variable). For example:

```
"This is a literal string of characters."
```

SUBROUTINE

A section of program code that operates as a unit and does a task. The main program "calls" the subroutine, and, when it has done its work, it returns control back to the next line of the main program.

SUBSCRIPTS

The numbers that represent the elements of an array.

SUBSTRING

A part of a string. A substring is produced by use of substring modifiers or some of the string functions. For example, if you had a string, and you only wanted to print the first five characters rather than the whole string, you could say:

```
PRINT INPUT'STRING$[1,5]
```

the first five characters printed are a substring of INPUT'STRING$.

SWITCH

An option that can be added to a command.

SYSTEM MEMORY

An area of memory that is available to all of the users of the computer. Any file that is loaded into this memory area (by use of the SYSTEM command within the system initialization file) can be used by any person or program. The advantage of using system memory is that, if a file is used regularly by many users, each user does not have to load a copy of that file into his or her own user memory. This saves both time and memory space.

UNARY (Plus/Minus)

An operative that changes the value of a number or variable to positive or negative. For example, if NUM = -2, then +NUM will change the value of NUM to 2. And if you add a minus sign to a positive number, it will change the value to a negative number.

USER MEMORY

An area of memory that is available to one user.

VARIABLE

A name that represents a number or character string that can change during the run of the program.

WILDCARD

A symbol that can represent a range of other characters. You might want to think of it as a joker in a deck of cards. For example, if you wanted to erase three files: FILE01.DAT, FILE02.DAT, and FILE03.DAT, you could type all this:

```
.ERASE FILE01.DAT RETURN
.ERASE FILE02.DAT RETURN
.ERASE FILE03.DAT RETURN
```

or you could type:

```
.ERASE FILE0?.DAT
```

where the question mark symbol "?" represents all characters, or:

```
.ERASE *.DAT RETURN
```

where the asterisk "*" represents all combinations of characters (in this case, all filenames). Of course, you would not use the second command if you had other .DAT files that you wanted to keep.

XCALL

An AlphaBASIC command that runs a subroutine that is external (outside) to the current BASIC program. The subroutines that XCALL can call are useful programs many people use, and thereby save you the time and trouble of writing such a subroutine yourself.

Document History

Revision	Date	Description
A00	May, 1986	New document. New version of the AlphaBASIC User's Manual, DWM-00100-01. Rewritten and re-organized. Corrected minor errors in the old manual and made it easier to read.
A01	May, 1986	Added new error codes, and added the index.
A02	December, 1988	Corrected minor problems.
A03	October, 1996	Corrected minor errors. Complete reformatting.
A04	June, 1998	Corrected bit definitions in Appendix E.
A05	April, 1999	Added /I option to Compiler Options on Page 5-4

INDEX

' · 2-3
! · 2-2
· 16-2
$FLSET · E-3
$GTARG · E-3
& · 5-2
; (· 8-1, 10-15
, · 8-1, 10-16
.BAS extension · 5-2
.LIT extension · 5-2
: · 2-3, 3-7
; · 8-1, 10-15
@ · 15-6, 15-11
++INCLUDE · 2-6

A

Absolute value · 11-2
Account specification · 5-4
ACS(X) · 11-4
ALLOCATE · 16-5
AlphaBASIC · 1-2
Alphabetic characters · 1-3
Alphanumeric characters · 1-3
AlphaVUE · 1-3, 5-1
AM-100 · 15-11
Ampersand symbol (&) · 5-2
AND · 7-1, 7-2
Apostrophe · 6-1
APPEND mode · 16-4
Argument list format · E-2
Arguments · 11-1, 11-4, 11-6
Arithmetic stack · E-1
Array variables · 6-2
  Default size · 6-4
  Numeric · 6-3
  String · 6-3
ASCII · 11-2, 11-5
ASCII collating sequence · 2-5
ASN(X) · 11-4
At sign (@) · 15-11
ATN(X) · 11-4
Atsign (@) · 15-6, E-3

B

B.A.S.I.C. - what it stands for · 1-1
BASIC · 1-2
  Compiler · 1-4
  Subprograms · 2-6
BASIC.LIT · 1-4, 3-5, 3-8
Binary format conversion · E-3
Binary variables · 15-4
BREAK · 4-1
Breakpoints · 3-1, 4-6
BYE · 3-2

C

CALL · 10-5, 10-14
Carriage return/lfeed · 10-16
CHAIN · 16-9, 17-1
Clock ticks · 12-3
CLOSE · 16-2
Closing ISAM files · 20-6
COBOL language · 1-2
Colon symbol · 3-7
Comma symbol (,) · 8-1, 10-16
Command files · 17-2
Comments · 2-2
COMMON · 10-2, 17-1
COMPIL · 1-4, 5-2, 5-3
Compiler option - interactive mode · 3-6
Compiling a program · 5-2
Constants · 7-3
CONT · 4-2
Continuation lines · 5-2
Control-C · 4-6, 5-5
Control-variables · 10-3
Converting arguments to binary · E-3
COS(X) · 11-4
Creating interactive programs · 3-2

D

DATA · 2-1, 10-18
Data formats · 1-3, 6-1
  Array structures · 1-3
  Binary variables · 1-3, 15-5
  Floating point variables · 1-3, 15-5
  Simple variables · 1-3
  String variables · 1-3, 15-5
  Unformatted variables · 1-3, 15-4
Data structures · 9-1
Data types · 8-3
DATN(X,Y) · 11-4
DELETE · 4-3
Device specification · 5-4
Devn: · 2
DIM · 6-4
Dimensioned arrays · 6-4, 10-2
Direct access files · 16-4

Direct statements · 10-1
Disk files · 1-3
Duplicate line numbers · 4-4

E

Editing lines in interactive mode · 3-3
Editing mask · 13-1
Efficiency · C-1
ELSE · 10-8
End-of-file · 16-10
EQV · 7-1, 7-2
ERR(X) · 18-2
Error trapping · 18-1
  Control-C · 18-3
Even addresses · 15-11
Exclamation symbol (!) · 2-1
Exiting interactive mode · 3-7
EXPAND · 2-4
Expanded mode · 2-4
Expanded TAB · 13-1
Expression list · 10-15
Expressions · 7-3, 10-15, 11-1, 11-4
  Functions · 10-15
  Numeric constants · 10-15
  Numeric variables · 10-15
  Operator symbols · 10-15
  String · 10-15
  String variables · 10-15
  With arguments · 10-15
Extensions
  .BAS · 3-4, 5-2
  .DAT · 16-15
  .LIT · 5-2
  .RUN · 3-4, 5-5

F

File input/output statements · 16-2
File locking · 16-7
File specifications · 5-4, 2
File statements
  ALLOCATE · 16-9
  CLOSE · 16-10
  FILEBASE · 16-10
  INPUT · 16-11
  INPUT LINE · 16-12
  KILL · 16-13
  LOOKUP · 16-13
  PRINT · 16-16
  READ · 16-16
  Sample program · 16-19
  UNLOKR · 16-17
  WRITE · 16-18
File-channel numbers · 16-2, 16-10
Floating point instructions · 6-2
Floating point numbers · 9-1
Floating point variables · 6-1, 9-3, 15-4
  Arrays · 15-4
  Format · 14-1
Format of argument lists · E-2
Formatted output · 13-1
Formatting symbols
  ! · 13-5
  " · 13-4
  # · 13-3
  $$ · 13-6
  ** · 13-7
  , · 13-7
  . (period) · 13-5
  ^^^^ · 13-8
  Z · 13-7
FRE(X) · 12-2
Free memory usage · E-3
Functions · 11-1
  Control · 11-4
  Numeric · 11-1
  String · 11-4
  Trigonometric · 11-4

G

GETMEM monitor call · E-3
GOSUB · 1-4
GOTO · 1-4
Graphics conventions · 1-5

H

Higher-level languages · 1-1

I

I/O ports · 12-1, 12-2
Increasing execution speed · C-1
INDEXED mode · 16-15, 20-2, 20-3
Indexed sequential files · 20-1
INDEXED'EXCLUSIVE mode · 16-15, 20-2, 20-3
INIT monitor call · E-3
INPUT · 2-5, 16-2, 16-10
INPUT mode · 16-15
INT(X) · 9-3
Integer constants · 9-3
Integer truncation · 9-3
Integer variables · 6-2, 9-3
Interactive BASIC · 1-2, 1-4, 10-1
Interrupting programs · 4-3, 5-5
ISAM
  Closing files · 20-6
  ERF(X) · 20-7
  Error codes · 20-7
  Error trapping · 20-7
  File locking · 20-5

File structure · 20-2
LOCK and UNLOKR · 20-6
READ and WRITE · 20-5
READL and WRITEL · 20-5
ISAM file keys · 20-2
Relative · 20-2
Symbolic · 20-2
ISMBLD · 20-2

L

Labels · 1-4, 2-3, 5-1
LCS(A$) · 2-5
Leading blanks · 10-15
Left-relative position · 8-2
Library Searching · 2-5
Line editing · 3-3
Line numbers · 3-3, 3-6, 5-1
Linefeed key · 4-5
List · 3-4
LOAD · 4-4
Loading files into system memory · 1-3
Locating open subroutine files · E-3
Locking files · 16-7
Logical operators · 9-2
AND · 7-2
NOT · 7-2
OR · 7-2
XOR · 7-2
Logical perators
EQV · 7-2
LOOKUP · 15-5
Loop · 10-3
Lower case characters · 2-5, 6-1

M

Making programs more efficient · C-1
Managing Memory · C-1
MAP · 9-1, 9-2
Mapped variables · 6-1
Mathematical expressions · 7-1
Mathematical operators · 7-1
AND · 7-1
MAX · 7-1
MIN · 7-1
NOT · 7-1
OR · 7-1
XOR · 7-1
Mathematical operators: · 7-1
MAX · 7-1
Maximum line length · 2-2
Memory
Allocation · 2-4
Efficiency · C-1
Memory mapping · 1-2
MIN · 7-1
Mode independence · 8-3, 11-1, 11-4
Monitor command level · 1-4, 3-2
Multiple statement lines · 2-1, 3-7

N

Name terminator · 6-1
New · 3-2
NEXT · 10-3
NOEXPAND · 2-4
NOT · 7-1, 7-2
Null strings · 9-1, 10-10, 11-7, 16-10
Numeric arguments · 11-1
Numeric conversion · 7-3, G-1
Numeric variables
Binary · 6-2, 15-4
Floating point · 6-1, 15-4
Integer · 6-2

O

Object code · 1-3
Object programs · 1-3, 3-4, 3-5, 3-6, 5-2
ON - CALL · 10-14
OPEN · 15-5, 16-6, 16-14, 16-16, 16-18, 20-2
Opening an ISAM file · 20-3
Operator · 7-4
OR · 7-1, 7-2
OUTPUT mode · 16-15

P

Percent sign (%) · 9-2
PHDR macro · E-3
Physical blocks · 16-5
Physical memory · 12-1
PPN · 1
Precision · 6-2
PRINT · 7-4, 13-1, 13-9, 14-2, 16-11
PRINT USING · 10-20, 13-1, 16-16
Program
Compilation · 3-6
Debugging · 3-1, 4-5
Execution · 3-6
Indentation · 5-1
Run · 10-2, 10-4, 10-20
Statements · 10-1
Program headers, subroutines · E-3
Prompt · 3-2

R

Random files · 16-5
RANDOM mode · 16-15

Random numbers · 10-18, 11-3
RANDOM'FORCED mode · 16-7, 16-15
Range checking · 15-5
READ · 16-2, 16-6, 16-15, 20-3, 20-4
Readability of source programs · C-2
READL · 16-2
READY · 3-2
Record size · 16-5
Record-number-variable · 16-15
Re-entrant ode · 1-3
Register Parameters · E-1
Relative keys · 20-2
REM · 2-2
Reserved words · 2-5, 15-11, B-1
RESTORE · 10-18
RESUME · 10-13, 18-3
RETURN · 10-5
Right-relative position · 8-2
RND(X) · 10-18
RTN instruction · E-1
RUN · 1-4, 3-6, 10-20, 16-9
Run-time package · 1-3, 1-4, 5-2

S

SAVE · 3-3
Saving a .RUN file · 3-4
Scaling factor · 14-2
Searching for a Library · 2-5
SIGNIFICANCE · 6-2
SIN(X) · 11-4
Single-step · 3-1
Source programs · 3-4, 10-1
  Readability · C-2
Speed · C-1
Statement modifiers · 10-1
STEP · 10-3
STOP · 4-6
STR(X) · 7-4
String arguments · 11-1
String conversion · 7-3
String formats · 6-3
String literals · 2-5, 6-3
String size · 6-3
String variables · 6-1, 15-4
  Arrays · 10-3
STRSIZ · 10-3
Subroutine library · E-3
Subroutine loading · 19-2
Subroutine program headers · E-3
Subroutines · 10-3
  Linking · 1-2
Subscripts · 6-2, 8-3, 10-2
Substrings · 8-1, 8-2, 11-5
  Modifiers · 6-3, 8-1, 8-3
  Overflow · 8-2
  Truncation · 8-2, 15-5
Syntax · 2-5, 12-1
Syntax errors · 3-3
SYSLIB.LIB · E-3
System functions · 12-1
System subroutine library · E-3

T

TAB · 13-9
TAN(X) · 11-4
TCRT codes · 13-9
THEN · 10-8
Timesharing · 1-3
Trailing blanks · 10-15
Trapping errors with Control-C · 18-3
Trigonometric functions
  ACS(X) · 11-4
  ASN(X) · 11-4
  ATN(X) · 11-4
  COS(X) · 11-4
  DATN(X,Y) · 11-4
  SIN(X) · 11-4
  TAN(X) · 11-4
Type codes · 6-1

U

UCS(A$) · 2-5
Unformatted variables · 6-3, 15-4
UNLOKR · 16-2, 20-6
Upper case characters · 2-5, 6-1
Use of free memory · E-3
User impure area · E-2
User memory partition · 1-2

V

VAL(A$) · 7-4
Variables · 2-5, 7-3, 9-1
  Length · 1-4
  Names · 1-3, 6-1
  Trees · 15-11

W

Word boundaries · 15-11
WORD(X) · 12-1
WRITE · 14-2, 16-6, 16-15, 20-3, 20-4, 20-5
WRITEL · 16-2, 20-5

X

XCALL · 19-1
XOR · 7-1, 7-2